
XFaceMaker/WF

XFaceMaker Version 4.0

XDrawMaker Version 4.0

XDraw Library Version 4.0

Tutorial

Nova Software Labs

© Copyright Nova Software Labs S.A.R.L. 1993 -2002.

All rights reserved. No part of this document may be stored in a retrieval system, transmitted, or otherwise copied or reproduced by any electronic, mechanical, photographic, or recording process without express written permission from Nova Software Labs S.A.R.L.

Published by:



Nova Software Labs

8, rue de Hanovre

75002 Paris, France

Tel: +33 1 58 18 61 61

Fax: +33 1 58 18 61 60

email: sales@NovaSoftwareLabs.com or sales@nsl.fr.

Web: <http://www.NovaSoftwareLabs.com> or <http://www.nsl.fr>.

Every precaution has been taken to ensure the accuracy of this document. Nova Software Labs S.A.R.L. assumes no responsibility for errors or omissions, or for damages resulting from the use of this document.

XFaceMaker, XDrawMaker, XnslDrw are registered trademarks of Nova Software Labs S.A.R.L. X Window System is a trademark of the X Consortium. UNIX is a registered trademark of Novell. OSF/Motif and Motif are trademarks of The Open Group, Inc. All trademarks are the property of their respective owners.

Document Number: XDMLRefMan v. 7.1.1.

Document date: November 29, 2002

Acknowledgments

The following people have contributed to the development of the XFaceMaker suite:

Alain Bastard, Genevieve Beaujard, Nicole Brion, Alan Char, Eric Durocher, Marc Durocher, France Geze, Eric Herbaut, Elizabeth Huffer, Ion Filotti, Jérôme Maloberti, Sara Sautin, Eric Touchard, Philippe Volle.

Many thanks to all of them and to several others.

Table of Contents

Preface	7
1. XFaceMaker/WF Components	1
1.1. The NSL Draw Library	2
1.1.1. The Primitive Graphic Gadgets Classes	2
1.1.2. The XnslDraw Manager Class	8
1.1.3. The XnslDrawTextWidget Class	11
1.1.4. XnslDraw Library Functions	11
1.2. XDrawMaker	13
1.2.1. Creating Passive Drawings	14
1.2.2. Animating Drawings	15
1.3. XFaceMaker	17
1.3.1. Generating Code for Classes	19
1.3.2. Integration with XDrawMaker	19
2. Classes and XFaceMaker/WF	23
2.1. X Classes Designed with XFaceMaker/WF	23
2.1.1. Features of X Classes	23
2.1.2. Using X Classes	24
2.2. C++ Classes	25
2.2.1. Features of C++ Classes	25
2.2.2. Using C++ Classes	25
3. Some Uses of XFaceMaker/WF	27
3.1. Dials	28
3.2. Graphs	31
3.3. Animated Diagrams	33
3.4. Animated Charts	35
3.5. Interactive Editors	36
4. A Guided Tour	37
4.1. Draw the Widget in XDrawMaker	38
4.2. Animate the Drawing	42
4.3. Test and save the animated drawing	46

4.4. Import drawings in XFaceMaker	48
4.5. Define new resources for the widget class.	52
4.5.1. Define a rulerPositionColor resource	52
4.5.2. Define a rulerMin resource.	54
4.5.3. Define a rulerMax resource	57
4.5.4. Define a rulerValue resource	57
4.5.5. Define a valueChangedCallback resource	59
4.6. Use the new class	61
4.6.1. Build the interface	62
4.6.2. Define callback resources for the minimum value widget	63
4.6.3. Define callback resources for the maximum value widget.	64
4.6.4. Define callback resources for the current value widget	64
4.6.5. Define callback resources for the Ruler widget	65
4.6.6. Define the PushButton's Callback	66
4.7. Test the interface in XFaceMaker	67
4.7.1. Add the rulerFont resource	67
4.8. Generate the C-code and compile it.	68
4.8.1. Generate the application files	68
4.8.2. Modify the functions file interfaceApp.c	69
4.8.3. Generate the executables	69
4.8.4. Choose the correct executable file	70
4.9. XFaceMaker and the Ruler Extension	71
4.9.1. Make the Class Include and Source Files.	71
4.9.2. Build the Ruler Widget Library	72
4.9.3. Define the Ruler.face Extension File	73
4.9.4. Use the Ruler Extension in XFaceMaker	76
4.10. Source listings	78
4.10.1. Draw file : ruler.draw	78
4.10.2. Ruler Xt class file : Ruler.fm	83
4.10.3. Interface example --- Fm file : interface.fm	88
4.10.4. Interface example --- Include file : interfaceApp.h	94
4.10.5. Interface example --- Functions file : interfaceApp.c.	94
4.10.6. Interface example --- Main file : interfaceMain.c	95
4.10.7. Interface example --- Makefile : interfaceMake.	99
4.10.8. Extend XFaceMaker - Main file: r_xfmXfmMain.c.	104

5. Advanced examples	107
5.1. Run Time Initialization	108
5.1.1. Modify the drawing in XDrawMaker	108
5.1.2. Modify the Ruler class	111
5.1.3. Modify the interface	119
5.1.4. Declare the rulerData Structure	123
5.1.5. Source listings	124
5.1.5.1. Draw file : ruler.draw	125
5.1.5.2. Ruler Xt class file : Ruler.fm	129
5.1.5.3. XFM Extension - Ruler.face file	137
5.1.5.4. Interface example --- Fm file : interface.fm	138
5.2. The Ruler widget class : FM only version	147
5.2.1. Create the Ruler.fm file in XDrawMaker	147
5.2.2. Modify the Ruler.fm file in XFaceMaker	148
5.2.2.1. Modify the RulerSelectZone gadget	148
5.2.2.2. Modify the RulerLabel gadget	150
5.2.2.3. Modify the draw_edit widget	150
5.2.2.4. Define a rulerPositionColor resource	150
5.2.2.1. Define an Initialize resource	152
5.2.2.2. Define a rulerMin resource	156
5.2.2.3. Define a rulerMax resource	156
5.2.2.4. Define a rulerValue resource	157
5.2.2.5. Define a rulerTicks resource	158
5.2.2.6. Define a rulerSubTicks resource	158
5.2.2.7. Define a rulerFormat resource	159
5.2.2.8. Define a valueChangedCallback resource	159
5.2.2.9. Define a rulerFont resource	159
5.2.3. Test the Ruler Class within XFaceMaker	160
5.2.4. Make the Class Library and Includes	160
5.2.4.1. Generate the Ruler class C file	160
5.2.4.2. Make the Ruler class library	161
5.2.5. Make an Extended XFaceMaker	161
5.2.6. Test the Extended XFaceMaker	162
5.2.7. Source listings	163
5.2.7.1. Ruler Xt class file : Ruler.fm	163
5.2.7.2. Ruler.face file	173

Preface

XFaceMaker/WF is a tool suite for designing animated graphics, and X or C++ widget classes interactively. These animated graphics and widget classes can be combined seamlessly with the standard Motif widgets or with third-party specialized widgets and can be used on any workstation that supports the X Window System. The standard Motif widget set cannot completely satisfy the almost infinite variety of requirements of modern graphic user interfaces. Multimedia, industrial control, data visualization, even simple business applications now require the display of sophisticated animated graphics.

XFaceMaker/WF provides a rapid and original solution to this problem. It lets you design animated graphics interactively, generate the widget class code from the animated drawing automatically, and then incorporate instances of the class in the user interface. Prior to XFaceMaker/WF, designing widget classes was so complex, the investment in the technical expertise so high, that many software development projects simply had to give up the possibility of designing genuine widget classes. The productivity gain brought by XFaceMaker/WF is huge when compared to the traditional edit-compile-debug cycle—several hundred lines of code corresponding to a class are generated in a few minutes.

XFaceMaker/WF is a large piece of software with around one million lines of source code. At the base is the NSL Draw Library—a comprehensive set of graphic primitives implemented as X widget classes. There are sixteen basic widget classes with several hundred resources, and over two hundred functions to manipulate resources, handle widget structures and animate objects. Implementing the graphic primitives as X widget classes provides numerous technical advantages; in particular, their seamless use in combination with the Motif widget set. XFaceMaker/WF makes it possible to mix Motif and the NSL Draw widgets in the same container and to create new classes from the combination.

Because of its flexibility, XFaceMaker/WF has numerous other applications in addition to the design of widget classes. For example, XDrawMaker, the tool for creating animated drawings, is also an excellent standalone tool for creating passive and animated drawings. XFaceMaker/WF is also useful for creating specialized graphic applications such as interactive graphic editors and /or multimedia applications.

About this manual

XFaceMaker/WF comprises three main components:

- The NSL Draw Library – a set of graphic components.

-
- XDrawMaker – an interactive tool for creating animated drawings.
 - XFaceMaker – a tool for designing graphical user interfaces.

This manual contains an overview of the product and a tutorial. Complete descriptions of the NSL Draw Library, of XDrawMaker, and of XFaceMaker are provided in separate manuals.

Related Documents

NSL Documents

- *The XFaceMaker/WF Tutorial* (this manual)
- *The NSL Draw Library*
- *The XFaceMaker Extensions*
- *The XDrawMaker User's Guide*
- *The XFaceMaker User's Guide*
- *The XFaceMaker Reference Manual*
- *The Wx User's Guide*
- *The Installation Guide for NSL Products*

Other Literature

The following manuals provide more information on the X Window System and the OSF/Motif toolkit. Items marked with an asterisk (*) are especially useful. Refer to the most recent edition that corresponds to your version of X and Motif.

From the **X Window System Series** by O'Reilly and Associates:

Adrian Nye, ed. *Volume 0: X Protocol Reference Manual*. O'Reilly and Associates, 1989

Adrian Nye. *Volume 1: Xlib Programming Manual*. O'Reilly and Associates, 1993

Adrian Nye. *Volume 2: Xlib Reference Manual*. O'Reilly and Associates, 1992

T. O'Reilly, V. Quercia. *Volume 3: X Window System User's Guide (OSF/Motif Edition)*, O'Reilly and Associates, 1993

Adrian Nye, Tim O'Reilly. *Volume 4: X Toolkit Intrinsics Programming Manual (OSF/Motif Edition)*. O'Reilly and Associates, 1992

Adrian Nye, Tim O'Reilly. *Volume 5: X Toolkit Intrinsics Reference Manual (OSF/Motif Edition)*. O'Reilly and Associates, 1993

* Dan Heller. *Volume 6: Motif Programming Manual For OSF/Motif*. O'Reilly and Associates, 1991

Paula M. Fergusson, D. Brennan. *Volume 6B: Motif Reference Manual*. O'Reilly and Associates, 1994

Tim O'Reilly. *The X Window System in a Nutshell*. O'Reilly and Associates, 1992

From the **Open Software Foundation**:

Open Software Foundation. *OSF/Motif User's Guide*, Prentice Hall, Englewood Cliffs, New Jersey, 1993

* Open Software Foundation, *OSF/Motif Style Guide*, Prentice Hall, Englewood Cliffs, NJ. 1993

* Open Software Foundation, *OSF/Motif Programmer's Reference*, Prentice Hall, Englewood Cliffs, NJ. 1993

Open Software Foundation, *OSF/Motif Programmer's Guide*, Prentice Hall, Englewood Cliffs, NJ. 1993

C++ Documentation

There is a wide range of C++ documentation available: The following are two basic manuals from the originators of C++.

B. Stroustrup: *The C++ Programming Language*. Addison-Wesley, Reading, Massachusetts. 1992.

T.L. Hansen: *The C++ Answer Book*. Addison-Wesley, Reading, Massachusetts. 1990.

Terminology, typesetting conventions

Keyboard Conventions

Key combinations are expressed as follows:

Format	Example	Meaning
Key1--Key2	Ctrl--A	Hold down the C ontrol key as you press, then release key A . The S hift key is not used.
Key1 Key2	Esc a	Press and release the E scape key, then press and release key a .
	Esc A	Press and release the E scape key, then press and release key a while pressing the S hift key.

Notice how in the case of the **ESC** modifier the two keys are pressed in succession, whereas in the case of the **Ctrl** modifier the key is pressed *while* the **Ctrl** key is held down.

Typesetting Conventions

The following typesetting conventions are used in this manual:

Type style	Used for
typewriter	Anything you should type as it appears, code, and key combinations.
<i>italic</i>	Mouse buttons and emphasis.
<i>bold slant</i>	Menu items and editing commands.

Technical Support

Technical Support is available from NSL in France :

- Monday to Friday (except for French national holidays).
- 9.00 am 12.30 pm – 2.00 pm to 6.00 pm (MET). (5 pm on Friday).
- From French and English speaking support staff.
- By phone (+33) [01] 44 08 70 80 (ask for the Technical Support Department).
- By electronic mail addressed to support@nsl.fr
- By FAX (+33) [01] 44 08 70 81 addressed to Technical Support including your return FAX number, and a telephone number where you can be reached.

Using the Mouse

We assume you are using a three button mouse. If not, you need to know which combinations of keyboard and mouse actions correspond to *Select*, *Menu*, and *Custom*.

Mouse buttons

Mouse buttons respond differently in Build mode and in Try mode. In Build mode the mouse buttons are used as follows:

Button1: *Select*---Used to select/deselect widgets on the desktop or in the Widget List or Tree, to select menu items, open a menu, or drag.

Button 2: *Menu*---Used to select widgets on the desktop, and popup the Resources or the Resource Editor boxes.

Button3: *Custom*---Used to select the *parent* of the *current* widget.

In Try mode most button actions are determined by the widget resources and callbacks defined for the particular interface. By default, PopupMenus are popped up using the Custom button.

Mouse actions

This manual uses the following terms for mouse actions:

Press Hold down a mouse button.

Click and double-click Quickly press and release once (click) or twice (double-click) the *Select* mouse button without moving the mouse. Clicking selects. Double-clicking is for editing actions.

Shift-click Hold down the Shift key, and click the left mouse button.

Drag Move the mouse while pressing the *Select* mouse button.

Rubber band A rectangular outline displayed when the cursor is dragged. One corner is fixed at the point where the button was pressed and the opposite corner follows the pointer. Releasing the button creates a rectangle that defines either the size of a widget or a selection area.

Outline box This is the outline of a widget displayed as the widget is moved so that it can be placed correctly.

What You Need to Run XFaceMaker/WF

To run XFaceMaker/WF you need a standard X/Motif environment, as is available from most Unix workstation manufacturers (e.g. Sun Microsystems, Hewlett-Packard, IBM, Silicon Graphics...) or from Unix providers on PC's (e.g. SCO, Unixware, or Solaris). For an up-to-date list of ports, check with the NSL Sales or Technical Support departments.

To run any application using the X Window System you need:

- A Unix host computer. The Unix host will run all the applications. The application makes calls to the *Xlib* library for displaying, and for processing keyboard and mouse events. Other libraries may be needed as well; e.g. the *Xi* (X Intrinsics), or the *Xm* (Motif) libraries. The host must be set up to communicate with other machines on a network (TCP/IP). The host system must include development software: compiler, linker, etc.
- A window manager. Although XFaceMaker can run with different window managers, it is guaranteed to work with the OSF/Motif window manager *mwm*.
- A graphic display consisting of a monitor, a keyboard and a mouse.

There are two main types of graphic displays on which you can run the X Window System (and thus XFaceMaker):

- *Unix workstations*. These are Unix computers with a graphic screen. They run the X server as a Unix process.
- *X terminals*. X terminals run only the X server as a stand-alone program. The application itself runs on the remote Unix host.

Motif based applications require a screen resolution of at least 1024 x 768 pixels. If you plan to use color, make sure you have 256 colors. It is unreasonable to attempt to create a Motif-based application with only 16 colors.

CHAPTER 1: XFaceMaker/WF Components

XFaceMaker/WF comprises the following tools:

- **The NSL Draw Library** – The NSL Draw Library contains the basic graphic objects out of which animated drawings will be created in XDrawMaker.
- **XDrawMaker** – This is a general-purpose drawing tool which lets you design a static drawing interactively and also to design and test the animation associated with it. The animation is specified in the form of scripts in the FACE language. XDrawMaker has a FACE interpreter, thus allowing the immediate testing of animation scripts within the tool.
- **XFaceMaker** – NSL's user interface management system. XFaceMaker is used in the design cycle for importing an animated drawing created with XDrawMaker, where it can be either used to build a user interface or to define new widget classes.

XFaceMaker can create user interfaces using Motif or other X widgets. It allows you to design the layout and the behavior or animation of the interface. As in XDrawMaker, behavior is specified in the form of FACE scripts. XFaceMaker has a FACE interpreter and debugger.

XFaceMaker/WF can generate C or C++ code that is independent of the FACE interpreter.

1.1 The NSL Draw Library

The NSL Draw Library comprises:

- A set of primitive gadget classes.
- The XnslDraw widget class.
- The XnslDrawTextWidget widget class.
- The XnslDraw Library functions.

The complete set is documented in the *XDraw Library* manual.

1.1.1 The Primitive Graphic Gadgets Classes

The following figure gives the hierarchy of the Draw Library Object Set. The main class is the **XnslDrawObject** class. This class is the base class for all Draw widgets. The **XnslDrawRect** class is the base class for rectangular widgets. The **XnslDrawPoly** class is the base class for polygonal widgets. Only widgets subclass of this class can be used as links (see below). Links can have arrows at their extremities.

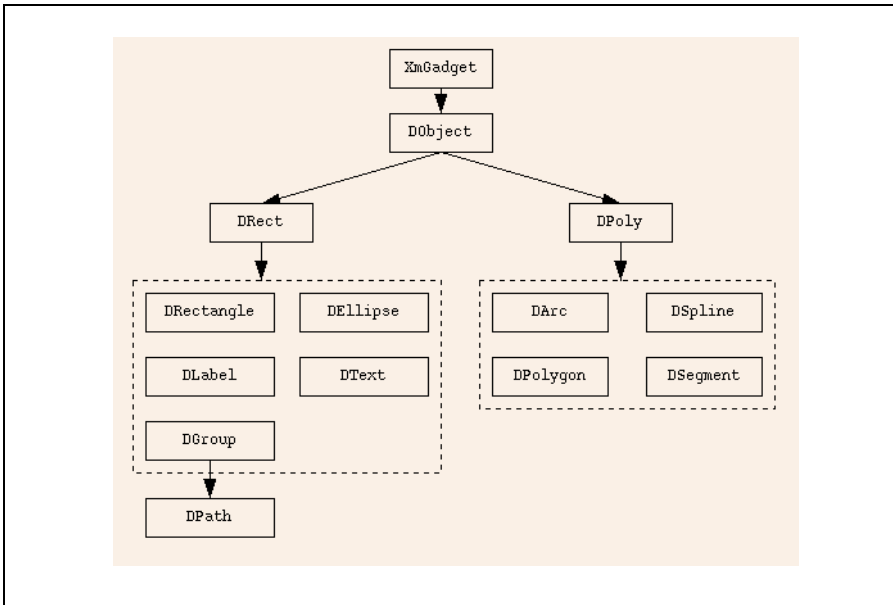


Figure 1.1: The Draw Gadget Classes

A. The Widget Set

Standard widgets are usually defined by providing the position and dimension of the widget. Unlike standard widgets, the Draw widgets are defined by directly giving a set of coordinates. These coordinates are a set of floating point numbers which define the shape of the widget. The core resources **XmNx**, **XmNy**, **XmNwidth** and **XmNheight** are computed from these points.

Most of the draw widgets need a list of points: for example, a line needs a start and an end point. These points can be given in different units (inches, millimeters...).

m **Line**: defined by two points.

m **Rectangle**: defined by a top left corner position, and a width, height dimension.

m **Polygon**: defined by a set of points. The points are connected to form a Polyline.

m **Spline**: defined by a set of control points.

m **Segment**: defined by a set of points, but the segments are drawn by disjointed pairs.

m **Ellipse**: defined by a top left corner position, a width, a height, a start and an end angle.

m **Arc**: defined by three points to form a quarter of an ellipse.

m **Label**: defined by a top left corner position, a width, a height and a static string.

m **Text**: The text gadget is editable. It handles many display features such as underlining, multiple fonts, background and foreground colors on text segments. The library also includes a text widget with similar display features.

m **Group**: defined by a list of **Draw** widgets. When grouped, a set of widgets becomes a new widget.

m **Path**: defined by a list of polygonal **Draw** widgets. As in groups, the set of widgets becomes a new widget, but the widgets are joined to form a continuous path.

B. Attributes

Geometry

All **Draw** widgets can be rotated, scaled and translated. Two kinds of scale are provided: a prescale and a postscale. The prescale is applied before the rotation and the postscale is applied after the rotation. A center is also part of the attributes. Rotation and scaling are done with respect to this center.

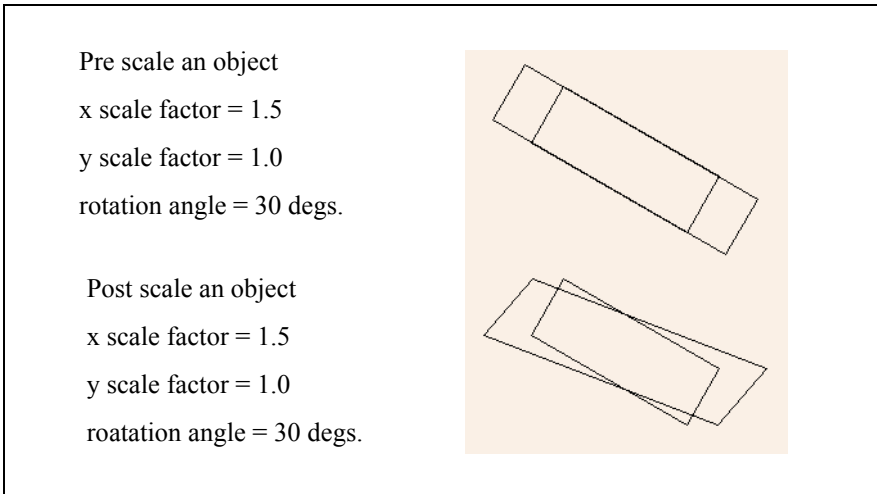


Figure 1.2: Pre Scale and Post Scale

Graphics:

Draw widgets have graphic attributes. This includes standard features such as foreground color, background color, line width, etc. Some of these attributes are directly taken from an X graphics context:

- m foreground
- m background
- m function
- m lineWidth
- m font
- m stipple
- m dashes
- m xTsOrigin
- m yTsOrigin

Additional attributes are available:

m closeMode: Some widgets can be closed. The modes available are:

- Open: The widget is not closed.
- Center Closed: The widget is closed through the center.
- Object Closed: The close mode is widget dependent.

m fillMode: Objects can be filled or not. They also can be partially filled.

m fillDir: When an object is partially filled, this attribute is used to determine the direction from which fill starts. An widget can be filled from top, bottom, left, or right.

m pctFill: This attribute is used to partially fill a widget. It specifies the percentage of the widget's surface that should be filled.

m opaque: A widget can be transparent or not. When a widget is opaque, its enclosing rectangle is filled with the background color.

m background_pixmap: The background of a widget can also be filled with a color pixmap.

C. Links

Each polygonal widget can be used as a link connecting two slots. The intended use is for the link to be redrawn and to follow the slots it is connected to. A widget becomes a link if link related resources have the right values. These resources are:

- m A source widget.
- m A destination widget.
- m A source slot.
- m A destination slot.

When these resources are assigned to a widget, the widget becomes a link. Moving or resizing a source or destination widget updates the link. The link can be automatically updated or callback updated. Automatic updating is done by superimposing the first point of the widget with the source slot and its last point with the destination slot. Callback updating is done by calling a callback. In this case, the application can compute the points of the link.

D. Scripts

Objects can be activated through two resources:

- m **XnslNdrawActivateCallback**: This callback resource is triggered when the user clicks in the overall rectangle surrounding the widget.
- m **XnslNdrawActivateCallbackList**: This resource is a list of **FACE** scripts. Each script has an event condition and a location condition. The form of such a script is:

```
on <Event> [ and <location> ]  
<Script body>  
end
```

The script is launched if the user event matches the script event and if the event occurred in the specified location.

The event condition can be the name of an event (e.g. `Button1Down`) or can be a translation specification (e.g. “<KeyPress> a”). The quotation marks must be included when specifying a translation.

E. Output Format

The XnslDraw widget and the Draw gadgets can be saved. Two formats are provided: a Draw format and an Fm format. The Fm format is readable by **XFaceMaker** and **XFaceMaker** can generate C code from this format.

The Draw format is:

```
DrawWidget    ::=  EMPTY
                |  BEGINBLOCK <ResourceList> ENDBLOCK
WidgetList    ::=  EMPTY
                |  <WidgetList> <WidgetRec>
WidgetRec     ::=  STRING STRING BEGINBLOCK <ResourceList>
                  ENDBLOCK
ResourceList  ::=  EMPTY
                |  <ResourceList> STRING IS STRING SEP
                |  <ResourceList> <WidgetRec>
                |  <ResourceList> STRING SEP
```

1.1.2 The XnsIDraw Manager Class

The **XnsIDraw** widget is a container widget (a subclass of the Motif Manager widget) able to contain any widget. It provides interactive editing capabilities. The **XnsIDraw** widget can operate in two modes:

Edit Mode: Selection is enabled; pointer events are not dispatched to children. Once a widget is created with an **XnsIDraw** widget as parent, the user can select, move and resize this widget. Selection is indicated by handles drawn on the selected widgets.

Run Mode: Events are dispatched to children. Children have their normal behavior.

The XnsIDraw widget has the following editing features:

Selection:

A feature of the **XnsIDraw** widget is the capability of selecting and modifying children. The user can select a child by clicking on it or by dragging a rubber band, to enclose the widgets which are to be selected. The selection is indicated by handles on the selected widgets. For non-Draw widgets, the handles are placed at the corners of the widget. For Draw widgets, the handle locations are widget dependent. The selection may contain a special widget: the edit widget. This widget is the last clicked on widget (Figure 1.3, “The Selection Mechanism,” on page 9)

Callbacks are called before and after a widget is selected. Before a selection, a **SelectVerifyCallback** is called. The application can then allow the widget to be selected or not. After a widget is selected, a **SelectCallback** is called to notify the application. A special callback is provided for the Edit Object. It is activated when an widget becomes the edit widget.

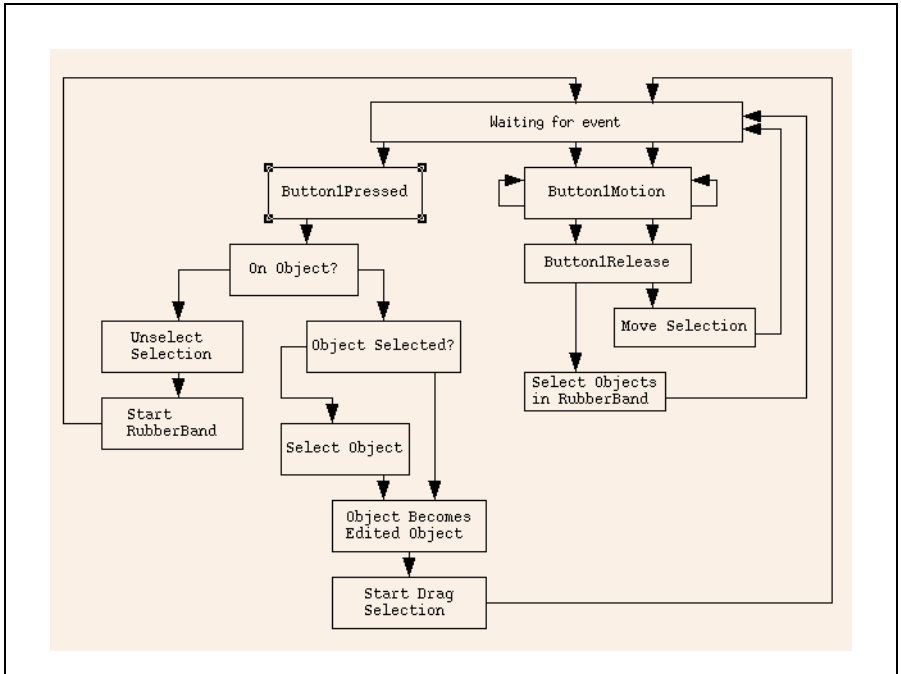


Figure 1.3: The Selection Mechanism

Grid:

The **XnslDraw** widget includes a snap-on-grid mechanism. The grid is an array of points to which widget points are constrained if the grid is active. The grid can be shown on the screen or not shown. User input can be snapped to the grid or not.

Collect Reactors:

The **XnslDraw** widget has a set of resources which provide a general mechanism for interactive user input. A set of functions associated to these resources handles the acquisition of the coordinates of a point, the drawing of the ghost images of rectangles or line segments which follow mouse motion, widget rotation, etc. When the action is complete, a callback is called to notify the application.

Stacking reorder:

The **XnslDraw** widget provides functions to modify the stacking order of its children. These function include:

- m XnslDrawCirculateUp
- m XnslDrawCirculateDown
- m XnslDrawSendToBack
- m XnslDrawBringToFront

Functions are also available to modify the stacking order of children inside a group. Changing the stacking order actually changes the creation order of the children.

Zoom

A zoom feature is available with the **XnslDraw** widget. A function is provided to zoom a part of the drawing. This function computes a scale factor to be applied to all widgets within the zoom area. The scale factor value is such that the zoomed area occupies as much of the viewing window as possible. The **XnslDraw** widget memorizes each zoom operation. Unzooming returns the drawing to the previous zooming operation.

Double Buffer

Double buffering is useful to avoid the flashing effects that may occur during animation and/or window redisplay. A double buffer is a non visible pixmap in which the drawings are done. If a part of the visible window is to be updated, this part is copied from the buffer to the window.

Slots

The **XnslDraw** widget can add slots to its children. Slots are rectangular regions which are used to plug in a link. These slots can be displayed as a rectangle or as a pixmap. Each slot can have a value attached to it. Slot placement is automatic. Slot related resources are constraint resources that the widget applies to its children.

1.1.3 The XnslDrawTextWidget Class

The XnslDrawTextWidget has much the same resources as the XnslDrawText gadget. When it is placed in an XmScrolledWindow, XnslDrawTextWidget handles its own scrolling, the scrollingPolicy resource being set to application_defined. Thus, when you need to display/edit text with variable fonts and/or hypertext links, you can use the XnslDrawTextWidget widget if you need scrolling, and the XnslDrawText gadget otherwise.

1.1.4 XnslDraw Library Functions

A comprehensive library of convenience functions is provided by the Draw Library. These functions can be called in FACE scripts or can be used in programs that use the Graph Library.

The functions provided can be classified as follows:

- Object Management - This group includes functions to access an object's identifier or name, to delete, resize, duplicate, show or hide objects. Other functions let you read or set the data points of an object.
- Image - These functions let you set or get the index of the pixmap of an XnslImage object.
- Save and Load - With these functions, you can load or save the contents of an XnslDraw widget using either the fm or the draw format. For a description of these formats, see the section on XDrawMaker's main features below.
- Object creation – This group includes creation functions for specific classes of objects and a generic draw object creation function in which you can specify the type of object as an argument.
- Display Management – Functions in this group control the re-display of the XnslDraw widget.
- Geometric Transformations – Functions in this group apply geometric transformations to objects.
- Graphic Transformations – Functions in this group help set and get the values of the graphic resources of objects.
- XnslDrawText and XnslDrawTextWidget Functions – These sets of functions lets you access and/or modify the resources specific to the XnslDrawText object and the XnslDrawTextWidget widget. Functions in these groups can be further subdivided into:
 - m Mark Functions – You can change text marks at any position. Text marks spec-

ify the color and font used to draw a text segment.

- m Alignment Functions – Text can be aligned to the left, right or center and can be justified or fully justified.
- m Position functions – Functions to manipulate the insertion position.
- m Killer functions – Functions to clear, erase or paste substrings.
- m Insertion Functions – Functions to insert text (Ascii or Dtif format) into the widget.
- m Input/Output Functions – Functions to read text from a file into the widget or to write text from the widget into a file.
- Group functions – Functions for adding an object to a group, for deleting an object from a group, and for disassembling a group.
- Arrow Functions – Functions to control the arrow resources of some of the graphic primitive gadgets such as the segment or the line.
- Ellipse Functions – functions for controlling resources specific to the ellipse, e.g. start and end angle of the ellipse.
- Event Functions – These return the location of a button event, the string entered for a key event, the angle formed by a button event and reference point. These functions are useful for interpreting mouse movements as input sources.
- Script Functions – Functions for triggering the execution of a script and enabling/disabling them.
- Miscellaneous Functions – Conversion from radians to degrees and conversely, trigonometric functions, generation of random numbers, timing functions.

1.2 XDrawMaker

XDrawMaker is a program for creating passive or animated drawings. Drawings created with XDrawMaker can be used in the following ways:

As passive drawings. As a tool for the creation of passive drawings, XDrawMaker is roughly equivalent to the common drawing tools available on PCs. XDrawMaker supports output in Postscript format. Passive drawings can thus be printed on PostScript printers (with variable resolution, since PostScript is a resolution-independent description of drawings) or displayed by PostScript viewers such as Display Postscript or public domain viewers.

As animated drawings to be displayed in XDrawMaker. Animated drawings can be displayed in action within XDrawMaker itself.

As components for graphic user interfaces. Animated drawings can be imported in a graphic user interface created with a tool such as XFaceMaker. One can do the following:

- m Import the animated drawing in an XnsIDraw widget that is part of the user interface, or
- m Import the animated drawing in some other Motif manager widget. XFaceMaker has all the functionality needed for quickly creating simple applications that will display such stand-alone animations or complex ones that will use the animation as a part of a larger user interface.
- m Use it as the basis for widget classes (Xt or C++). Animated drawings imported into XFaceMaker can be transformed into Xt or C++ widget classes in XFaceMaker. This process may require the addition of other Motif widgets and of additional behavior in the form of scripts attached to the widgets. This is of course the first *raison d'être* for XFaceMaker/WF, although the other usages have proved almost as valuable.
- m Use it as the basis of an application that supports interactive editing of graphic components. Applications that use drawings are not usually thought of as letting the user alter interactively a drawing, animated or not. XFaceMaker/WF however, makes it possible to create such editable drawing. As a simple example, in the case of a graphing application, it is possible to create a graphing application in which the user could change the shape of the graph; for example the height of a point of a curve, or the level of a histogram. More complex interactive applications of this kind are possible, including a tool as complex as XDrawMaker which has been created this way itself.

XDrawMaker itself uses the widgets of the Draw graphic library. The graphic objects used in the design of the animated drawing are the primitive graphic gadgets of the Draw library.

1.2.1 Creating Passive Drawings

Variable drawing size. You can specify the size of your drawing by specifying a page format and the number of pages your drawing will cover, both in width and in height. Several standard page formats are supported, and custom page sizes can be defined.

Groups and Paths – A set of graphic objects can be grouped into an entity that can be treated as a single object. For example, it is easier to move, turn, or transform a complex drawing such as an airplane as a single object than to move, turn, or transform the individual pieces. Paths are complex shapes created by connecting DrawPoly objects (see the Draw library above) with lines. Paths can be used to connect slots associated with objects (see the Draw library). Groups can include other groups or paths.

Grid – To assist in the orderly placement of objects in the drawing, you can display a grid in the background of the drawing (the grid will not appear in the final drawing – it is there just as a design aid). Objects can be made to *snap* to the grid. You have control over the size of the grid's step and over how the grid is displayed. A form of grid step is also provided for angular rotation increments.

Interactive layout – The drawing is laid out interactively, very much in manner similar to that of popular PC drawing tools. Objects can be selected and moved or resized directly on the drawing or in the tree display of objects in the drawing (see below). More sophisticated geometric transformations are supported and these can also be applied through scripts, as will be described below. In fact, the provision of scripts is useful not only for the animation of drawings, but also for the quick creation of large sets of repetitive elements (it suffices to write and execute a small script that will create the repetitive structure for you).

Display of the object hierarchy – XDrawMaker provides a tree view of the structure of you drawings. All objects are identified by their name and groups and paths are shown to the left of their components. You can select objects in the tree and you can change their name directly in the tree (in addition to changing it through the object's attribute box.)

Editing graphic attributes – Graphic attributes of an object can be specified and then later edited in an attribute box. Attributes that can be changed this way include foreground and background color and background pixmap, line widths and styles, line arrow endings, the fill mode, whether the object is open or closed and whether an object is opaque (it then conceals any objects behind it) or transparent.

In addition to the interactive editing of graphic and geometric attributes, XDrawMaker also supports the construction and editing of attributes through scripts. Scripts, whose main features are described in the next section, are not useful just for expressing animation but also for the creation of drawing components and the modification of their attributes.

As an example, consider the problem of creating a ladder with a large number of rungs. One could draw the rungs individually, which is rather time-consuming (although in this case a little ingenuity can save time -- one can create two rungs, then copy and paste them, then copy and paste the four rungs obtained and so on, cutting creation time logarithmically). Instead, it might be more economical to write a little FACE script that will create the rungs.

1.2.2 Animating Drawings

Drawings can be created by attaching FACE scripts to its graphic components. Graphic objects support mouse events: inside, outside and on-border. One can attach a FACE script to each of them.

FACE is a C-like programming language enhanced with syntax for naming objects and their resources. The features of the FACE programming language are described in more detail in the XFaceMaker Reference Manual. From a FACE script one can call any of the functions described earlier. FACE supports the basic control and data structures of C, recursive function definition and the invocation of scripts.

XDrawMaker also supports the testing of animation within XDrawMaker, or within a separate dedicated process. When the animation has been triggered, graphic object editing is disabled, mouse events are received by the objects in the drawing, they trigger the scripts defined for these events. Animation scripts are often triggered by an

initiating mouse event. They can also be triggered by events caused by an input variable or a resource changing values (e.g. the input resource of a dial exceeding an alarm level may trigger an alert.)

To increase the robustness of the tool two testing modes are supported. The first takesplace in the same process as the editing process. In the second mode, a new process is spawned within which the animation takes place. Thus in the case of an unrecoverable execution error, only the second process dies, leaving the main process alive and allowing the correction of the script without crashing the whole tool.

1.3 XFaceMaker

For a full overview of XFaceMaker, see the XFaceMaker User's Guide and Reference Manual. We shall briefly describe here the features that are relevant to XFaceMaker/WF.

XFaceMaker is a UIMS, that is, it helps in designing and testing both the interface layout and its behavior.

XFaceMaker provides developers with the capability to quickly specify on a display precisely what a particular GUI will look like. This is done by creating widget instances from a palette containing the OSF/Motif widget classes, and arranging them into the desired layout. Then by setting the attributes (resources) of each widget instance, the interface layout can be further refined.

In addition to specifying the layout, with XFaceMaker the developer can also define the behavior of the interface. The behavioral component of the interface determines how the GUI and the underlying application (e.g. a business database application, or a numerical industrial simulation) interact with one another.

This behavioral component is specified using the FACE scripting language; i.e. the same language used for expressing the animation of drawings in XDrawMaker. Scripts written in FACE can be attached to any of the resources of a widget instance that describe dynamic behavior (i.e. the callback and translation resources).

XFaceMaker interprets scripts during the design phase and this makes it possible for developers to test both layout and behavior in tandem from within XFaceMaker by switching from "Build" to "Test" mode. This process is interactive and highly flexible allowing developers and end-users alike to work in a logical step-wise manner trying out different possibilities before deciding on a particular design. At the end of the day, XFaceMaker generates C code for the layout and the behavior. The code generated is genuinely compiled, with no calls to the FACE interpreter.

Interaction between the GUI designed with XFaceMaker and the application layer is performed by calling functions defined in the application from FACE scripts, and by using a mechanism known as "active values".

Application functions called from FACE scripts can also be loaded directly into the tool allowing the test of the interface together with the application proper. In addition to application functions, it is also possible to call any other library functions from scripts. The standard version of XFaceMaker is linked with the Xlib, Xt and the Xm libraries and with a rich set of convenience functions.

The FACE scripting language is a dialect of C and has been enriched with syntactic sugar for referring to widgets and their resources, and for active values. In the context of XFaceMaker/WF, active values play a very important role-- they allow the addition of new resources to widget classes defined in XFaceMaker.

Active values can be viewed as new resources attached to a widget. An active value sets up a two directional connection between a variable in a FACE script and one in the application layer. Through active values the application layer can directly access the resources of a specific widget in the interface without reference to the widget name.

Both active values and function calls provide a clean separation between interface and application, which will help reduce the cost of any maintenance changes made to either the GUI or the application layer.

Having developed the GUI to a required level, XFaceMaker permits the integration of the GUI component with the compiled application layer to produce a complete stand-alone X client program. During this integration phase developers can choose either:

- to compile i.e. to convert the GUI component directly into C code (this will result in a complete C translation of the whole GUI, including the FACE behavior scripts) or
- to integrate the GUI in an “interpreted” manner in the final application. In this case, the GUI is stored in a symbolic description file which is read in and interpreted by an appropriate function call within the application. This method has the advantage of facilitating the further testing and debugging of the GUI in conjunction with the application layer.

1.3.1 Generating Code for Classes

XFaceMaker is a very complete tool that furthers an object-oriented style for the design of graphic interfaces through two mechanisms: templates and the Xt or C++ classes. We shall not discuss templates here. The generation of classes in XFaceMaker is a unique feature of the tool that distinguishes it from the competition. You generate a class by simply drawing a prototype member of it. XFaceMaker then generates automatically the Xt or C++ code for the class from the prototype.

As an example, one can design an instance of a dialog box and then generate the code for it. The class is not simply a collection of widgets, say the widgets of the dialog box, but also includes its behavior. For example, it may include code for popping up a file selection box when pressing a button, for closing a window and for handling information collected in the box. XFaceMaker will generate the Xt and C++ code for all of this. To complete designing a class, one can define additional resources to it through active values.

Classes so generated can be used in an application for creating instances or for deriving new ones, just as if they had been written by a developer: Xt classes in C programs, and C++ in C++ applications. Furthermore, Xt classes are immediately available, with no further compilation or linking, in XFaceMaker's widget store, where they will be represented by icons, for immediate use in designing interfaces.

Classes created in the way described can be built from the widget classes available in XFaceMaker at any given time. They can include only Motif widgets as in the example of the dialog box. They could also include other widgets such as the widgets of the NSL Draw graphic library or third party widgets.

1.3.2 Integration with XDrawMaker

If you are running xfmwf (or an extended XFaceMaker executable with the Draw extension) and using the XnsIDraw widget, you may want to edit your drawing in XFaceMaker.

NSL's graphic editor XDrawMaker is integrated with XFaceMaker to add dynamic graphics in Motif interfaces. The XFaceMaker command *Edit Graphics with XDrawMaker* (in the *Edit* menu) can be called when any composite widget is selected, and in particular an XnsIDraw widget. It launches XDrawMaker and sets up a connection between XFaceMaker and XDrawMaker.

Once the connection is established, the existing graphics in the selected XFaceMaker widget are transferred to XDrawMaker and can be edited using XDrawMaker.

When the graphic editing is finished, the *Save* command in XDrawMaker transfers back the graphics to XFaceMaker and the selected object is rebuilt with the new graphic gadgets in it. XnslDraw graphic gadgets can be edited in XFaceMaker like any other widgets, and you can save them as part of a `.fm` file to build your application. XFaceMaker let you instantiate, select, edit Draw gadgets in ways that are similar to those available in XDrawMaker. Some resources, such as slots and links, or animation scripts, are easier to work with in XDrawMaker, others are just as easy to work with in XFaceMaker as in XDrawMaker. Within XFaceMaker, you can associate to objects in your drawing entities that are not known to XDrawMaker, such as active values, attachments, the *create* callback. These would be lost if the drawing were transferred to XDrawMaker; an alert box is popped to warn you of this and let you confirm or cancel your editing command.

As long as the XDrawMaker program is kept active, it is updated automatically every time the user selects a composite widget or an XnslDraw gadget in XFaceMaker. If a composite widget contains XnslDraw gadgets, the gadgets are loaded in XDrawMaker. If an XnslDraw widget is selected, all the gadgets contained in the parent widget are loaded. Otherwise, the XDrawMaker window is cleared.

You can exit the connected XDrawMaker program at any time, and launch it again using the command *Edit Graphics with XDrawMaker*.

- When the command *Edit Graphics with XDrawMaker* is called on an XnslDraw widget which has its `drawFile` resource set, only the name of the `.draw` file is transferred between XFaceMaker and XDrawMaker. This means that, when the *Save* command is called in XDrawMaker, the XnslDraw widget is simply rebuilt in XFaceMaker to read the modified `.draw` file, but the XnslDraw gadgets do not appear in the XFaceMaker widget tree and cannot be edited using XFaceMaker: they are recorded separately in the `.draw` file.
- When a drawing made with XDrawMaker has been exported in XFaceMaker format, and the `.fm` file is loaded in XFaceMaker the resources of all the graphic object are accessible in XFaceMaker. The objects may be kept in a Draw widget, or they can be cut/pasted into a different manager, e.g. an XmForm.
- The XDrawMaker object tree represents groups or paths and their members hierarchically. The XFaceMaker widget tree represents groups or paths as siblings of their members (no hierarchical representation) but the group or path object still has its children list and associated information. It continues to behave as an entity.
- The C code of a drawing can be generated by XFaceMaker (the application will execute the drawing in compiled mode, not interpreted mode) only if the drawing

has been loaded into XFaceMaker in XFaceMaker format. If you load a .draw file in XFaceMaker, the drawing will be executed in interpreted mode, even if the C code for the interface has been generated.

CHAPTER 2: Classes and XFaceMaker/WF

XFaceMaker/WF is designed to work in the X window environment and can generate both X (also known as Xt or X Intrinsics) classes and C++ classes.

X and C++ classes are designed in almost the same way with XFaceMaker/WF. Both the visual aspect and the animation of a widget class can be specified for widget classes designed with XFaceMaker/WF.

- **Visual Aspect.** The visual aspect of a class is fully under the control of the class designer. It is designed with XDrawMaker, the tool for creating animated drawings that is part of XFaceMaker/WF.
- **Animation.** The animation of a class is specified by means of scripts. Scripts are programs that are attached to a component of the drawing and that are triggered conditionally by events. A typical event could be that of clicking with the left mouse button, and it can be qualified by the condition that the click occur within the object. Scripts are specified in FACE, an easy-to-learn C-like programming language that is also used for scripts through its built-in FACE interpreter. The scripts are compiled into standalone code that is no longer interpreted at class generation time.

2.1 X Classes Designed with XFaceMaker/WF

2.1.1 Features of X Classes

X classes designed with XFaceMaker/WF have the following features:

- *Resources* – For any Xt class designed with XFaceMaker/WF, new resources can be designed at will. For any new resource, one can define its name, its type, and the desired side effects when the resource value is set (XtSetValues) or read (XtGetValues).
- *Subclassing* – Xt classes created can be subclasses of any of the standard Motif widget classes, of the XnslDraw widget class, or of a newly created widget class. The newly defined widget class can combine primitive widgets or gadgets and manager widgets in an arbitrarily deep hierarchy with the manager widget at the top. The newly designed widget class is then subclassed from the manager widget at the top.

- *Methods* – The methods of the new widget class (Initialize, Realize, Destroy, etc.) can be inherited from the superclass, redefined, or inherited from the superclass with additional side effects.

2.1.2 Using X Classes

Xt classes designed with XFaceMaker/WF can be used in one of the following three ways:

- Within a program – Xt classes designed with XFaceMaker/WF can be used in a C program just as if they had been written by a programmer. XFaceMaker/WF can generate stand-alone code for the class. The program using the class is usually a C program that also uses Xlib, Xt and Motif library calls. In addition to these libraries, any other library can also be used in the program. Programs written in other languages (e.g. C++, Ada, etc.) can also use the class and this depends on the usual techniques available for a particular language and its compiler for working with the Xlib.
- With XFaceMaker – XFaceMaker is unusual among interface design tools in being able to load a widget class within the tool. Furthermore, if the class has been designed with XFaceMaker/WF, the loading operation can take place on the fly; i.e. without requiring an extra rebuilding operation of the tool. This provides unparalleled simplicity and ease of operation. Once the class has been loaded into XFaceMaker it can be used to instantiate or derive new widget classes.
- With a third party user interface design tool – Certain third party user interface design tools allow the loading of third party widget classes. Such tools can then load widget classes designed with XFaceMaker/WF. On the fly loading of widget classes is not possible in user interface tools other than XFaceMaker.

2.2 C++ Classes

2.2.1 Features of C++ Classes

C++ classes designed with XFaceMaker/WF have the following features:

- **Base Class.** Any C++ class designed with XFaceMaker/WF is derived from a base class. The base class depends on the style of code generation selected. Two predefined styles are supported:
 - m NSL style – where the class created is derived from a base class called NslUIC.
 - m Rogue Wave Style – where the class created is derived from a class of the View.h++ library of Rogue wave. The View.h++ class library of Rogue Wave is available from NSL.
- **Class Data Members** – Data members correspond to Xt resources. For a newly created class you can specify, as is customary in C++, public, private and protected data members. Public data members are accessible to other classes. Private data members are to be used only within the definition of the class itself. Protected data members are not accessible from other classes but are visible. XFaceMaker/WF can generate automatically access functions to data members for reading (get) or writing (set). You can also specify side effects for read and write access.
- **Class Member Functions** – For a newly created C++ class, you can specify member functions. Member functions are always declared virtual. Their body is composed of a script which you can write in the FACE scripting language. FACE is a fully-featured programming language, easy-to-learn and C-like, also used in XFaceMaker and specially designed for the specification of widget animation.

2.2.2 Using C++ Classes

C++ classes designed with XFaceMaker/WF can be used as follows:

- **In a C++ program** – Since XFaceMaker/WF generates stand-alone code for the C++ class, the class can be used exactly as it would be if it had been designed by a C++ programmer: for creating instances and for defining derived classes.
- **With XFaceMaker** – A C++ class generated with XFaceMaker/WF can be used for the further generation of new classes derived from it.

CHAPTER 3: Some Uses of XFaceMaker/WF

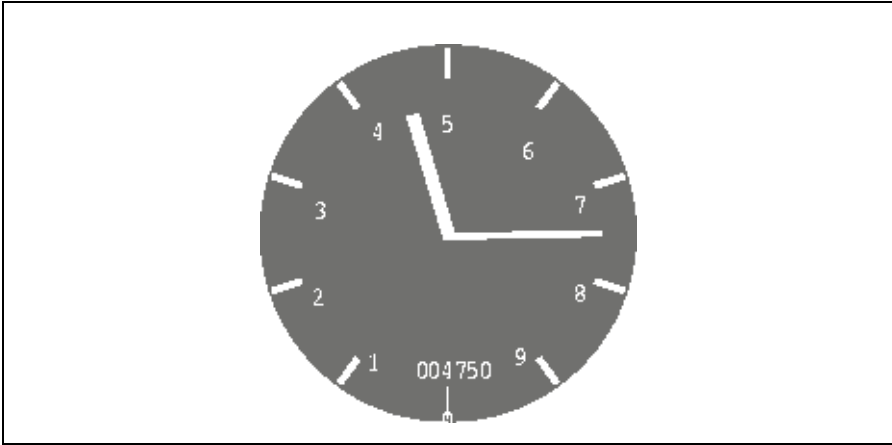
XFaceMaker/WF helps you design interactively object classes with functionality not available with a standard library of objects such as the Motif library.

Many applications cannot rely on the Motif widget set only. In the following sections we'll display examples of user interface components or fragments that can be designed with XFaceMaker/WF and that Motif couldn't handle. As we discuss them, we will also illustrate features of XFaceMaker/WF that make it different from other tools.

3.1 Dials

Many applications need dials: industrial control simulation, flight simulators, and other applications that may want to use a special dial metaphor.

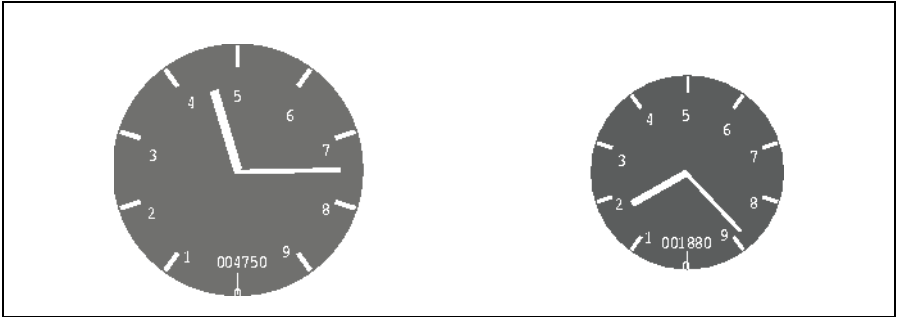
Dials, such as the altimeter below, can be easily designed with XFaceMaker/WF.



XFaceMaker/WF will generate not just one dial but the class corresponding to a whole set of altimeters that will differ from one another by the values of their resources. You can create as many instances of it as you wish, either within a program or with a user interface building tool such as XFaceMaker.

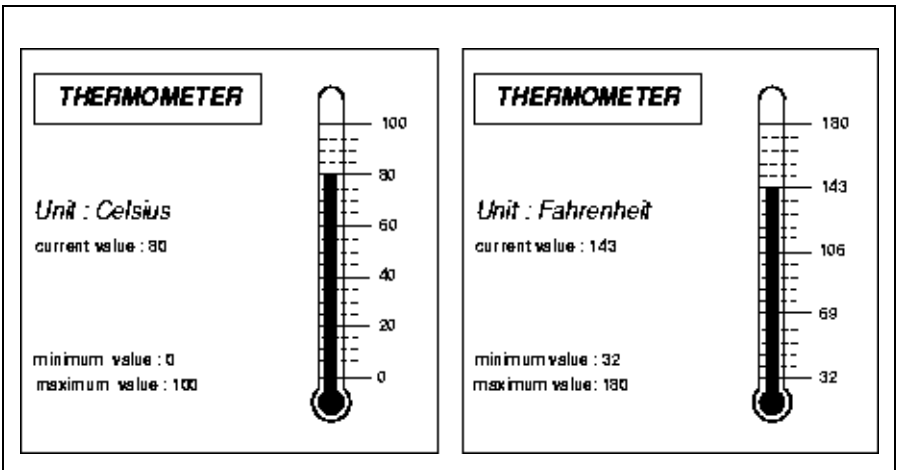
The dial is scalable; that is, you can create different instances with different sizes. The drawing of the dial is always drawn in relative coordinates. When used with XFaceMaker, you simply drag the icon in the desired position and the widget will automatically scale the drawing accordingly.

Here are two instances of the same altimeter class with different sizes:

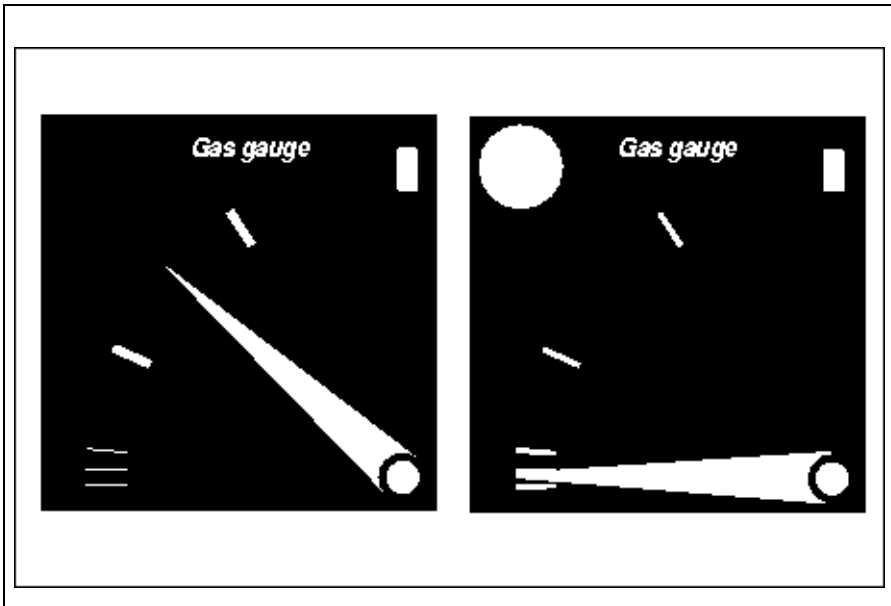


You can define additional resources corresponding to various features of the dial class you create. For example, you can define a resource called “altitude” that will control the position of the needle. Like any X resource, it can be set or controlled at runtime from within a program or from X resource files.

Almost any parameter can be defined as an X resource. Here are two instances of a thermometer whose resources are the minimum and the maximum temperature, the number of divisions and subdivisions.



Dials may change at run time depending on the values of resources. Here is an example of an automobile gas gauge with an alert lamp that lights up when the level is too low. This behavior is part of the widget and has been programmed into it. The application that uses the widget does not have to worry about lighting the alarm lamp when the level is too low. The widget will automatically do it for any value of the level below a predefined threshold value. By the way, the value of the threshold can also be defined as a resource, thus allowing you to design different instances with different alarm levels.



Scripts

Behavior in XFaceMaker/WF is specified in scripts. Scripts are programs written in a C-like programming language. XFaceMaker/WF has a built-in script interpreter that lets you edit and test scripts as you work.

Scripts are useful in many situations. One not so obvious situation in which scripts are useful is worth talking about at this point. It concerns the two resources mentioned above for setting the number of divisions and subdivisions of the dial. The code for the scale generated by XFaceMaker/WF contains a procedure for drawing the right number of divisions and subdivisions after reading the corresponding re-

sources. This script is called when the dial is drawn on the screen and the scale designer can specify it in the form of a script. Thus scripts have many uses and give a lot of power and flexibility to XFaceMaker/WF.

3.2 Graphs

Many applications must draw data graphs in different shapes. With XFaceMaker/WF one can easily draw charts and decorate them in ways not possible with ready-made data visualization widgets.

Ready-made widgets are the right choice in many cases, but are not without problems altogether. Typically, they are loaded with features but in any given case you use only a very small fraction of them. For example, a good graphing widget can display data in the form of data graphs, bar charts, or pie charts. But if you display the data as a pie chart you don't need the bar chart facility. XFaceMaker/WF solves this problem, by letting you design a charting widget that has just the features that you need.

Here are some examples of charting applications drawn with XFaceMaker/WF.

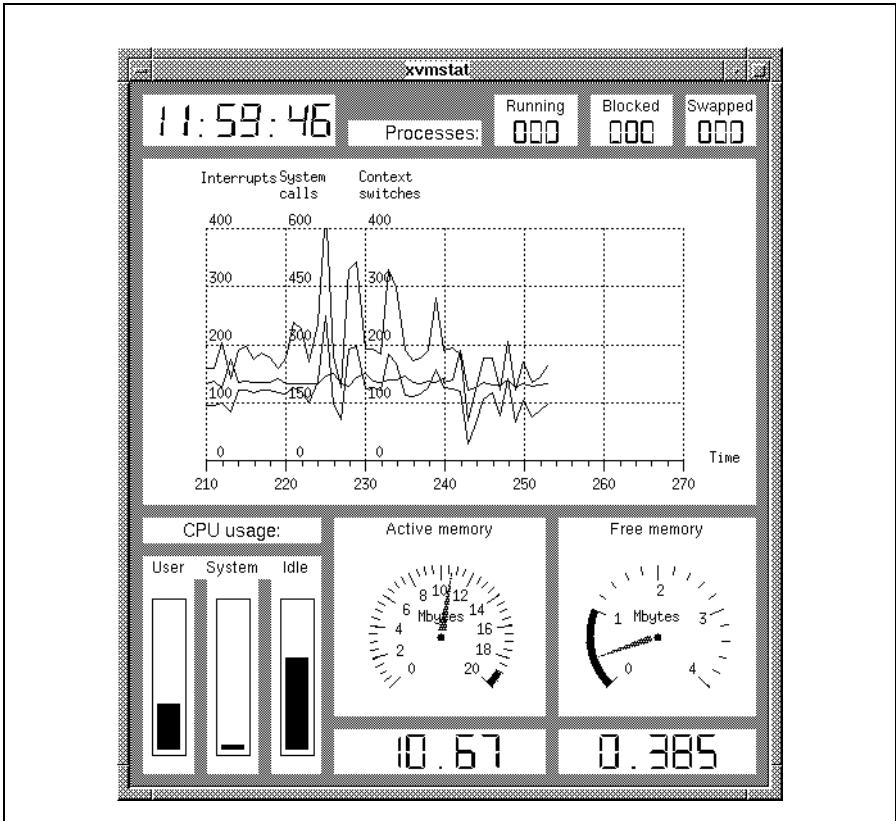
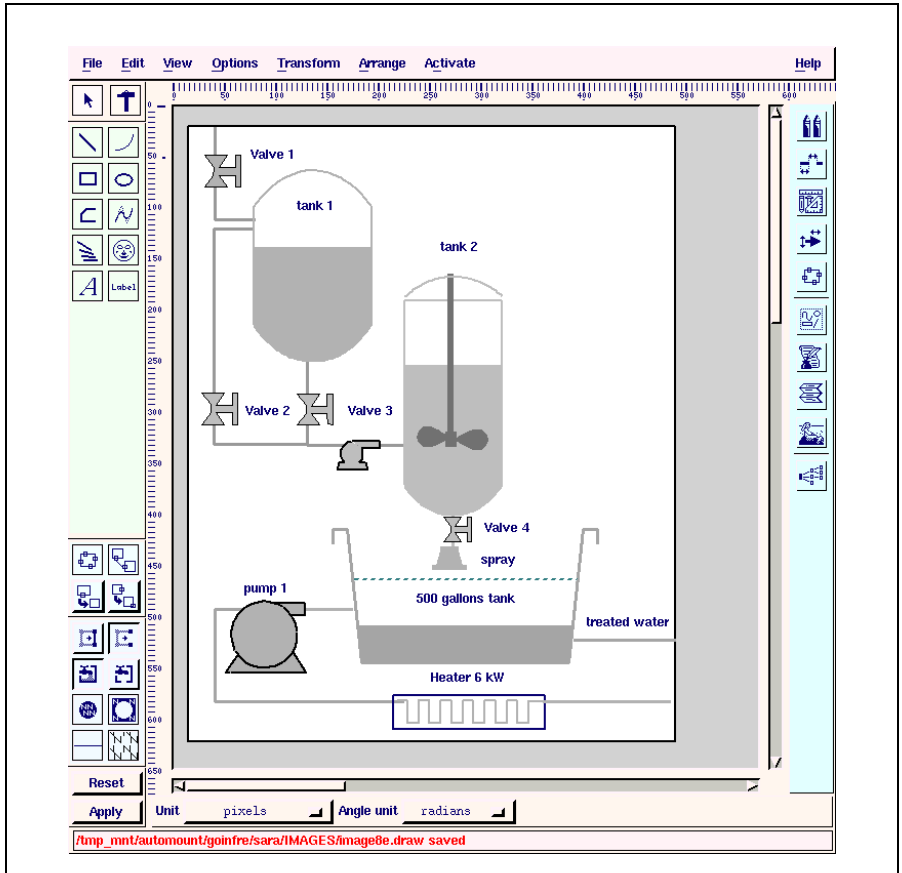


Figure 3.1: A Control Panel

The charts can be made to react to user input. The line on a chart is reactive: by clicking on it, the user can move its control points. The machinery necessary for using such a line as an input device is built into the polyline gadget. In XFaceMaker/WF it is possible to define a “callback” of the polyline that will transmit to the application the position of its control points when the mouse is released.

3.3 Animated Diagrams

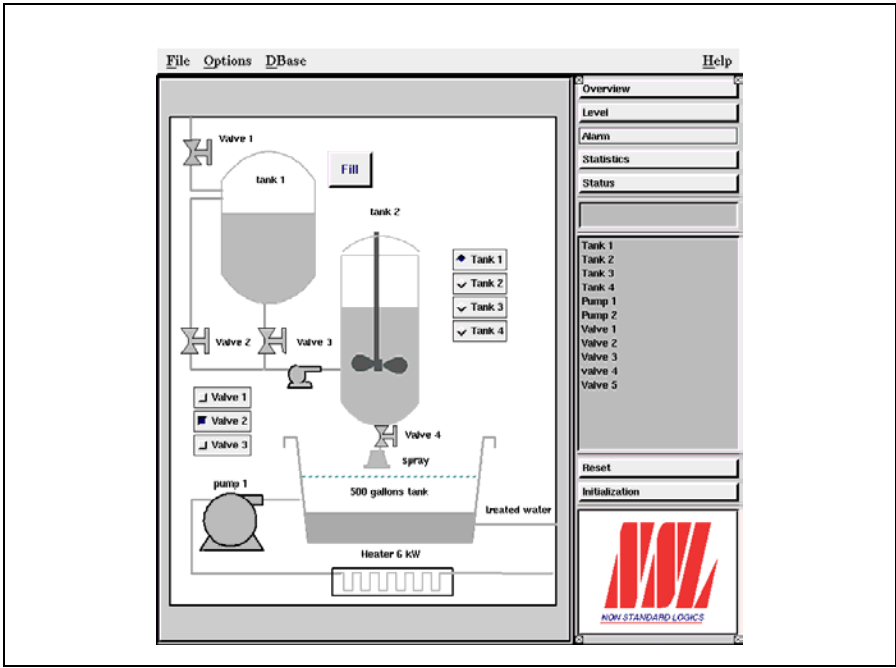
XFaceMaker/WF can also be used to design animated diagrams. The example below represents a heating installation.



This is not designed to become a widget class, but it could very well become one. The passive drawing, that is, the drawing without the animation part, was created with XDrawMaker. XDrawMaker has the functionality of a good drawing tool on personal computers. The animation was added later in the form of scripts. For example, there is a script that moves the propeller in the mixing tank and another one that describes the motion of fluid in the pipes or in the tanks.

Once created, the drawing can be printed and it can be imported in a special window in the interface of the final application. The interface of the final application, comprising the window and perhaps additional Motif widgets can be created in a number of different ways---either by direct programming or, better, by a tool for the design of Motif user interfaces such as XFaceMaker.

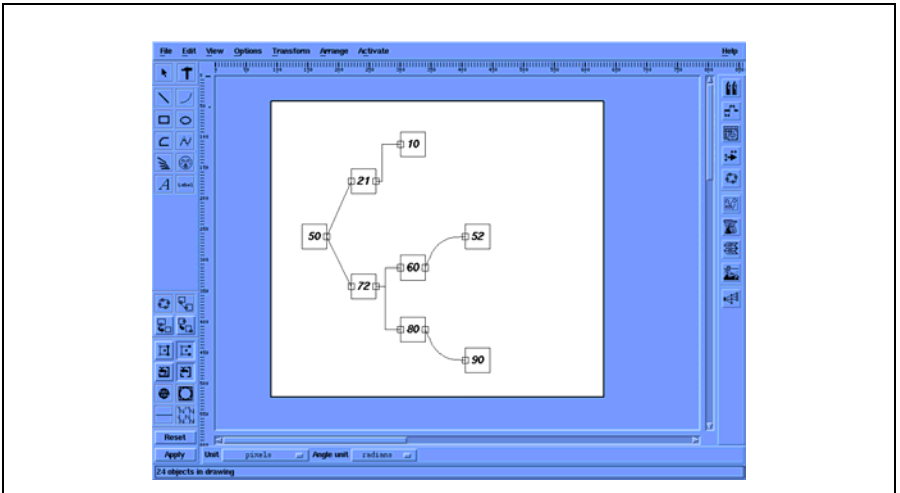
The picture below illustrates an interface in which the animated drawing created with XDrawMaker has been imported into a user interface created with XFaceMaker.



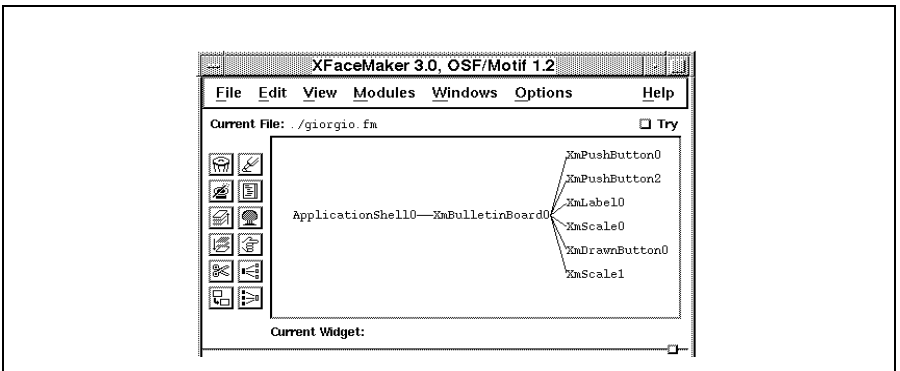
Notice the various additional Motif widgets that surround the drawing. Some of these Motif widgets have been placed within the drawing, others surround the drawing. The inclusion of Motif widgets and of primitive graphic widgets is only possible because both are X widgets. In older systems that implement the graphic primitives in proprietary object models instead of the Xt model, it is only possible to surround an animated drawing with Motif objects but it is not possible to *include* Motif widgets in the drawing and to animate in a uniform manner graphics and Motif objects.

3.4 Animated Charts

Another interesting application of XFaceMaker/WF is the design of animated charts, such as flowcharts. XFaceMaker/WF lets you construct links between objects. Slots can be associated with a graphic object such as a rectangle or a picture. Slots can be connected with a line segment, a polyline, or a Bezier curve. The links are maintained dynamically; that is, if an object is moved, either by program or interactively, the link follows.

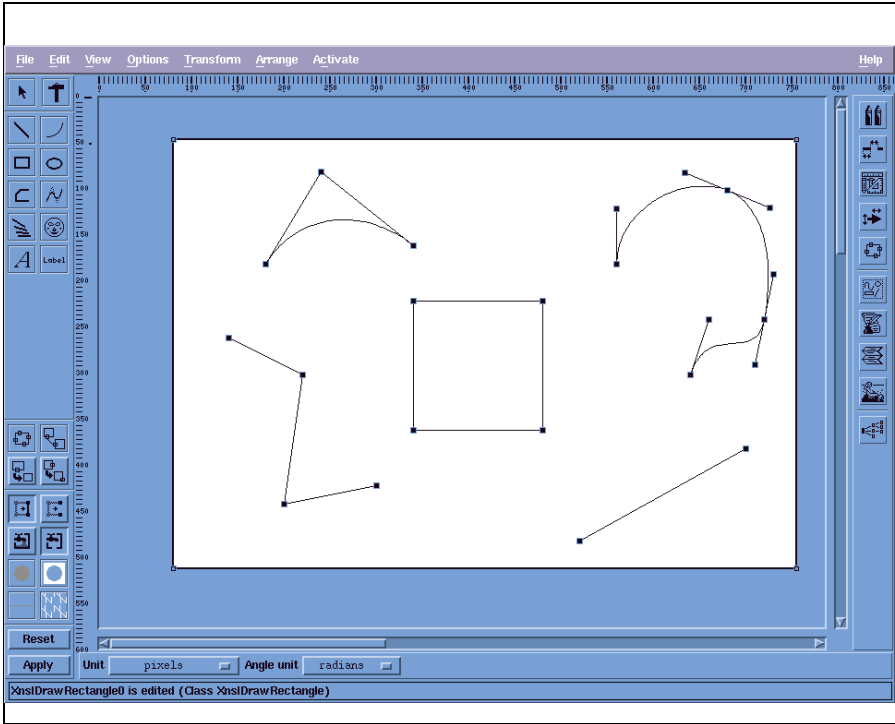


One can easily construct flowcharts or any kind of diagram that requires links between elements that can move. A good example is the tree-like representation of hierarchies.



3.5 Interactive Editors

The graphic objects of XFaceMaker/WF have *reactive* capabilities. To take a simple example, XFaceMaker/WF rectangle, line, spline, arc, and polyline support a reactive mode that displays handles, and have functionality that allows the user to program behavior associated with the corners. This functionality can then be easily used to build interactive tools such as editors.



The XDrawMaker editor used in XFaceMaker/WF was itself built this way. This capability is useful quite often as many applications require customized interactive editors for special purposes.

CHAPTER 4: A Guided Tour

This chapter will take you step by step through XFaceMaker/WF. We will design a new widget class from scratch and then show how to use it in an application.

The widget we shall design will be the simple ruler shown in Figure 4.1, “The ruler drawing,” on page 38. Notice that this kind of widget cannot be assembled out of existing Motif components.

There are three main stages in the process of designing and using a new widget class:

1. First, XDrawMaker is used to draw the widget and to implement some of its internal behavior.
2. Then, XFaceMaker is used to define the widget’s resources and the associated behavior.
3. Once the new widget class is ready, an interface may use instances of that new widget.

The full procedure comprises the following steps:

1. Draw the widget in XDrawMaker.
2. Attach scripts to animate the drawing.
3. Test and save the animated drawing.
4. Import the drawing in XFaceMaker.
5. Define new resources for the widget class.
6. Use the newly created widget class.
7. Test the interface in XFaceMaker.
8. Generate C-code for the widget class and the interface, then compile the whole application.

Following that are the source listings for the complete example.

4.1 Draw the Widget in XDrawMaker

The ruler we are designing is essentially an arrow sliding within a defined rectangle area, with labels and ticks indicating the available range.

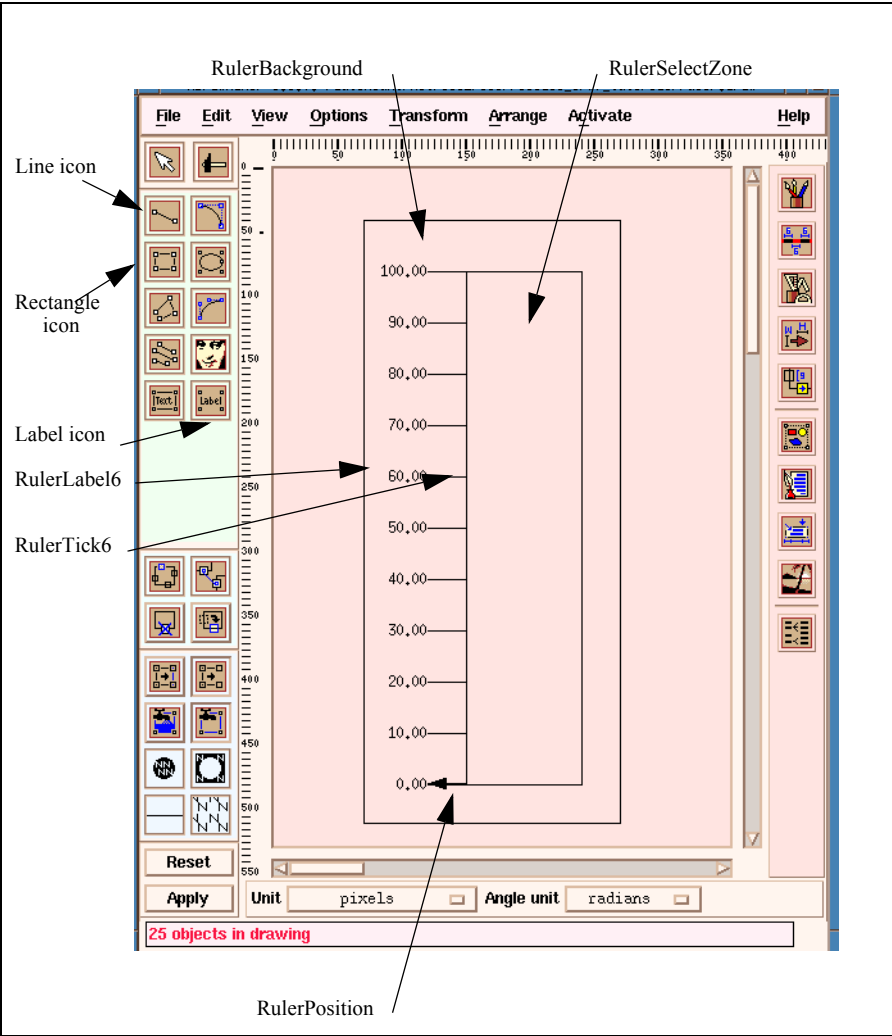


Figure 4.1: The ruler drawing

This ruler is drawn using five main objects:

- The background is an unfilled rectangle named `RulerBackground`. It is used to show the space used by the widget.
- The ruler area is an unfilled rectangle named `RulerSelectZone`. It defines the selection zone of the ruler.
- The ticks are lines named `RulerTickn`, where `n` varies from 0 to 10. They are used to divide the ruler area into ten parts. The cursor movement is limited between `RulerTick0` and `RulerTick10`.
- The labels are labels named `RulerLabeln`, where `n` goes from 0 to 10. They are used to give values to the tick marks.
- The cursor is an arrowed line named `RulerPosition`. This is the moving part of the ruler.

Other objects can be included for decorative purposes.

To draw this widget, launch XDrawMaker by typing:

```
dm &
```

If this command is not known, you should check with your product administrator that everything is correctly installed.

You should have an environment variable named `NSLHOME` set to the directory where NSL products are installed. You should also have included the path “`$NSLHOME/bin`” in your `PATH` environment variable.

Finally, your `LD_LIBRARY_PATH` (or `LIBPATH` on `ibm` or `LPATH` on `HP`) environment variable should include the path `$NSLHOME/lib` so that you can access the dynamic libraries and generate compiled versions of your XFM applications with dynamic libraries. If these environment variables are correctly set, you should be able to run XDrawMaker and XFaceMaker with the commands :

```
dm &
```

or

```
xfmwf &
```

If XDrawMaker is not yet running, please launch it. You should now have a running version of XDrawMaker on the screen.

To draw the first rectangle, click on the icon situated in the upper left of the XDraw-Maker window that represents a rectangle. Once the icon is selected, click in the drawing area where the top left corner of the rectangle should be, then click where the bottom right corner should be. Between the two clicks, a rubber band box shows what size the rectangle will be when you click for the bottom right corner.

Another method of drawing the rectangle is to press down on the left mouse button where the top left corner of the rectangle should be, drag the mouse without releasing the left button to the place where the bottom right corner should be and then release the left mouse button.

These two methods can be used indifferently to draw all the rectangular objects in the drawing. You either give the two characteristic points with two clicks or you press down the left mouse button on the first characteristic point and drag the mouse to the second characteristic point where you release the mouse button.

Select ***Options/Tree*** in the menu bar. The Tree window pops up. The Tree window shows all the objects of your drawing as a hierarchy. At this time, it will show only one object: the rectangle you created.

Double click on the name of the rectangle in the Tree window. This allows you to edit the object's name. Replace the object's name with the new name: "RulerBackground" and then press ***Return***.

Draw the second rectangle inside the first, representing the ruler area, in the same way. Change its name to "RulerSelectZone" in the Tree window.

Notice that both rectangles can be selected directly on the drawing or in the Tree window.

To draw the ticks, click on the icon in the upper left of the XDrawMaker window that represents a line. Once the icon is selected, click twice in the drawing area to choose the left and right ending points of the line. Position it at the bottom and to the left of the RulerSelectZone rectangle. Change its name to "RulerTick0".

Repeat this operation to create the other nine lines, spacing them to cover the same height as the RulerSelectZone.

You might want to use the grid to help you position the objects in XDrawMaker. The grid can be set up in the Grid Settings window which can be called through the ***Options/Grid Settings*** menu option. It is then shown by selecting the ***View/Grid*** menu option. Finally, it is rendered active by selecting the ***Options/Snap to Grid*** menu option. We will let you experiment with the grid on your own.

Once all 10 lines are created and renamed, you need to create the labels which will add meaning to them. To draw the labels, click in the upper left of the XDrawMaker window that says Label. Once the icon is selected, click twice in the drawing area to choose the top left and bottom right corners of the box representing the space taken by the label in the drawing. The label will appear with its name as default text. Change its name to “RulerLabel0” in the Tree window.

Repeat this operation to create the other nine labels, spacing them to cover the same height as the RulerSelectZone.

A label may have text that is different from its name. For the moment, the text is the same as the name, which is the default option. To change the text of the labels, open the Label Box by choosing the **Options/Object Boxes/Label Box** menu option. Select the label RulerLabel0 in the Tree window. The text of the label is now in the Label box where you can change it. Set it to 0.00. Select the east option for Text Alignment, click on **Apply**. Repeat this operation for all the labels, giving them values ranging from 0.00 to 100.0, then close the Label box.

The only missing object now is the arrow called RulerPosition. This arrow is a line such as those previously drawn, with some special attributes. Draw the line as before at the bottom of the RulerSelectZone. To change its attributes, pop up the Arrow Box by choosing the **Options/Attributes Boxes/Arrow Box** menu option. In the Arrow Box, select the left pointing arrow and click on **OK**. Change the default name of this object to RulerPosition.

The widget is now drawn and you have learned in this section how to:

1. launch XDrawMaker.
2. set up a Grid.
3. draw objects.
4. Change object names.
5. Use Object Boxes and Attributes Boxes to modify objects.

Take a look at the object tree shown in the Tree window. Notice that all the objects are shown as separate objects, without any link between them. When the new widget class will be prepared, you will use an XnsIDraw widget which will reference the drawing you have just prepared. This XnsIDraw widget will be the parent of all the objects shown in the Tree window, but these objects will not be visible in the widget hierarchy. To access and manipulate these objects, you will have to use the XnsIDraw functions, some of which will be introduced in the next paragraph.

4.2 *Animate the Drawing*

Once you have created a drawing of the ruler, the next step is to write a script which defines the ruler's behavior.

Scripts are written in the FACE language. Face is similar to C and is easy to learn and use. The small differences between Face and C have been introduced to direct references to widgets and their resources.

Face is interpreted in XDrawMaker. This means that you can test scripts on the fly without exiting XDrawMaker and reduce your development time.

You will see how to generate genuine C or C++ code from Face scripts in XFace-Maker when the new widget class and the sample interface are prepared.

For more information on the Face language and on the functions used in the following scripts, please read the XDrawMaker User's Guide and the Draw Library Reference Manual.

The following script describes how the cursor moves when the user clicks in the ruler area. Pop up the script definition box by choosing the *Activate/Script* menu option. Select the RulerSelectZone object. The text "Script of widget RulerSelectZone" should appear in the Script Editing box.

Please enter the following script in the Script Editing box. You can use the Actions menu to specify which action (on Button1Down) will trigger the script, and the Functions menu to fetch the function skeletons. You can thus reduce typing to a minimum.

The script is explained after the listing.

```
on Button1Down
/* Retrieve RulerTick0, RulertTick10 and RulerPosition
   objects */
Widget RMin = XnsldDrawNameToObject (parent,"RulerTick0");
Widget RMax = XnsldDrawNameToObject (parent,"RulerTick10");
Widget RPos = XnsldDrawNameToObject (parent,"RulerPosition");
if (!RMin || !RMax || !RPos)
    printf("Ruler Error: Ruler widget id's not found \n");
else {
/* Get the positions of Tick0 and Tick10 */
    Float ymin = XnsldDrawGetCenterY (parent, RMin);
    Float ymax = XnsldDrawGetCenterY (parent, RMax);
/* Get the Y position of the mouse */
    Float y = citof( XnsldDrawEventY (parent, $));
/* Test if the click was within limits */
```



```

        if (y < ymax) y = ymax;
        if (y > ymin) y = ymin;
/* compute a translation to move the cursor */
        Float ty = y - ymin;
        XnslDrawSetTranslation (parent, RPos, 0.0, ty);
/* Show the changes */
        XnslDrawRedisplay (parent);
    }
end

```

What should this script do? It should move the `RulerPosition` object to the `Y` position where the user has clicked the `Select` button of the mouse if this click occurred between the bottom and top tick marks.

To do this, the script should react only to a `Button1Down` event. That is the purpose of the first line of the script. You may define different scripts for different user actions in this way.

First, you need to retrieve the widget ids of the drawing's objects. The `XnslDrawNameToObject` function returns a pointer to an object, given the object's name and the widget id of the `XnslDraw` widget acting as its parent.

Here, the widget id of the parent widget is not yet known. It will only be accessible when the drawing is associated with the `XnslDraw` widget in `XFaceMaker`. However, the `Face` language defines some reserved names to reference widgets. The reserved names `self` and `parent` represent the widget ids of the current widget and of its parent. You can thus use the `parent` reserved name as the first argument to the `XnslDrawNameToObject` function. The name of the object is passed as the second parameter. `XnslDrawNameToObject` is most frequently used this way.

The necessary objects in our example are the first and last tick marks and the position. Notice that it is possible in `Face` scripts to declare the type of a variable at the same time it is given a value. If you do not declare the type of the variable, `Face` will choose the type according to the context in which the variable is used - it does dynamic typing and will not warn you of any type change of that variable in the rest of the script. If you use variables whose type should not change, e.g `Float`, you should always declare the type of the variable when it is first used.

The second step is to retrieve the current positions of the tick marks. The `XnslDrawGetCenterY` function returns the `Y` coordinate of the center of the object.

You should then get the Y coordinate of the click event. The event is referred by the \$ variable and the XnslDrawEventY function returns the Y coordinate of the event (it is a Button1Down event because the script is declared only for this event). The citof function is used here to convert the integer value returned by XnslDrawEventY into a float, because further computations will be done in floating point notation.

Then, you should check if the event has occurred at a valid location. The cursor movement must be constrained within the limits defined by the RulerTick0 and RulerTick10 locations.

Once the click value is correct, you can compute a translation for moving the cursor to the location specified by the user's click. This computation is simply the difference between the Min limit and the location of the event.

The actual translation is performed by means of the XnslDrawSetTranslation function. It takes the parent's widget id as the first parameter, the object's widget id as the second and the x and y floating-point translation values as third and fourth parameters.

Since the previous operations may have changed the appearance of the drawing, we have to refresh the screen. This is done by calling the XnslDrawRedisplay function.

Suppose that you now want the cursor to move when the user drags the mouse (i.e. when the user clicks and moves the mouse with the button pressed). Instead of re-writing the script for the Button1Motion event, the Button1Down script can be named and called directly. To name a script, you use the script command followed by the name of the script and by :. The contents of the script follow that first line. To call a named script, you use the same command with ; instead of : as the finishing character. Our example becomes:

```
on Button1Down
script MoveCursor :
.....
.....
.....
XnslDrawRedisplay(parent);
}
end
```

The script is now named, (its name is MoveCursor) and can be used for another event, typed after the end line of event Button1Down:

```
on Button1Motion
script MoveCursor;
end
```

The behavior of the ruler is now the same whether the mouse is dragged or clicked.

You have learned in this section how to:

1. attach a script to an object and a specific event.
2. use the functions `XnslDrawNameToObject`, `XnslDrawGetCenterY`, `XnslDrawEventY` and `XnslDrawSetTranslation`.
3. use the parent reserved name and the `$` variable.
4. declare the type of your variables to enable type checking.
5. name a script and call it in another script.

4.3 Test and save the animated drawing

Having written these scripts, you can now try your drawing by using the *Activate/Launch a Separate Process* menu option. When the XDrawMaker Exec window appears, click within the ruler area with the *Select* button (mouse Button 1). The ruler should move to the specified location along the Y axis. You can now return to the editor by clicking on the *Exit* button.

You can also test the scripts by using the *Activate/Run in DrawMaker window* menu option. The difference between the two modes is that in this mode, the animation will work in the window in which you designed the drawing.

If your scripts have errors, running them in XDrawMaker's window may lead to a crash of XDrawMaker. If you launch a separate process for your tests, crashing will only interrupt the separate testing process, not XDrawMaker. You will then be able to correct your scripts more rapidly, since you will not need to reload XDrawMaker. Thus, this two-process arrangement makes XDrawMaker an error-tolerant tool.

To save your draw file, select the *File/Save As* menu option, call the file **ruler.draw** in the Selection field of the File Selection box and *OK* your choices.

Take a look at the **ruler.draw** file. It contains the description of the animated drawing in an ascii format called the draw format. The draw format uses a simple syntax, very close to that of C structures.

You can examine the draw file with any text editor, for example with the Wx editor that is a part of XFaceMaker/WF.

To launch Wx and load the **ruler.draw** file, type :

```
wx ruler.draw
```

in an xterm window. The draw file can easily be read and modified directly in the editor, although we do not recommend it.

Here are some parts of the **ruler.draw** file:

```
.....
XnsldrawRectangle RulerBackground {
drawShowBorder = True;
drawPoints = "70, 40, 200, 470";
drawNumPoints = 2;
}
XnsldrawRectangle RulerSelectZone {
drawShowBorder = True;
drawEventMask = Button1Down|Button1Motion;
```

```
drawActivateCallbackList = "on Button1Down
script MoveCursor:
/* Retrieve RulerTick0, RulerTick10 and
   RulerPosition objects */
Widget RMin = XnsldDrawNameToObject (parent, \"RulerTick0\");
.....
end
\";
drawPoints = \"150, 80, 90, 400\";
drawNumPoints = 2;
}
```

Notice how the draw file contains the script you defined. The whole script is saved as a string (between “”), which explains why you see “\” in the XnsldDrawNameToObject function call when the name of the object is specified.

The draw file’s contents are interpreted to recreate the drawing in XDrawMaker’s main window. In run mode, the scripts are also passed to XDrawMaker’s built-in interpreter for execution. Note however that the Face interpreter is not needed to run the compiled version of an application using scripts.

You have learned in this section how to:

1. Test an animated drawing.
2. Save a drawing in a draw file.
3. Look up the draw file format in the Wx editor.

4.4 Import drawings in XFaceMaker

The drawing created with XDrawMaker is not yet a widget since no resources or callbacks were defined. However, you can now load the drawing in XFaceMaker, try its internal behavior, and define its resources. To do so, you must launch a version of XFaceMaker that contains the Draw widget. On the standard delivery, this version is called **xfmwf**. Launch xfmwf with the following command:

```
xfmwf &
```

The XFaceMaker window comes up after the initialization has completed (see Figure 4.2, “XFaceMaker main window,” on page 49).

Select the XnslDraw widget icon in the NSL widgetstore of the Toolkit and stretch a new window on your screen when the cursor changes to a cross. XFaceMaker will automatically create an application shell to contain the XnslDraw widget.

The Draw widget is called xnslDraw0 by default. Select it in the Widget Tree and open a new resource window; you can either

- open a popup menu (press mouse Button 3) on the Draw widget and select the option Resources for xnslDraw0 widget.
- click mouse Button 2 on the Draw widget
- choose the **Windows/New Resource Window** menu option.

Any of these actions will popup a new resource window for the widget, like the one in Figure 4.3, “XFaceMaker Ressources Window,” on page 50.

In the Current Widget Resources window, find the **drawFile** resource and set it to **ruler.draw**. Set the **autoResize** resource to **True**. Select the **Auto size** and **Auto position** togglebuttons. Setting these two togglebuttons is necessary for your class to be resizable when you instanciate it. Otherwise, it will keep the size you gave it during creation and you will not be able to adapt it to your interface. Apply your changes by clicking on the Rebuild button and the drawing created in XDrawMaker will appear in the xnslDraw0 widget.

Switch to Try mode. The behavior of the ruler is the same as in XDrawMaker; i.e., the ruler moves following the mouse. Return to Build mode.

Select the xnslDraw0 widget in the Tree. This widget can now be saved as a new class, using the **Modules/Classes/Save As** menu option. Call the file **Ruler.fm** and set the default name toggle, to name the new class **Ruler**.

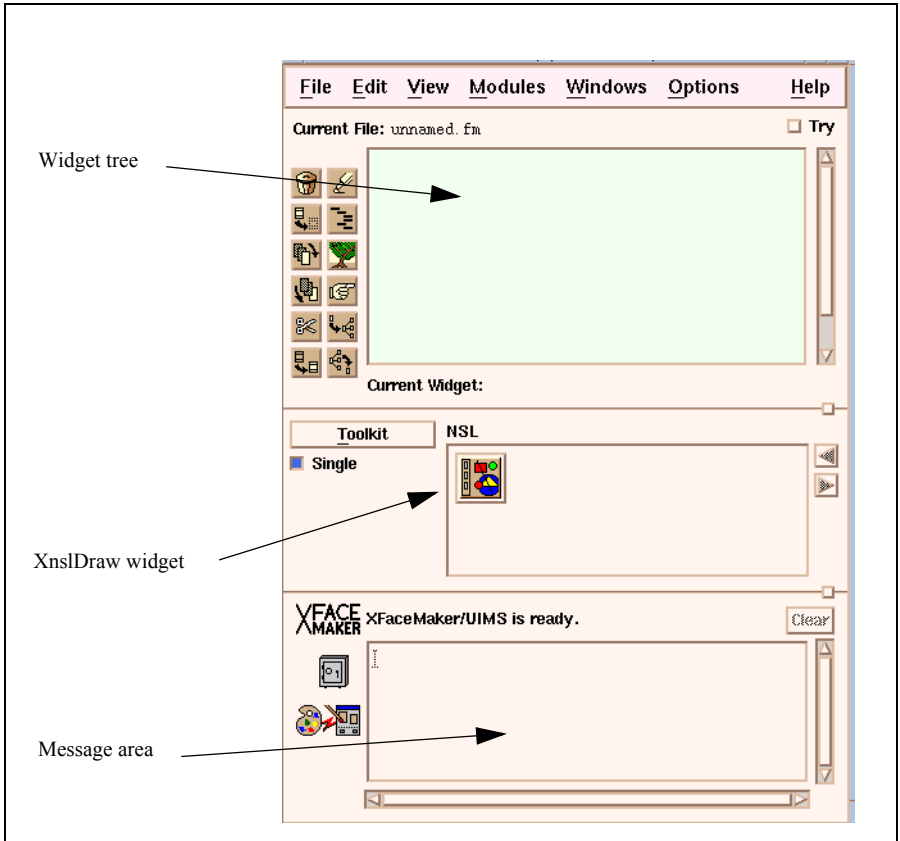


Figure 4.2: XFaceMaker main window

You can create a new icon for this class. Use the *Modules/Classes/Choose Icon* menu option. Then choose the Ruler class name in the selection list. The icon selection box shown in Figure 4.4, “The Ruler icon,” on page 51 comes up. Select the *New* button, choose a 32x32 size and then draw the new icon.

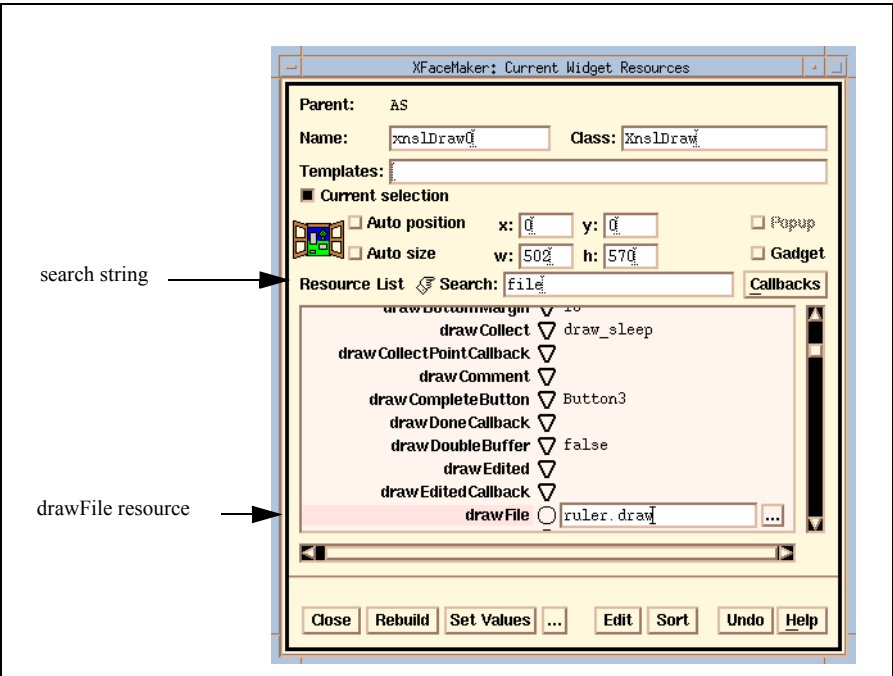


Figure 4.3: XFaceMaker Ressources Window

When you are finished, Save the icon under the name Ruler.bm, then OK the box. Icons such as this one are saved in X bitmap format, hence the .bm suffix.

To ensure that the new class and its icon are loaded every time you run XFaceMaker, save your preferences in the file `.xfm.startup` in your login directory by selecting the **Save Pref**s option in the File menu.

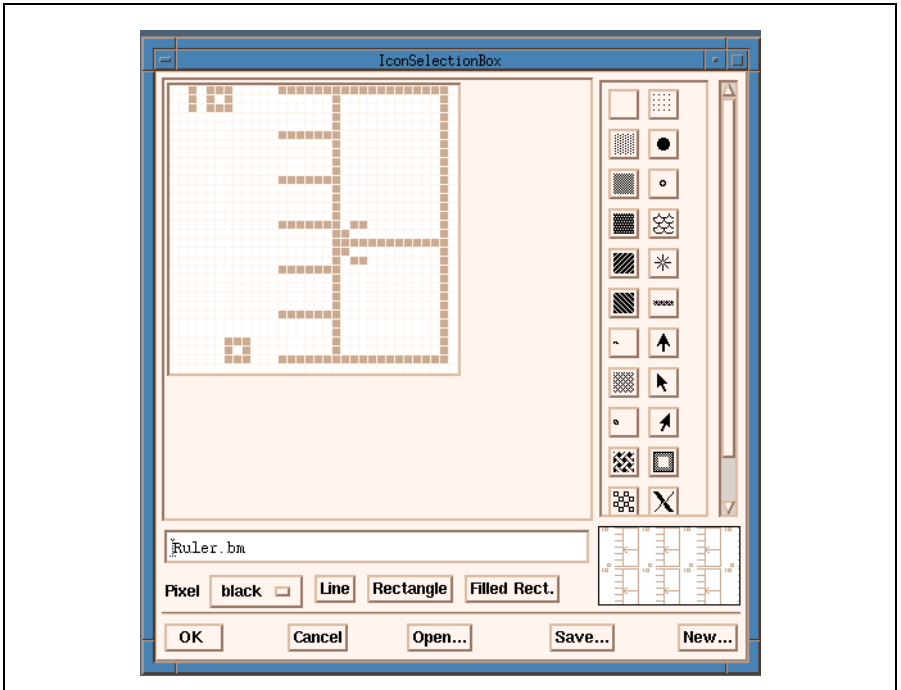


Figure 4.4: The Ruler icon

You have learned in this section how to:

1. Launch XFaceMaker and open a resource window.
2. Associate a draw file with an XnsIDraw widget.
3. Edit a widget's resources.
4. use the build and try modes.
5. Save part of an interface as a widget class.
6. Attach an icon to a widget class.
7. Save preferences.

4.5 Define new resources for the widget class

4.5.1 Define a rulerPositionColor resource

The new class having been defined, you can now add resources to it.

New resources are defined in XFaceMaker as **active values**. As long as you are defining the new Widget class, you usually say that you are adding new active values to the widget being edited. Once you have saved that widget as a class and you are using an instance of it in an interface, then you talk of resources. Both terms actually refer to the same thing, but in different contexts.

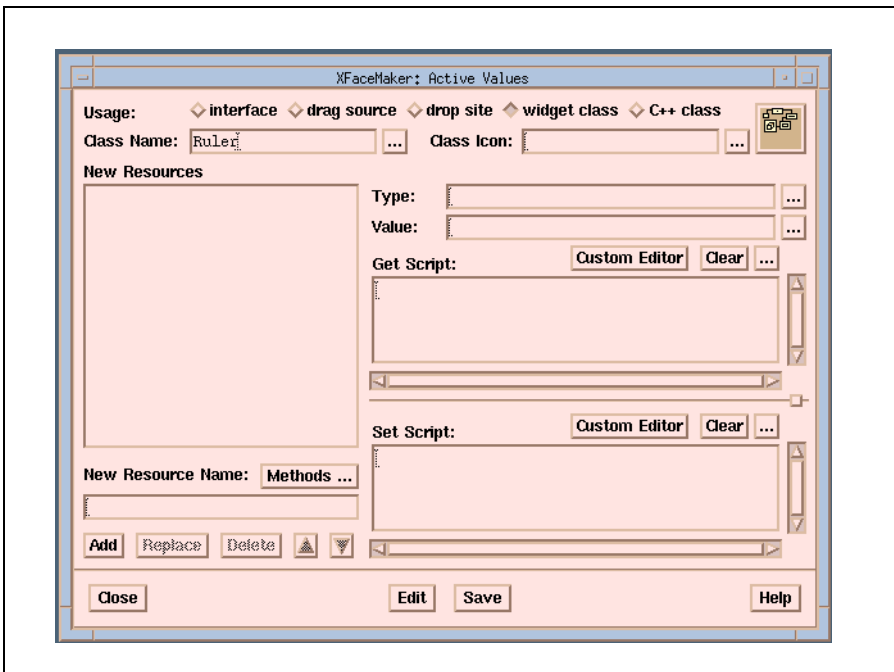


Figure 4.5: XFaceMaker Active Values Window

To define a new resource, use the Active Values window by selecting the *Windows/Active Values* menu option. Check also that the *xnsIDraw0* widget is selected in the Tree.

Set the usage to widget class by clicking on the “Widget class” toggleButton at the top of the *Active Values* window.

Suppose that you now want to define a resource called **rulerPositionColor** which controls the color of the cursor. To do so, type its name in the text field below the label “New Resource Name”. Then, give it a type by typing “Pixel” in the **Type:** text field or by choosing one of the options offered when you click on the ... button next to it. Give it a default value by typing “black” in the **Value:** text field or by choosing one of the colors offered when you click on the ... button next to it.

You also need to define the behavior of the widget when this resource is set to a new value by the application or when the application wants to get its value.

Any active value or future resource has two scripts associated with it: the **Set** script and the **Get** script. The Set script will be automatically triggered when the Initialize method of the class is activated and whenever the application modifies the active value. The Get script will be triggered whenever the application reads the value of the active value.

The Get script is required only when side effects are needed, e.g., when the information is actually kept in a resource of a child widget. It should fetch the value of the variable, transform it if necessary and then put the transformed value into the active value variable. The application then gets this transformed value.

In the present case, the get script is not needed since the application can use the resource value directly when it accesses it. If the application changes the value of this resource, however, the drawing has to be updated to reflect the new value. This is done by the following Set script, to be typed in the Set script text field:

```
Widget pos_wid = XnsldrawNameToObject(self,
                                     "RulerPosition");
if (pos_wid) XnsldrawSetForegroundPixel(self, pos_wid, @);
XnsldrawRedisplay(self);
```

Notice the use of the **@** variable. It is a special variable in Face, referring to the value of the active value or resource. If you wish to access the address of the active value or resource, you should use the **\$** variable.

The Set script defines the action to take when the resource is set. The first line retrieves the RulerPosition object’s widget id. The XnsldrawNameToObject function is used here with the “self” reserved name for the widget id of the Xnsldraw widget, since the resource is defined for this widget. The second line sets the color according to the received value **@** with the XnsldrawSetForegroundPixel function and the last line refreshes the drawing in the xnsldraw0 widget.

Once all the fields (name, type, value, get and set scripts) have been prepared, click on the **Add** button to add this new resource to the widget class. The name of the new resource appears in the “New Resources” list.

To modify an already defined resource, you first select it in the “New Resources” list. All the fields in the box are filled with their current values and you can modify them. Once all the modifications are done, you click on the **Replace** button to replace the resource definition.

Save the class. When you create an instance of this new widget in an application, the list of resources for this widget will now include the new resource named **rulerPositionColor**. You will be able to edit this resource to change the color of the cursor.

You can also define other resources. A ruler would have a resource which controls the position of the cursor, varying between a minimum and a maximum. For this purpose, you need three resources:

Resource	defines	associated draw gadget
rulerMin	the minimum value allowed	RulerLabel0
rulerMax	the maximum value allowed	RulerLabel10
rulerValue	the current value in between	RulerPosition

You will define these three resources in the next sections.

You have learned in this section how to:

1. use the Active Values window in widget class mode.
2. define a new resource for a widget class.

4.5.2 Define a rulerMin resource

Define a new resource named rulerMin with “Type: Float” and “Value: 0.00” and then edit its set script. The set script needs to set the value of the resource and then to change the label of the widget to display the new value. Since this will need to be done also for the rulerMax resource, it is best to define a function to change the ruler’s labels. The source of the set script listed below is self explanatory.

You can enter it directly in the script editing window - enlarge the window by clicking on the ellipsis (...) above the scrolled text. You will work more comfortably, however, if you open the Custom Editor. The editor you will launch depends on how your FMEDITOR environment variable is set. We recommend you set FMEDITOR

to wx242, to use NSL's wx editor. You can also use emacs, vi, etc. When you click on the **Custom Editor** button, the editor is launched and communication is established between XFaceMaker and the editor, such that, when you save your work in the editor, XFaceMaker receives the data. You then have to save the data in XFaceMaker, here with the **Add** or **Replace** button of the ActiveValue box, and subsequently with the **Save** button when you save the new definition of your class. If you are working with an editor that implements language modes, e.g., wx, your task will be made easier if you select the C language mode (in wx, **Galactic-C**).

```
/* Define the function to change the labels */
function None RulerChangeLabels()
{
String str; str = malloc(20);

/* Change min label */
Widget label_wid = XnslDrawNameToObject(self, "RulerLabel0");
sprintf(str, "%7.2f", Float(self:rulerMin));
if (label_wid) label_wid:drawLabelString:String = str;

/* Change max label */
label_wid = XnslDrawNameToObject(self, "RulerLabel10");
sprintf(str, "%7.2f", Float(self:rulerMax));
if (label_wid) label_wid:drawLabelString:String = str;

/* Change all intermediate labels */
Float delta = Float(self:rulerMax) - Float(self:rulerMin);
Float min = Float(self:rulerMin);

label_wid = XnslDrawNameToObject(self, "RulerLabel1");
Float value = min + 0.1 * delta;
sprintf(str, "%7.2f", value);
if(label_wid) label_wid:drawLabelString:String = str;

label_wid = XnslDrawNameToObject(self, "RulerLabel2");
Float value = min + 0.2 * delta;
sprintf(str, "%7.2f", value);
if(label_wid) label_wid:drawLabelString:String = str;

label_wid = XnslDrawNameToObject(self, "RulerLabel3");
Float value = min + 0.3 * delta;
sprintf(str, "%7.2f", value);
if(label_wid) label_wid:drawLabelString:String = str;
```

```
label_wid = XnslDrawNameToObject(self, "RulerLabel4");
Float value = min + 0.4 * delta;
sprintf(str,"%7.2f",value);
if(label_wid) label_wid:drawLabelString:String = str;

label_wid = XnslDrawNameToObject(self, "RulerLabel5");
Float value = min + 0.5 * delta;
sprintf(str,"%7.2f",value);
if(label_wid) label_wid:drawLabelString:String = str;

label_wid = XnslDrawNameToObject(self, "RulerLabel6");
Float value = min + 0.6 * delta;
sprintf(str,"%7.2f",value);
if(label_wid) label_wid:drawLabelString = str;

label_wid = XnslDrawNameToObject(self, "RulerLabel7");
Float value = min + 0.7 * delta;
sprintf(str,"%7.2f",value);
if(label_wid) label_wid:drawLabelString:String = str;

label_wid = XnslDrawNameToObject(self, "RulerLabel8");
Float value = min + 0.8 * delta;
sprintf(str,"%7.2f",value);
if(label_wid) label_wid:drawLabelString:String = str;

label_wid = XnslDrawNameToObject(self, "RulerLabel9");
Float value = min + 0.9 * delta;
sprintf(str,"%7.2f",value);
if(label_wid) label_wid:drawLabelString:String = str;

/* free the string buffer & redisplay */
free(str);
XnslDrawRedisplay(self);
}
/*call the function to change the labels */
RulerChangeLabels();
```

In this script, you have defined and used a function using the Face language. One of the characteristics of the Face language is its ability to modify the resources of a particular widget using a very simple syntax:

```
widget:resource_name = resource_value;
```

or

```
widget:resource_name:resource_type = resource_value;
```

You use the second form when you intend to generate the C code as Stand Alone rather than using the `Fm_c` library: you have to specify the type of the resource as the `Fm_c` library will not be there to determine types dynamically. The C code for classes should be generated as Stand Alone, it makes them easier to use.

Here, you have used the typed syntax to change the value of the `drawLabelString` resource of the label widgets.

No get script need be defined for the `rulerMin` resource since the value of this resource can be retrieved directly by the application.

You have learned in this section how to:

1. define a function in a Face script.
2. call a user defined Face function in a script.
3. modify a resource value.
4. use a Custom Editor to define scripts.

4.5.3 Define a rulerMax resource

Proceed similarly to define a new resource named `rulerMax` with “Type: Float” and “Value: 100.00”, then edit its set script. The set script needs to set the value of the resource and then to change the labels of the widget to reflect this new value. Since this operation has already been defined in the `rulerMin` resource, you only need to call the `RulerChangeLabels` function. Here is the source of the set script:

```
RulerChangeLabels();
```

No get script is necessary, as for the `rulerMin` resource.

4.5.4 Define a rulerValue resource

The `rulerValue` resource is more complicated: you have to connect the position of the cursor and the value of this new resource.

Define a new resource named `rulerValue` with “Type: Float” and “Value: 0.00” and then edit its scripts.

The Set script must take the resource value and move the ruler accordingly. All the functions used in this script have been used before, so you should have no problem understanding it. Here is the source of the Set script:

```
/* Retrieve the useful objects and their positions */  
Widget RMin = XnsldrawNameToObject(self, "RulerTick0");
```

```
Widget RMax = XnslDrawNameToObject(self, "RulerTick10");
Widget RPos = XnslDrawNameToObject(self, "RulerPosition");

if( !RMin || !RMax || !RPos)
    printf("Ruler Error: Ruler widget Ids not found\n");
else {
    Float ymin = XnslDrawGetCenterY(self, RMin);
    Float ymax = XnslDrawGetCenterY(self, RMax);
    Float y = XnslDrawGetCenterY(self, RPos);
    /* Get the resource value contained in the @ variable */
    Float v = Float(@);
    /* Check it is valid */
    Float min = Float(self:rulerMin);
    Float max = Float(self:rulerMax);
    if (v > max) v = max;
    if (v < min) v = min;

    /* compute the translation for the new position */
    Float ty = (v - min) * (ymax - ymin) / (max - min);
    XnslDrawSetTranslation(self, RPos, 0.0, ty);
    /* Redisplay the widget */
    XnslDrawRedisplay(self);
}
```

The get script must compute the value of the resource according to the ruler position.
The get script can be:

```
/* Retrieve the useful objects and their positions */
Widget RMin = XnslDrawNameToObject(self, "RulerTick0");
Widget RMax = XnslDrawNameToObject(self, "RulerTick10");
Widget RPos = XnslDrawNameToObject(self, "RulerPosition");

if( !RMin || !RMax || !RPos)
    printf("Ruler Error: Ruler widget Ids not found\n");
else {
    Float ymin = XnslDrawGetCenterY(self, RMin);
    Float ymax = XnslDrawGetCenterY(self, RMax);

    /* Retrieve the cursor's current Y value */
    Float y = XnslDrawGetTranslationY(self, RPos) + ymin;

    /* Compute the value */
    Float min = Float(self:rulerMin);
    Float max = Float(self:rulerMax);
```



```
Float v = min + (y - ymin) * (max - min) / (ymax - ymin);

/* Copy it to the resource value */
@ = v;
}
```

Save your class. You might want to create an instance of your class now and verify that it has several new resources. To do so, open the User Defined widget store, click on your class icon and draw out an instance of Ruler. Open the resource editor for the widget and verify its resources.

4.5.5 Define a valueChangedCallback resource

Widgets usually have Callbacks. You can define a callback resource for your new widget class, to be triggered whenever the position of the RulerPosition object in the drawing changes. Select the Draw widget again. Define a resource named valueChangedCallback with “Type: Callback”. No set or get script need be defined for this resource.

This callback must be triggered in the draw file because the behavior of the cursor is defined in that file in the MoveCursor script.

If a developer using this widget class in his interface specifies some actions for this valueChangedCallback resource, they will be executed every time the cursor moves. Otherwise, nothing will happen.

The valueChangedCallback will be used to act upon a modification of the rulerValue or of the position of the cursor. We need to transmit these two parameters through the `call_data` in the `XtCallCallbacks` function call. This will be achieved through the use of a specific structure.

Save your class. Go back to XDrawMaker and edit the **ruler.draw** file. Select the ruler area and open the Script Editing window. Modify the end of the MoveCursor script to be the following:

```
/* Compute a translation to move the cursor */
Float ty = y - ymin;
XnsldrawSetTranslation(parent, RPos, 0.0, ty);

/* Define the rulerData structure */
struct rulerData
{
    Float curpos;
    Float curval;
};
```

```
/*      Initialize the structure and set values to its
        fields */
rulerData rd = new_struct(rulerData);
rd->curpos = y;
rd->curval = parent:rulerValue;

/*      Call the valueChangedCallback with the structure as
        call_data */
XtCallCallbacks(parent, "valueChangedCallback",
                rd);
Free(String(rd));

/*      Show the changes */
XnsldDrawRedisplay(parent);
}
end
```

The `XtCallCallbacks` function calls the `valueChangedCallback` callback with the value of the `rulerData` structure `rd` as `call_data`. Notice that this is a standard Xt function. Xt and Xm functions can also be used within Face scripts. Once this function returns, the `rulerData` structure is no longer needed and can be freed.

You can now save the **ruler.draw** file.

The new widget class named `Ruler` is now ready for use. It is defined both by the **ruler.draw** and the **Ruler.fm** files.

You have learned in this section how to:

1. define a callback resource.
2. trigger a callback upon a modification of the drawing.
3. use structures in Face.

4.6 Use the new class

In this section, you will use XFaceMaker to create a small interface containing standard Motif widgets and an instance of the new Ruler widget class.

Launch the version of XFaceMaker containing the Draw widget. If you saved your preferences as indicated earlier, you should not need to load the Ruler class. Check the toolkit to see if the User Defined widgetstore contains the Ruler icon. If not, load the Ruler class with the **Modules/Classes/Load** menu option. The toolkit pulldown menu should now have a widgetstore named “User Defined”. If you open that widgetstore, you will see the icon representing the Ruler widget. Let us create a sample interface looking like this:

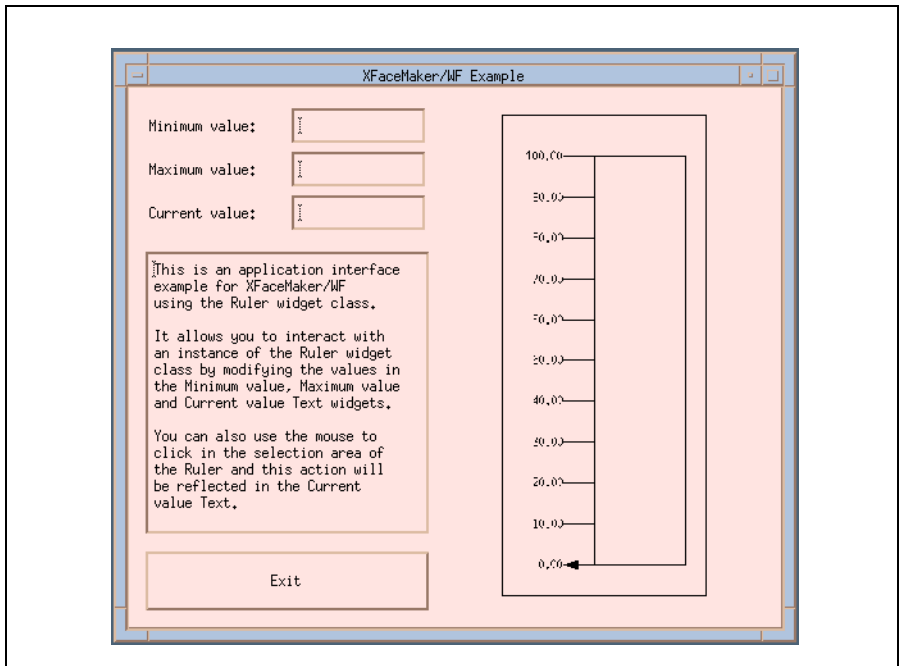


Figure 4.6: The Ruler application

Three text/label widget pairs let the user specify the characteristics of the ruler. The instructions displayed in another Text widget explain what the interface is supposed to do. An Exit PushButton at the bottom is used to exit the application. This is an application interface example for XFaceMaker/WF, using the Ruler widget class.

It allows you to interact with an instance of the Ruler widget by modifying the values in the Minimum value, Maximum value and Current value Text fields. You can also use the mouse to click in the selection area of the Ruler and this action will be reflected in the Current value Text field.

4.6.1 Build the interface

Start by creating this interface, setting the correct attachments and default values and giving an icon name and a title to this example. You might group the label/text wid- get pairs in a rowcolumn, to reduce the number of attachments you need to specify. The suggested widget tree for this interfaceis:

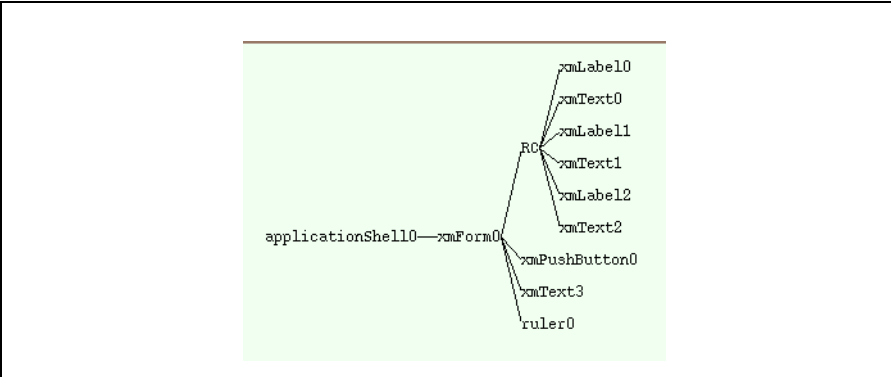


Figure 4.7: Suggested Interface Widget Tree

If you already know how to use XFaceMaker, you should not have any problems. If you do not, you will find XFaceMaker to be rather easy to use. All the necessary wid- gets can be found in the widgetstores of the toolkit, where we also previously found the XnslDraw widget. The resources of each widget can be set by first selecting the widget and then modifying the resources in the Resources window as explained pre- viously.

Create the widgets with these names:

Widget Name	Interface use
applicationShell0	Shell of the interface
xmForm0	Contains the widgets
RC	RowColumn for the text/label pairs

Widget Name	Interface use
xmLabel0	Label for the minimum value
xmText0	Input for the minimum value
xmLabel1	Label for the maximum value
xmText1	Input for the maximum value
xmLabel2	Label for the current value
xmText2	Input for the current value
xmText3	Displays the instructions (not editable)
xmPushButton0	PushButton to exit the application
ruler0	instance of the Ruler class

Once this is done and saved in a file named **interface.fm**, using the *File/Save As* menu option, you can start adding functionality to the interface.

The .fm suffix is used to indicate XFaceMaker files. The .fm file format is similar to the .draw file format and can also be read and modified with a text editor.

4.6.2 Define callback resources for the minimum value widget

Start with the widget xmText0, displaying the minimum value of the ruler0 widget. Set its default value to 0.0. Set the activateCallback of widget xmText0 to:

```
script min_changed:
Float min = Float(ConvertString(self, self:value, "Float"));
/* Get relative path to ruler widget - depends on tree */
Widget w_ruler = parent.parent.ruler0;
w_ruler:rulerMin = min;
Float max = w_ruler:rulerMax;
if (max < min) {
    w_ruler:rulerMax = min + 10.0;
    String str; str = malloc(10);
    sprintf(str, "%7.2f", w_ruler:rulerMax);
    parent.xmText1:value = str;
    free(str);
}
Float v = Float(ConvertString(self, parent.xmText2:value,
                             "Float"));
w_ruler:rulerValue = v;
```

This is a named script because you might want to have the same functionality in the `valueChangedCallback` of widget `xmText0`.

This script gets the value of the Text widget (`self:value`), converts it from type `String` to type `Float` and assigns it to the `rulerMin` resource of widget `ruler0`.

It also checks that the new minimum value is not greater than the maximum value. If that is the case, it forces the maximum value to the minimum value plus 10.0 and updates the `xmText1` widget accordingly.

Since modifying either of the `rulerMin` or `rulerMax` resources leads to a relabeling and redrawing of the scale of the `ruler0` widget, it is necessary to also reposition the arrow showing the current value in the `ruler0` widget to reflect that shown in the `XmText2` widget. That is done by setting the `rulerValue` resource.

4.6.3 Define callback resources for the maximum value widget

The `activateCallback` of widget `xmText1` is constructed on the same principles. Set its default value to 100.0.

Set the `activateCallback` of widget `xmText1` to:

```
script max_changed:
Float max = Float(ConvertString(self, self:value, "Float"));
/* Get relative path to ruler widget - depends on tree */
Widget w_ruler = parent.parent.ruler0;
w_ruler:rulerMax = max;
Float min = w_ruler:rulerMin;
if (min >= max) {
    w_ruler:rulerMin = max - 10.0;
    String str; str = malloc(10);
    sprintf(str, "%7.2f", w_ruler:rulerMin);
    parent.xmText0:value = str;
    free(str);
}
Float v = Float(ConvertString(self, parent.xmText2:value,
                             "Float"));
w_ruler:rulerValue = v;
```

4.6.4 Define callback resources for the current value widget

Set the `activateCallback` of widget `xmText2` to:

```
script cur_changed:
Float v = Float(ConvertString(self, self:value, "Float"));
/* Get relative path to ruler widget - depends on tree */
```

```

Widget w_ruler = parent.parent.ruler0;
if (v > w_ruler:rulerMax) {
    v = w_ruler:rulerMax;
    String str; str = malloc(10);
    sprintf(str, "%7.2f",v);
    self:value = str;
    free(str);
}
if (v < w_ruler:rulerMin) {
    v = w_ruler:rulerMin;
    String str; str = malloc(10);
    sprintf(str, "%7.2f",v);
    self:value = str;
    free(str);
}
w_ruler:rulerValue = v;,,,,,,,,,,,,,

```

This script sets the rulerValue resource of widget ruler0 to the value typed in the xmText2 widget. It checks whether the value typed is within the correct boundaries (the rulerMin and rulerMax resources) and if not it changes the value in the xmText2 widget to reflect either the minimum or the maximum allowed value.

4.6.5 Define callback resources for the Ruler widget

Now since you set up a call to the valueChangedCallback of the ruler0 widget when you click in the selection zone, you need to define this callback. Set the valueChangedCallback of widget ruler0 to:

```

/* Define the rulerData structure */
struct rulerData
{
    Float curpos;
    Float curval;
};
/* Initialize the structure */
rulerData rd = $;
/* Put the current value in the text widget */
String str; str = malloc(10);
sprintf(str, "%7.2f", rd->curval);
parent.RC.xmText2:value = str;
free(str);

```

This callback will update the xmText2 widget with the current value passed in the rulerData structure that is the call_data for this callback.

Until we define the rulerData structure in the class Initialize method,(see “Declare the rulerData Structure” on page 123) we have to define it in the interface in order to use it. Once the structure is properly defined in our class files, we won’t need to declare it in the script as we did here.

4.6.6 Define the PushButton’s Callback

The activateCallback of widget xmPushButton0 has to call the exit function to close the application. But that is not all: the XDraw library we are using may be the development version of the library, which is protected by the license server. When the widget is first instantiated, it takes a license which should be released by calling the **XnslDrawComplete** function when the program exits. The other version of the Draw library, the runtime version, has no license protection so we would not have to call XnslDrawComplete on exit. We can call it anyway, though, since it does nothing in the runtime version of the library. This way, we have the same program for both versions of the library.

We can call XnslDrawComplete either directly from a Face script or from an application function. We will do it from the application, because most applications have some other cleanup tasks to do before exiting.

The activate callback of the xmPushButton0 widget calls a C function:

```
/* declare the Ruler_exit function */
function None Ruler_exit();
/* call it */
Ruler_exit();
```

This function will be defined in the **interfaceApp.c** file described later.

Save your interface in the **interface.fm** file.

4.7 Test the interface in XFaceMaker

Switch to Try mode and move the ruler. The current value is updated in the Text field. Change the values in the Text fields and the ruler changes accordingly.

Check first that XFaceMaker does not output error messages in the message area while you interact with the interface. Then, check that the interface behaves as expected. If errors occur in the interface scripts, correct them and save the interface. If errors occur in the ruler0 object, you have to edit the model of the class: do **Modules/Classes/Edit**, select the Ruler class and the Ruler.fm file. Once the model of the class is loaded, modify the active value scripts, or the .draw file as needed. Once you have made all the corrections, save your class. The instance of the Ruler class in your interface is immediately updated, you can proceed with your tests.

You will notice that the tick labels are not always very legible. This is because a scaling factor is applied to the labels, and to the text inside them, to account for the fact that the widget can be of any size. If we could be sure the labels are always large enough for the values to fit with the default font, we would prevent text scaling: in the XDrawMaker label editing box, unset the **Transform Value** toggle. Do it, and you will see that the values stay crisp but that they are truncated when you reduce the widget's size. That means you should give the user the option to change the font used for the labels.

4.7.1 Add the rulerFont resource

To add this resource, load the model of your Ruler class, select it, and open the Active Values box. Add a resource *rulerFont*, of type *DrawFont* and default value *fixed*. Enter the set script listed below.

```
Int nchildren = self:numChildren;
Widget child[Int];
child = new_c_array(Pointer(self:children),nchildren);
for (i = 0; i < nchildren; i = i+1) {
    w = child[i];
    if (XnslIsDrawLabel(w)) w:drawFont:DrawFont = @;
}
XnslDrawRedisplay(self);
```

Save the modified class, check that you are able to set the font resource.

4.8 Generate the C-code and compile it

In this section, you will generate and compile the C code corresponding both to the Ruler widget class defined in the **Ruler.fm** file and to the interface you have just built in the **interface.fm** file.

4.8.1 Generate the application files

Your application, besides running in try mode inside XFaceMaker, can be generated so as to operate in three different modes:

- interpreted mode - the application uses the .fm file for the graphical user interface.
- compiled mode - the application is compiled and linked with the C version of the graphical user interface.
- editor mode - the application is compiled and linked with the C version of the graphical user interface and with XFaceMaker itself. You get an extended XFaceMaker executable in which any application functions you defined are known. This does not include the class, however, so we will not consider the editor mode in this example. We will make an extended XFaceMaker in a separate section.

When you generate the application files, make sure the interface.fm file is the only file loaded in the design process, i.e. any Ruler class model has been deleted.

Select the **File/Save Appli Files/All** menu option.

XFaceMaker will generate four files:

- **interfaceMain.c**, containing the main function of the application.
- **interfaceApp.c**, containing the skeletons of the application functions you may have declared. You can then fill them in. In our example, the only function is the **Ruler_exit** function.
- **interfaceApp.h**, containing the declarations of the application functions.
- **interfaceMake**, containing the rules to build the application.

You can also choose to generate any of these files individually by choosing one of the options of the **File/Save Appli Files** submenu.

The application files generated have all the code necessary to generate the executables listed above.

4.8.2 Modify the functions file interfaceApp.c

This is where the skeletons of application functions called from Face scripts are placed by XFaceMaker. You just have to fill in the body. In this interface, the only application function is Ruler_exit.

Open the interfaceApp.c file. Before you modify it, it looks like this:

```
/*
 * C file generated by XFaceMaker Version 3.5.
 * Do not remove this comment.
 * Timestamp:          907315675
 */

/*
 * XFaceMaker 3.5 Application Functions Definitions.
 */

#ifdef STANDALONE
#include <Fm.h>
#endif

#ifdef NeedFunctionPrototypes
void Ruler_exit()
#else
void Ruler_exit()

#endif
{
}
```

Add the body of the function:

```
XnslDrawComplete();
exit(0);
```

and save the file.

4.8.3 Generate the executables

If your environment has been set correctly, the LD_LIBRARY_PATH (LIBPATH on IBM, LPATH on HP) environment variable includes the \$NSLHOME/lib directory. You might want to check this before you attempt to compile your interface.

To generate the interpreted mode application, type:

```
make -f interfaceMake interpreted
```

or

```
make -f interfaceMake iinterface
```

To generate the compiled mode application, type:

```
make -f interfaceMake compiled
```

or

```
make -f interfaceMake cinterface
```

4.8.4 Choose the correct executable file

XFaceMaker allows you to make three different executable files for the same application. These executable files run in three possible modes: interpreted, compiled or editor mode. We won't use the editor mode in this example as it is not adapted to adding a new class, but to adding application functions. We'll see later how to add our class to XFaceMaker.

In interpreted mode, the .fm files defining the interface part of your application (this includes the Ruler class definition, not just the interface) are not compiled and linked with the rest of the application. They are instead interpreted at run time by the libFm library, i.e. the .fm files are loaded as such at run time. This allows you to see and test very quickly the newest modifications made to your interface, or class, since you need not recompile and link the application.

In compiled mode, the .fm files defining the interface part of your application, and the Ruler class, are compiled and linked with the rest of the application and with the libFm_c library. The draw part of the Ruler class is still interpreted, though. This is because the ruler.fm file reads the ruler.draw file (we used the drawFile resource in our Ruler class model). As a consequence, the compiled application we are building is linked with the XnsIDf library which contains the Face interpreter. We could have built a fully compiled application, by loading the drawing into XFaceMaker in XFaceMaker format. This mode would allow you to port your application across different platforms, since you can generate a new libFm_c library on the destination platform (you have the source code for the library) and then compile and link your application, starting from the .c files.

4.9 XFaceMaker and the Ruler Extension

Rather than loading our class as a .fm file into XFaceMaker as we have done so far, we can build an XFaceMaker executable that knows the Ruler class directly. We will have to

1. build the class files with the Stand Alone C code option
2. build the widget library
3. build the Ruler.face extension file
4. build and test our extended XFaceMaker

In order to keep this work separate from the application we have built, we will do the extension in a separate directory. Create a directory, e.g. `w_face_extend` and copy `Ruler.bm`, `Ruler.fm` and `ruler.draw` to it. Exit XFaceMaker, change to the new directory, launch `xfmwf`.

4.9.1 Make the Class Include and Source Files

We are going to generate the `Ruler.c`, `Ruler.h`, `RulerP.h` files again but this time, we will set the Stand Alone C code option. In general, when making the C files for a class, you should always set this option: you won't always want to link your class with the `libFm_c` library. For example, you might want to use your class in an interpreted application.

Classes usually have a class prefix, which is prepended to all resource names. This avoids resource name conflicts when you use classes from different origins. We will specify a class prefix for the Ruler class, say `Ewf`.

Load the `Ruler.fm` class (**Modules/Classes/Load**), then *Edit* the class (**Modules/Classes/Edit**). Open the C code options box (**Options/C Options**) and set the following C-Code options:

- set the Save C code toggle
- set the class radio button
- type `Ewf` in the Class Prefix text field
- set the stand alone radio button
- click the Apply button

The box should look like Figure 4.8, “C Code Options for the Ruler Class,” on page 72.

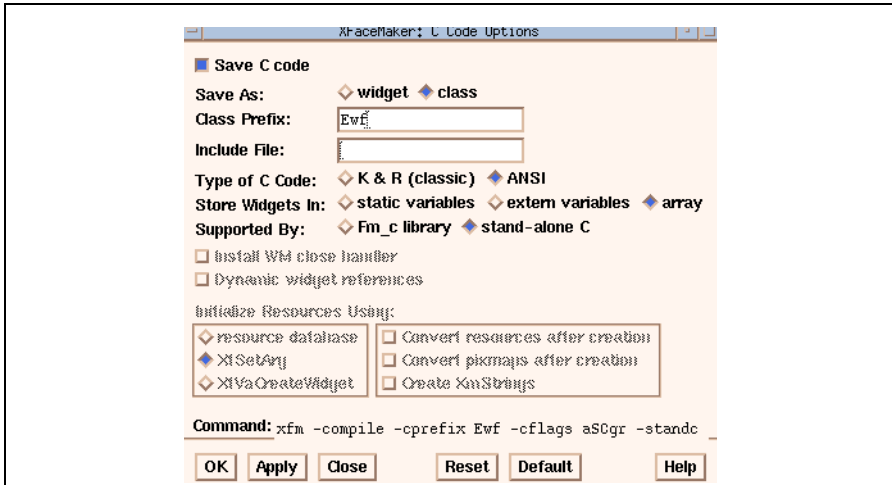


Figure 4.8: C Code Options for the Ruler Class

To generate the desired `Ruler.c`, `Ruler.h` and `RulerP.h`, save the class using the Modules menu: **Modules/Classes/Save**. The files are created in the current directory. Just as the Motif include files are in a directory `Xm`, where `Xm` is the class prefix, we will put our include files in a directory `Ewf`, our chosen class prefix. Create directory `Ewf` in the current directory, and move the two `.h` files just created to `Ewf`. If you have a look at the `Ruler.c` file, you will see that it has an include directive that is `#include <Ewf/RulerP.h>`, i.e. our `Ruler.c` file follows the class prefix convention, as expected. The class name changes from `Ruler` to `EwfRuler` - the prefix is the first part of the class name.

4.9.2 Build the Ruler Widget Library

Now, we have to compile the `Ruler.c` file and build the `libRuler.a` library. The set of commands will look like the following, depending on the operating system you are using.

```
cc -c Ruler.c -I. -I$NSLHOME/include
ar cr libRuler.a Ruler.o
ranlib libRuler.a
```

Again, class prefix conventions will lead us to tell XFaceMaker to fetch this library in the Ewf directory so we have to move our newly created library, `libRuler.a`, to the Ewf directory.

We can do away, now, with `Ruler.o`

4.9.3 Define the Ruler.face Extension File

To make our `EwfRuler` class known to XFaceMaker, we must now create a Face language file that defines the Ruler extension and tells XFaceMaker how to proceed when building applications, i.e., how to generate application Makefiles.

First, since our class uses the Draw widget facilities, it should include the Draw extension.

```
include "XnslDraw.face";
```

Adding the Draw extension ensures that the libraries and includes needed for the Draw part of our widget will be correctly specified.

The rest of the `Ruler.face` file will deal only with the specifics of the Ruler class. The declarations that provide the class information to the XFaceMaker extension mechanism are of the form:

```
devine _XFM_EXT_<NAME>_<EXTENSION_NAME> '<VALUE>';
```

The `EXTENSION_NAME` has to be the same as the name of the extension file, in our case, `Ruler`. The variables that have to be defined are documented in the XFaceMaker User's Guide.

To define the function call that adds the class to XFaceMaker, use the constant `_XFM_EXT_CODE_Ruler`. Set it as:

```
define _XFM_EXT_CODE_Ruler '  
  (*fns->add_widget_class)(  
    &ewfRulerWidgetClass,  
    0,  
    FM_PRIMITIVE,  
    FM_NORMAL,  
    0,  
    "#include <Ewf/Ruler.h>",  
    "ewfRulerWidgetClass",  
    0,  
    0,  
    0,  
    "Ruler.bm");';
```

This does an `FmAddWidgetClass`. It will be used to build a new `XFaceMaker` executable; see below.

Now, define the function name and calling syntax for the function that will be called to extend `XFaceMaker`:

```
define _XFM_EXT_BUILD_INIT_FUNCTION_Ruler    'ERulerInit';
define _XFM_EXT_BUILD_INIT_FUNCALL_Ruler    'ERulerInit();';
```

The function *ERulerInit* will be generated by the `XFaceMaker` extension mechanism. Its body is, in our case, the `FmAddWidgetClass` we defined above. It is needed whenever you build interpreted or editor (new `XFaceMaker` executable) applications. The source will be in a file called `RulerLibExt.c`.

Now, specify the name of the library that will contain this function call:

```
define _XFM_EXT_E_LIBRARY_Ruler            RulerFm';
```

The `libRulerFm` library is the name of the library that is to be built from `RulerLibExt.c` and which, as you will see, is linked with interpreted or editor applications in the `XFaceMaker` Makefiles.

If we had specified external functions for our class, we would have to link compiled applications with that library. The constant in our case is not needed, we define it for the sake of completeness.

```
define _XFM_EXT_C_LIBRARY_Ruler            '';
```

For compiled applications, we need to initialize our widget class. With the line below, we ask `XFaceMaker` to add this function call to the `Main.c` files it generates for compiled applications.

```
define _XFM_EXT_BUILD_C_INIT_FUNCALL_Ruler
    'XtInitializeWidgetClass(ewfRulerWidgetClass);';
```

Now we can specify the include file :

```
define _XFM_EXT_BUILD_INCLUDES_Ruler
    #include <Ewf/Ruler.h>;
```

the libraries with which to link all applications:

```
define _XFM_EXT_BUILD_LIBRARIES_Ruler      '-lRuler';
```

the additional library with which to link interpreted or editor applications:

```
define _XFM_EXT_BUILD_E_LIBRARIES_Ruler    '-lRulerFm';
```


We could also define additional libraries for compiled or interpreted applications. Here, we don't have any such libraries. The corresponding variables are kept for the sake of completeness and are empty:

```
define _XFM_EXT_BUILD_C_LIBRARIES_Ruler    '';
define _XFM_EXT_BUILD_I_LIBRARIES_Ruler    '';
```

The include path, to tell XFaceMaker where to fetch the includes when building applications is specified by:

```
define _XFM_EXT_BUILD_INCLUDE_PATH_Ruler
    '-I. -I$(NSLHOME)/include';
```

and the library path is specified by:

```
XFM_EXT_BUILD_LIBRARY_PATH    '-L./Ewf -L$(NSLHOME)/lib';
```

The complet listing for Ruler.face is:

```
include "XnslDraw.face";

/*****
 * Class information.
 *****/

define _XFM_EXT_CODE_Ruler '
(*fns->add_widget_class)(
&ewfRulerWidgetClass,
0,
FM_PRIMITIVE,
FM_NORMAL,
0,
"#include <Ewf/Ruler.h>",
"ewfRulerWidgetClass",
0,
0,
0,
"Ruler.bm");';

/*****
 * Build information.
 *****/

define _XFM_EXT_C_LIBRARY_Ruler    '';
define _XFM_EXT_E_LIBRARY_Ruler    'RulerFm';
define _XFM_EXT_BUILD_C_INIT_FUNCALL_Ruler
    'XtInitializeWidgetClass(ewfRulerWidgetClass);';
define _XFM_EXT_BUILD_INIT_FUNCTION_Ruler    'ERulerInit';
```

```
define _XFM_EXT_BUILD_INIT_FUNCALL_Ruler    'ERulerInit();';
define _XFM_EXT_BUILD_INCLUDES_Ruler
        '#include <Ewf/Ruler.h>';
define _XFM_EXT_BUILD_LIBRARIES_Ruler      '-lRuler';
define _XFM_EXT_BUILD_C_LIBRARIES_Ruler    '';
define _XFM_EXT_BUILD_I_LIBRARIES_Ruler    '';
define _XFM_EXT_BUILD_E_LIBRARIES_Ruler    '-lRulerFm';
define _XFM_EXT_BUILD_INCLUDE_PATH_Ruler
        '-I. -I$(NSLHOME)/include';
define _XFM_EXT_BUILD_LIBRARY_PATH
        '-L./Ewf -L$(NSLHOME)/lib';
```

4.9.4 Use the Ruler Extension in XFaceMaker

To build the extended XFaceMaker, do **Load Extensions** in the **File** menu. This opens a dialog with the list of extensions available. Click the **Browse...** button to open the File Selector and find the Ruler.face extension. Add it to the list, select it, build a new XFaceMaker executable named **r_xfm** with just the Ruler extension. Watch the different steps it goes through as the commands are printed in the build dialog box. If errors occur, correct any mistakes in the Ruler.face file and, if the errors occurred before the libRuler library was successfully built, remove the Ruler-LibExt files and the libRuler file before you rebuild.

Once the r_xfm executable has been successfully built, the last line of the build box will say *command succeeded*.

Exit xfmwf, launch r_xfm, check that the EwfRuler class is accessible in the *User Defined* Toolkit, that the class name is EwfRuler, that the icon is the Ruler.bm you defined earlier, that all the resources specific to the class are accessible and behave as expected. Have a look at the source files that went into building this XFaceMaker executable to see how the information you provided in the Ruler.face file was used.

Now, copy the interface.fm file to the current directory. Load the file in your favorite text editor, find the string Ruler (the class name of the ruler0 instance) and change it to EwfRuler (the new class name) and save the file. Load it into your extended XFaceMaker. Check that you can set the resources of the ruler object and that the right editing boxes are popped when you click on the ellipsis for each resource.

The last thing we need to do is verify that we can build the compiled and interpreted applications. Generate the application files, edit the interfaceApp.c file to add the body of the Ruler_exit function, build the interpreted and compiled version of the application. As you execute the make command, observe which libraries are linked,

where they are fetched, which include files, etc. You will see that the Draw libraries are in the link line - that is because we included the Draw.face file in our Ruler.face file.

In this section you have seen how to:

1. Specify and use a class prefix.
2. Generate stand alone C code for a class.
3. Build a widget library.
4. Make an extension file for your class.
5. Make an extended XFaceMaker executable with the class extension you have defined.

4.10 Source listings

In the XFaceMaker files, you will see that the lines in the callback scripts are followed by the `\` character to indicate that the callback continues on the next line.

In the XDrawMaker files, you will see that the “ inside a script must be preceded with a `\` because the whole script is surrounded with “ characters.

In a few of these listings, line breaks have been introduced for better readability. Do not copy them when editing or creating your files.

4.10.1 Draw file : ruler.draw

```
{
    drawUnits = no_units;
    drawAngleUnits = no_angle_units;
    drawGridOneOf = 1;
    drawGridSnapTo = True;
}
XnslDrawRectangle RulerBackground {
    drawShowBorder = True;
    drawPoints = "60, 40, 200, 470";
    drawNumPoints = 2;
}
XnslDrawRectangle RulerSelectZone {
    drawShowBorder = True;
    drawEventMask = Button1Down|Button1Motion;
    drawActivateCallbackList = "on Button1Down
script MoveCursor:
/* Retrieve RulerTick0, RulerTick10 and
                                RulerPosition objects */
Widget RMin=XnslDrawNameToObject (parent, \"RulerTick0\");
Widget RMax = XnslDrawNameToObject (parent,
                                \"RulerTick10\");
Widget RPos = XnslDrawNameToObject (parent,
                                \"RulerPosition\");
if (!RMin || !RMax || !RPos)
    printf(\"Ruler Error: Ruler widget id's not found \n\");
else {
/* Get the positions of Tick0 and Tick10 */
    Float ymin = XnslDrawGetCenterY (parent, RMin);
    Float ymax = XnslDrawGetCenterY (parent, RMax);
/* Get the Y position of the mouse */
    Float y = citof( XnslDrawEventY (parent, $));
```

```
/* Test if the click was within limits */
    if (y < ymax) y = ymax;
    if (y > ymin) y = ymin;
/* compute a translation to move the cursor */
    Float ty = y - ymin;
    XnsldrawSetTranslation (parent, RPos, 0.0, ty);
/* Define the rulerData structure */
    struct rulerData
    {
        Float curpos;
        Float curval;
    };
/* Create and initialize the structure */
    rulerData rd = new_struct(rulerData);
    rd -> curpos = y;
    rd -> curval = parent:rulerValue;
/*Call valueChangedCallback with call_data=the structure*/
    XtCallCallbacks(parent, \"valueChangedCallback\", rd);
    Free (String(rd));
/* Show the changes */
    XnsldrawRedisplay (parent);
}
end
on Button1Motion
    script MoveCursor;
end
";
    drawPoints = "150, 80, 90, 400";
    drawNumPoints = 2;
}
XnsldrawLine RulerTick0 {
    drawShowBorder = True;
    drawPoints = "120, 480, 150, 480";
    drawNumPoints = 2;
}
XnsldrawLine RulerTick1 {
    drawShowBorder = True;
    drawPoints = "120, 440, 150, 440";
    drawNumPoints = 2;
}
XnsldrawLine RulerTick2 {
    drawShowBorder = True;
    drawPoints = "120, 400, 150, 400";
```

```
    drawNumPoints = 2;
}
XnslDrawLine RulerTick3 {
    drawShowBorder = True;
    drawPoints = "120, 360, 150, 360";
    drawNumPoints = 2;
}
XnslDrawLine RulerTick4 {
    drawShowBorder = True;
    drawPoints = "120, 320, 150, 320";
    drawNumPoints = 2;
}
XnslDrawLine RulerTick5 {
    drawShowBorder = True;
    drawPoints = "120, 280, 150, 280";
    drawNumPoints = 2;
}
XnslDrawLine RulerTick6 {
    drawShowBorder = True;
    drawPoints = "120, 240, 150, 240";
    drawNumPoints = 2;
}
XnslDrawLine RulerTick7 {
    drawShowBorder = True;
    drawPoints = "120, 200, 150, 200";
    drawNumPoints = 2;
}
XnslDrawLine RulerTick8 {
    drawShowBorder = True;
    drawPoints = "120, 160, 150, 160";
    drawNumPoints = 2;
}
XnslDrawLine RulerTick9 {
    drawShowBorder = True;
    drawPoints = "120, 120, 150, 120";
    drawNumPoints = 2;
}
XnslDrawLabel RulerLabel0 {
    drawShowBorder = False;
    drawSelectCondition = "draw_inside draw_on_border";
    drawPoints = "60, 470, 60, 20";
    drawNumPoints = 2;
    drawLabelString = "0.00";
```

```
        drawLabelAlignment = draw_label_align_east;
        drawLabelTransformString = False;
    }
    XnslDrawLabel RulerLabel1 {
        drawShowBorder = False;
        drawSelectCondition = "draw_inside draw_on_border";
        drawPoints = "60, 430, 60, 20";
        drawNumPoints = 2;
        drawLabelString = "10.00";
        drawLabelAlignment = draw_label_align_east;
        drawLabelTransformString = False;
    }
    XnslDrawLabel RulerLabel2 {
        drawShowBorder = False;
        drawSelectCondition = "draw_inside draw_on_border";
        drawPoints = "60, 390, 60, 20";
        drawNumPoints = 2;
        drawLabelString = "20.00";
        drawLabelAlignment = draw_label_align_east;
        drawLabelTransformString = False;
    }
    XnslDrawLabel RulerLabel3 {
        drawShowBorder = False;
        drawSelectCondition = "draw_inside draw_on_border";
        drawPoints = "60, 350, 60, 20";
        drawNumPoints = 2;
        drawLabelString = "30.00";
        drawLabelAlignment = draw_label_align_east;
        drawLabelTransformString = False;
    }
    XnslDrawLabel RulerLabel4 {
        drawShowBorder = False;
        drawSelectCondition = "draw_inside draw_on_border";
        drawPoints = "60, 310, 60, 20";
        drawNumPoints = 2;
        drawLabelString = "40.00";
        drawLabelAlignment = draw_label_align_east;
        drawLabelTransformString = False;
    }
    XnslDrawLabel RulerLabel5 {
        drawShowBorder = False;
        drawSelectCondition = "draw_inside draw_on_border";
        drawPoints = "60, 270, 60, 20";
```

```
    drawNumPoints = 2;
    drawLabelString = "50.00";
    drawLabelAlignment = draw_label_align_east;
    drawLabelTransformString = False;
}
XnslDrawLabel RulerLabel6 {
    drawShowBorder = False;
    drawSelectCondition = "draw_inside draw_on_border";
    drawPoints = "60, 230, 60, 20";
    drawNumPoints = 2;
    drawLabelString = "60.00";
    drawLabelAlignment = draw_label_align_east;
    drawLabelTransformString = False;
}
XnslDrawLabel RulerLabel7 {
    drawShowBorder = False;
    drawSelectCondition = "draw_inside draw_on_border";
    drawPoints = "60, 190, 60, 20";
    drawNumPoints = 2;
    drawLabelString = "70.00";
    drawLabelAlignment = draw_label_align_east;
    drawLabelTransformString = False;
}
XnslDrawLabel RulerLabel8 {
    drawShowBorder = False;
    drawSelectCondition = "draw_inside draw_on_border";
    drawPoints = "60, 150, 60, 20";
    drawNumPoints = 2;
    drawLabelString = "80.00";
    drawLabelAlignment = draw_label_align_east;
    drawLabelTransformString = False;
}
XnslDrawLabel RulerLabel9 {
    drawShowBorder = False;
    drawSelectCondition = "draw_inside draw_on_border";
    drawPoints = "60, 110, 60, 20";
    drawNumPoints = 2;
    drawLabelString = "90.00";
    drawLabelAlignment = draw_label_align_east;
    drawLabelTransformString = False;
}
XnslDrawLine RulerTick10 {
    drawShowBorder = True;
```



```

        drawPoints = "120, 80, 150, 80";
        drawNumPoints = 2;
    }
    XnslDrawLabel RulerLabel10 {
        drawShowBorder = False;
        drawSelectCondition = "draw_inside draw_on_border";
        drawPoints = "60, 70, 60, 20";
        drawNumPoints = 2;
        drawLabelString = "100.00";
        drawLabelAlignment = draw_label_align_east;
        drawLabelTransformString = False;
    }
    XnslDrawLine RulerPosition {
        drawShowBorder = True;
        drawPoints = "120, 480, 150, 480";
        drawNumPoints = 2;
        drawYTranslation = -1.000000;
        drawArrowDir = draw_from_arrow;
    }
}

```

4.10.2 Ruler Xt class file : Ruler.fm

```

widget XnslDraw {
    flag {
        autosize
        autopos
    }
    activevalue {
        name = rulerPositionColor
        set = Widget pos_wid = XnslDrawNameToObject(self,
            "RulerPosition");\
    }
    if (pos_wid) XnslDrawSetForegroundPixel(self, pos_wid, @);\
    XnslDrawRedisplay(self);
    type = Pixel
    storage = object
    automatic = true
    usage = class
    default_type = String
    default_value = black
}
activevalue {
    name = rulerMin
}

```

```
set = /* Define the function to change the labels */\
function None RulerChangeLabels()\
{
    String str; str = malloc(20);
    /* Change min label */
    Widget label_wid = XnslDrawNameToObject(self,
                                              "RulerLabel0");
    sprintf(str,"%7.2f",Float(self:rulerMin));
    if (label_wid) label_wid:drawLabelString:String = str;

    /* Change max label */
    label_wid = XnslDrawNameToObject(self,"RulerLabel10");
    sprintf(str,"%7.2f",Float(self:rulerMax));
    if (label_wid) label_wid:drawLabelString:String = str;

    /* Change all intermediate labels */
    Float delta = Float(self:rulerMax) - Float(self:rulerMin);
    Float min = Float(self:rulerMin);

    label_wid = XnslDrawNameToObject(self, "RulerLabel1");
    Float value = min + 0.1 * delta;
    sprintf(str,"%7.2f",value);
    if(label_wid) label_wid:drawLabelString = str;

    label_wid = XnslDrawNameToObject(self, "RulerLabel2");
    Float value = min + 0.2 * delta;
    sprintf(str,"%7.2f",value);
    if(label_wid) label_wid:drawLabelString:String = str;\

    label_wid = XnslDrawNameToObject(self, "RulerLabel3");
    Float value = min + 0.3 * delta;
    sprintf(str,"%7.2f",value);
    if(label_wid) label_wid:drawLabelString:String = str;\

    label_wid = XnslDrawNameToObject(self, "RulerLabel4");
    Float value = min + 0.4 * delta;
    sprintf(str,"%7.2f",value);
    if(label_wid) label_wid:drawLabelString:String = str;\

    label_wid = XnslDrawNameToObject(self, "RulerLabel5");
    Float value = min + 0.5 * delta;
    sprintf(str,"%7.2f",value);
    if(label_wid) label_wid:drawLabelString:String = str;\
```

```
label_wid = XnslDrawNameToObject(self, "RulerLabel6");
Float value = min + 0.6 * delta;
sprintf(str,"%7.2f",value);
if(label_wid) label_wid:drawLabelString:String = str;\

label_wid = XnslDrawNameToObject(self, "RulerLabel7");
Float value = min + 0.7 * delta;
sprintf(str,"%7.2f",value);
if(label_wid) label_wid:drawLabelString:String = str;\

label_wid = XnslDrawNameToObject(self, "RulerLabel8");
Float value = min + 0.8 * delta;
sprintf(str,"%7.2f",value);
if(label_wid) label_wid:drawLabelString:String = str;\

label_wid = XnslDrawNameToObject(self, "RulerLabel9");
Float value = min + 0.9 * delta;
sprintf(str,"%7.2f",value);
if(label_wid) label_wid:drawLabelString:String = str;\

/* free the string buffer & redisplay */
free(str);
XnslDrawRedisplay(self);\
}\
/*call the function to change the labels */\
RulerChangeLabels();
    type = Float
    storage = object
    automatic = true
    usage = class
    default_type = String
    default_value = 0.00
}
activevalue {
    name = rulerMax
    set = RulerChangeLabels();
    type = Float
    storage = object
    automatic = true
    usage = class
    default_type = String
    default_value = 100.00
```

```
    }
    activevalue {
        name = rulerValue
        get = /* Retrieve the useful objects & their positions */\
Widget RMin = XnslDrawNameToObject(self,"RulerTick0");\
Widget RMax = XnslDrawNameToObject(self,"RulerTick10");\
Widget RPos = XnslDrawNameToObject(self,"RulerPosition");\
\
if( !RMin || !RMax || !RPos)
    printf("Ruler Error: Ruler widget Ids not found\n");\
else {
    Float ymin = XnslDrawGetCenterY(self,RMin);
    Float ymax = XnslDrawGetCenterY(self,RMax);
\
/* Retrieve the cursor's current Y value */
    Float y = XnslDrawGetTranslationY(self,RPos) + ymin;\
\
/* Compute the value */
    Float min = Float(self:rulerMin);
    Float max = Float(self:rulerMax);
    Float v = min + (y - ymin) * (max - min) / (ymax - ymin);
\
/* Copy it to the resource value */
    @ = v;\
}

    set = /* Retrieve the useful objects & their positions */\
Widget RMin = XnslDrawNameToObject(self,"RulerTick0");\
Widget RMax = XnslDrawNameToObject(self,"RulerTick10");\
Widget RPos = XnslDrawNameToObject(self,"RulerPosition");\
\
if( !RMin || !RMax || !RPos)
    printf("Ruler Error: Ruler widget Ids not found\n");\
else {
    Float ymin = XnslDrawGetCenterY(self,RMin);
    Float ymax = XnslDrawGetCenterY(self,RMax);
    Float y = XnslDrawGetCenterY(self,RPos);
/* Get the resource value contained in the @ variable */
    Float v = Float(@);
/* Check it is valid */
    Float min = Float(self:rulerMin);
    Float max = Float(self:rulerMax);
    if (v > max) v = max;
    if (v < min) v = min;
```

```

    /* compute the translation for the new position */
    Float ty = (v - min) * (ymax - ymin) / (max - min);
    XnslDrawSetTranslation(self,RPos, 0.0,ty);
    /* Redisplay the widget */
XnslDrawRedisplay(self);\
}
    type = Float
    storage = object
    automatic = true
    usage = class
    default_type = String
    default_value = 0.00
}
activevalue {
    name = valueChangedCallback
    type = Callback
    storage = object
    automatic = true
    usage = class
}
activevalue {
    name = rulerFont
    set = Int nchildren = self:numChildren;\
Widget child[Int];\
child = new_c_array(Pointer(self:children),nchildren);\
for (i = 0; i < nchildren; i = i+1) {
    w = child[i];
    if (XnslIsDrawLabel(w)) w:drawFont:DrawFont = @;\
}\
XnslDrawRedisplay(self);
    type = DrawFont
    storage = object
    automatic = true
    usage = class
    default_type = String
    default_value = fixed
}
resources {
    autoResize = true
    drawFile = ruler.draw
}
} xnslDraw0

```

4.10.3 Interface example --- Fm file : interface.fm

```
widget ApplicationShell {
  flag {
    mapped
  }
  resources {
    x = 313
    y = 4
    width = 441
    height = 421
    title = XFaceMaker/WF Example
  }
  children {
    widget XmForm {
      flag {
        mapped
        autosize
        autopos
      }
      resources {
      }
      children {
        widget XmRowColumn {
          flag {
            mapped
            autosize
            autopos
          }
          resources {
            leftAttachment = attach_form
            leftOffset = 12
            topAttachment = attach_form
            topOffset = 12
            orientation = horizontal
            packing = pack_column
            numColumns = 3
          }
          children {
            widget XmLabel {
              flag {
```

```

        mapped
        autosize
        autopos
    }
    resources {
        labelString = Minimum value:
    }
} xmLabel0
widget XmText {
    flag {
        mapped
        autosize
        autopos
    }
    resources {
        columns = 15
        value = 0.0
activateCallback = script min_changed:\
Float min = Float(ConvertString(self,
                                self: value, "Float"));\
/* Get relative path to ruler widget - depends on tree */\
Widget w_ruler = parent.parent.ruler0;\
w_ruler:rulerMin = min;\
Float max = w_ruler:rulerMax;\
if (max < min) {
    w_ruler:rulerMax = min + 10.0;
    String str; str = malloc(10);
    sprintf(str, "%.2f", w_ruler:rulerMax);
    parent.xmText1: value = str;
    free(str);\
}\
Float v ;\
v = Float(ConvertString(self,
                        parent.xmText2: value, "Float"));\
w_ruler:rulerValue = v;
    }
} xmText0
widget XmLabel {
    flag {
        mapped
        autosize
        autopos
    }
}

```

```
        resources {
            labelString = Maximum value:
        }
    } xmLabel1
    widget XmText {
        flag {
            mapped
            autosize
            autopos
        }
        resources {
            columns = 15
            value = 100.0
        }
        activateCallback = script max_changed:\
        Float max = Float(ConvertString(self,
            self:value, "Float"));\
/* Get relative path to ruler widget - depends on tree */\
Widget w_ruler = parent.parent.ruler0;\
w_ruler:rulerMax = max;\
Float min = w_ruler:rulerMin;\
if (min >= max) {
    w_ruler:rulerMin = max - 10.0;
    String str; str = malloc(10);
    sprintf(str, "%7.2f",w_ruler:rulerMin);
    parent.xmText0:value = str;
    free(str);\
}\
Float v = Float(ConvertString(self,
    parent.xmText2:value, "Float"));\
w_ruler:rulerValue = v;
    }
} xmText1
widget XmLabel {
    flag {
        mapped
        autosize
        autopos
    }
    resources {
        labelString = Current value:
    }
} xmLabel2
widget XmText {
```

```

        flag {
            mapped
            autosize
            autopos
        }
        resources {
            columns = 15
            activateCallback = script cur_changed:\
            Float v = Float(ConvertString(self,
                                   self:value, "Float"));\\
/* Get relative path to ruler widget - depends on tree */\\
            Widget w_ruler = parent.parent.ruler0;\\
            if (v > w_ruler:rulerMax) {
                v = w_ruler:rulerMax;
                String str; str = malloc(10);
                sprintf(str, "%7.2f",v);
                self:value = str;
                free(str);\\
            }\\
            if (v < w_ruler:rulerMin) {
                v = w_ruler:rulerMin;
                String str; str = malloc(10);
                sprintf(str, "%7.2f",v);
                self:value = str;
                free(str);\\
            }\\
            w_ruler:rulerValue = v;
                                   value = 0.00
        }
    } xmText2
}
} RC
widget XmPushButton {
    flag {
        mapped
        autopos
    }
    resources {
        bottomAttachment = attach_form
        bottomOffset = 12
        leftAttachment = attach_opposite_widget
        leftWidget = RC
        rightAttachment = attach_opposite_widget

```

```
        rightWidget = RC
        height = 50
        width = 225
        labelString = Exit
    activateCallback = /* declare the Ruler_exit function */\
    function None Ruler_exit();\
/* call it */\
    Ruler_exit();
}
} xmPushButton0
widget XmText {
    flag {
        mapped
        autosize
        autopos
    }
    resources {
        editMode = multi_line_edit
        topAttachment = attach_widget
        topOffset = 10
        topWidget = RC
        leftAttachment = attach_opposite_widget
        leftWidget = RC
        rightAttachment = attach_opposite_widget
        rightWidget = RC
        bottomAttachment = attach_widget
        bottomOffset = 10
        bottomWidget = xmPushButton0
    }
    value = This is an application interface\

```

```
example for XFaceMaker/WF \
using the Ruler widget class.\

```

It allows you to interact with \
an instance of the Ruler widget\
class by modifying the values in\
the Minimum value, Maximum value \
and Current value Text widgets.\

You can also use the mouse to \
click in the selection area of\
the Ruler and this action will \
be reflected in the Current\
value Text.

```

        editable = false
        traversalOn = false
    }
} xmText3
widget Ruler {
    flag {
        mapped
        autopos
    }
    resources {
        rightAttachment = attach_form
        rightOffset = 17
        topAttachment = attach_form
        topOffset = 12
        bottomAttachment = attach_form
        bottomOffset = 16
        height = 393
        width = 175
        leftAttachment = attach_widget
        leftOffset = 12
        leftWidget = RC
        valueChangedCallback = /* Define the rulerData structure */\
struct rulerData\
{
    Float curpos;
    Float curval;\
};\
/* Initialize the structure */\
rulerData rd = $;\
/* Put the current value in the text widget */\
String str; str = malloc(10);\
sprintf(str, "%7.2f", rd->curval);\
parent.RC.xmText2:value = str;\
free(str);
    }
} ruler0
}
} xmForm0
}
} applicationShell0

```

In this file, Ruler has to be changed to EwfRuler when the interface is loaded in the extended XFaceMaker - because we defined an Ewf class prefix, the class name changed to EwfRuler.

4.10.4 Interface example --- Include file : interfaceApp.h

```
/*
 * C file generated by XFaceMaker Version 3.5.
 * Do not remove this comment.
 * Timestamp:          907315675
 */

/*
 * XFaceMaker 3.5 Application Functions Declarations.
 */

#ifndef STANDALONE
#include <Fm.h>
#endif
#include <Xnsl/Stand.h>
#include <Xnsl/DAll.h>

#if NeedFunctionPrototypes
extern void Ruler_exit();
#else /* NeedFunctionPrototypes */
extern void Ruler_exit();
#endif /* NeedFunctionPrototypes */
```

4.10.5 Interface example --- Functions file : interfaceApp.c

```
/*
 * C file generated by XFaceMaker Version 3.5.
 * Do not remove this comment.
 * Timestamp:          907315675
 */

/*
 * XFaceMaker 3.5 Application Functions Definitions.
 */

#ifndef STANDALONE
#include <Fm.h>
#endif

#if NeedFunctionPrototypes
void Ruler_exit()
#else
void Ruler_exit()
```

```
#endif
{
    XnslDrawComplete();
    exit(0);
}
```

4.10.6 Interface example --- Main file : interfaceMain.c

```
/*
 * C file generated by XFaceMaker Version 3.5.
 * Do not remove this comment.
 * Timestamp:          907315675
 */

/*
 * XFaceMaker 3.5 Application Main Function
 */

#include <stdio.h>
#ifndef STANDALONE
#include <Fm.h>
#endif

#include "interfaceApp.h"

#ifndef REMOTE

#ifdef CCODE
#ifdef NeedFunctionPrototypes
extern Widget FmCreateinterface(String,Widget,
                                ArgList,Cardinal);
#else /* NeedFunctionPrototypes */
extern Widget FmCreateinterface();
#endif /* NeedFunctionPrototypes */
#endif

#endif /* REMOTE */

#ifdef REMOTE

#ifdef NeedFunctionPrototypes
extern int FaceIPCCoconnectApplication(Widget, char*);
```

```
#else /* NeedFunctionPrototypes */
extern int FaceIPCCoconnectApplication();
#endif /* NeedFunctionPrototypes */

static XrmOptionDescRec AddressOptions[] = {
{"-host", "*xfmRemoteHost", XrmoptionSepArg},
{"-port", "*xfmRemotePort", XrmoptionSepArg},
};

#define options AddressOptions
#define num_options XtNumber(AddressOptions)

#else /* REMOTE */

#define options 0
#define num_options 0

#endif /* REMOTE */

main(
#if NeedFunctionPrototypes
int argc, char **argv)
#else
argc, argv)
int argc;
char **argv;
#endif
{
Widget toplevel, parent;
Widget w[1];
int i;
XtAppContext app_context;

/*
 * Initialize toolkit.
 */
#if defined(CCODE) && defined(STANDALONE)
toplevel = XtAppInitialize(&app_context, "Interface",
options, num_options, &argc, argv, 0, 0, 0);
#else
toplevel = FmAppInitialize(&app_context, "Interface",
options, num_options, &argc, argv, 0, 0, 0);
#endif
}
```

```
XnslRegisterConverters();
XnslDrawInit(toplevel);

#ifdef CCODE
/*
 * Call initialization routines for extension libraries.
 */
FmInitStand(FmGetFunctionsVector());
XnslInitDrawLibrary(toplevel);

/*
 * Attach application functions.
 */
FmAttachFunction("Ruler_exit", (FaceFunctionPtr)Ruler_exit,
                 "None", 0);
#endif CALLEDITOR
/*
 * Load user-defined classes.
 */
FmLoadWidgetClass("Ruler", "Ruler.fm");
#endif
#endif

#ifdef STANDALONE
/*
 * Attach active values.
 */
#endif

#ifdef REMOTE

#ifdef CCODE

#ifdef STANDALONE
/*
 * Register creation functions for dynamic creation
 * in FACE scripts.
 */
FmRegisterCreateFunction("interface.fm",
                         (FmCreationFunction)FmCreateinterface);
#endif /* STANDALONE */
```

```
/*
 * Call interface creation functions.
 */
w[0] = toplevel = FmCreateinterface(0, toplevel, 0, 0);
#else
#ifndef CALLEDITOR
/*
 * Load interface files.
 */
w[0] = toplevel = FmLoadCreate(0,
    "/User/wf_tutorial/interface.fm", toplevel, 0, 0);
#endif
#endif

#endif /* REMOTE */

#ifdef REMOTE

FaceIPCCConnectApplication(toplevel, argv[0]);

#endif /* REMOTE */

#ifndef REMOTE

/*
 * Show widgets.
 */
#ifndef CALLEDITOR
#ifdef STANDALONE
XtRealizeWidget(w[0]);
#else
FmShowWidget(w[0]);
#endif
#endif

#endif /* REMOTE */

/*
 * Enter main event loop.
 */
#ifdef CALLEDITOR
FmCallEditor();
#else
```



```
#ifdef STANDALONE
XtAppMainLoop(app_context);
#else
FmLoop();
#endif
#endif
}
```

4.10.7 Interface example --- Makefile : interfaceMake

```
#
# File generated by XFaceMaker Version 3.5.
# Do not remove this comment.
# Timestamp:          907315675
#

#
# XFaceMaker 3.5 Application Makefile.
#

        XFM = xfmwf
        XFMCFLAGS = -cflags "caC"
        XFMCLFLAGS = -cflags "caCg"

MOTIF_INCPATH =
MOTIF_LIBPATH =
MOTIF_LIBNAME = -lXm

        XT_INCPATH =
        XT_LIBPATH =
        XT_LIBNAME = -lXt

        XLIB_INCPATH =
        XLIB_LIBPATH =
        XLIB_LIBNAME = -lX11

        XEXT_INCPATH =
        XEXT_LIBPATH =
        XEXT_LIBNAME = -lXext

XFM_I_LIBNAME = -lFm
XFM_C_LIBNAME = -lFm_c
XNSLCTRL_LIBNAME = -lXnsLctrl
XFM_E_LIBNAME = -lFm_e
```

```
EXTRA_INCPATH = -I../Fm -I$(NSLHOME)/include -I.. \
               -I../Xnsl -I$(NSLHOME)/include $(LOC_INCPATH)
EXTRA_LIBPATH = -L../Fm -L$(NSLHOME)/lib -L../Lib \
               -L$(NSLHOME)/lib $(LOC_LIBPATH)
EXTRA_I_LIBS = -lXnslFm -lXnslDraw -lXnslTiff -lXnslStand
EXTRA_C_LIBS = -lXnslDraw -lXnslTiff -lXnslDf -lXnslStand
EXTRA_E_LIBS = -lXnslFm -lXnslDraw -lXnslTiff -lXnslStand

XM_INCPATH = $(MRM_INCPATH) $(MOTIF_INCPATH) \
             $(XT_INCPATH) $(XEXT_INCPATH) $(XLIB_INCPATH)

A_INCPATH = -I. $(EXTRA_INCPATH) $(XM_INCPATH)
E_INCPATH = -I. $(EXTRA_INCPATH) $(XM_INCPATH)
X_LIBS = $(XT_LIBPATH) $(XT_LIBNAME) $(XEXT_LIBPATH) \
         $(XEXT_LIBNAME) $(XLIB_LIBPATH) $(XLIB_LIBNAME)
MOTIF_LIBS = $(MRM_LIBPATH) $(MRM_LIBNAME) \
             $(MOTIF_LIBPATH) $(MOTIF_LIBNAME) $(X_LIBS)

SYS_LIBS =

SYS_CFLAGS =
SYS_LDFLAGS =
CDEBUGFLAGS =

CCODEOPTS = -DCCODE

STD_LIBS = $(MOTIF_LIBS) -lm $(SYS_LIBS)

A_LIBPATH = -L. $(EXTRA_LIBPATH)
E_LIBPATH = -L. $(EXTRA_LIBPATH)

XFM_I_LIBS = $(XFM_I_LIBNAME)
XFM_C_LIBS = $(XFM_C_LIBNAME) $(XNSLCTRL_LIBNAME)
XFM_E_LIBS = $(XFM_E_LIBNAME)

A_CFLAGS = $(A_INCPATH) $(SYS_CFLAGS) $(CDEBUGFLAGS)
E_CFLAGS = $(A_INCPATH) $(SYS_CFLAGS) $(CDEBUGFLAGS)
CFLAGS = $(E_CFLAGS)

A_LDFLAGS = $(A_LIBPATH) $(SYS_LDFLAGS) $(CDEBUGFLAGS)
E_LDFLAGS = $(E_LIBPATH) $(SYS_LDFLAGS) $(CDEBUGFLAGS)
```

```
RM = rm -rf
CP = cp
LN = ln -s

XFM_TARGET_OS = -solaris
XFM_TARGET_MK=$(NSLHOME)/lib/xfm/GenAppli/
                                     Mk$(XFM_TARGET_OS)
include $(XFM_TARGET_MK)

all:: interpreted compiled

interpreted:: iinterface

compiled:: cinterface

editor:: einterface

remote:: rinterface

iinterface: iinterfaceMain.o interfaceApp.o
@sh -c "if [ -f $$ ] ; then mv $$ $$bak ; $(RM) $$bak ; fi"
$(CC) $(A_LDFLAGS) -o iinterface iinterfaceMain.o \
    interfaceApp.o \
$(XFM_I_LIBS) $(EXTRA_I_LIBS) $(STD_LIBS)

rinterface: rinterfaceMain.o interfaceApp.o
@sh -c "if [ -f $$ ] ; then mv $$ $$bak ; $(RM) $$bak ; fi"
$(CC) $(A_LDFLAGS) -o rinterface \
    rinterfaceMain.o interfaceApp.o \
$(XFM_I_LIBS) $(EXTRA_I_LIBS) $(STD_LIBS)

cinterface: cinterfaceMain.o interfaceApp.o /User/
                                     wf_tutorial/interface.o Ruler.o
@sh -c "if [ -f $$ ] ; then mv $$ $$bak ; $(RM) $$bak ; fi"
$(CC) $(A_LDFLAGS) -o cinterface cinterfaceMain.o \
    interfaceApp.o \
    /User/wf_tutorial/interface.o Ruler.o \
$(XFM_C_LIBS) $(EXTRA_C_LIBS) $(STD_LIBS)

einterface: einterfaceMain.o interfaceApp.o
@sh -c "if [ -f $$ ] ; then mv $$ $$bak ; $(RM) $$bak ; fi"
$(CC) $(E_LDFLAGS) -o einterface einterfaceMain.o \
    interfaceApp.o \
```

```
$(XFM_E_LIBS) $(EXTRA_E_LIBS) $(STD_LIBS)

iinterfaceMain.o: interfaceMain.c
$(CC) $(A_CFLAGS) -c interfaceMain.c
mv interfaceMain.o iinterfaceMain.o

rinterfaceMain.o: interfaceMain.c
$(CC) $(A_CFLAGS) -DREMOTE -c interfaceMain.c
mv interfaceMain.o rintterfaceMain.o

cinterfaceMain.o: interfaceMain.c
$(CC) $(A_CFLAGS) $(CCODEOPTS) -c interfaceMain.c
mv interfaceMain.o cinterfaceMain.o

einterfaceMain.o: interfaceMain.c
$(CC) $(E_CFLAGS) -DCALLEDITOR -c interfaceMain.c
mv interfaceMain.o einterfaceMain.o

/User/wf_tutorial/interface.c: /User/wf_tutorial/
                                     interface.fm Ruler.h
$(XFM) -compile /User/wf_tutorial/interface.fm \
    -cinclude \"interfaceApp.h\" \
    -classes \"Ruler:./Ruler.fm\" \
    -cfile /User/wf_tutorial/interface.c -cname interface \
    $(XFMCFLAGS)

applifiles:
$(XFM) -gen /User/wf_tutorial/interface.fm \
    -classes \"Ruler:./Ruler.fm\" \
    -cfile /User/wf_tutorial/interface.c -cname interface \
    $(XFMCFLAGS)

clean::
$(RM) *.o iinterface einterface cinterface core emergency.fm

cleanall:: clean
$(RM) /User/wf_tutorial/interface.c *~

/User/wf_tutorial/interface.o: /User/wf_tutorial/
                                     interface.c
$(RM) /User/wf_tutorial/interface.o
$(CC) $(E_CFLAGS) -c /User/wf_tutorial/interface.c
```

```

-mv interface.o /User/wf_tutorial/interface.o

clean::
$(RM) /User/wf_tutorial/interface.o

Ruler.c Ruler.h RulerP.h: ./Ruler.fm
$(XFM) -compile Ruler.fm -cfile Ruler.c $(XFMCLFLAGS)

#
# Windows source files generation:
#
    WINDOWSDIR = Windows
    WINOUTDIR = WinOut

    WINAPPPFILES = \
$(WINDOWSDIR)/interface.mak \
$(WINDOWSDIR)/interface.h \
$(WINDOWSDIR)/interface.cpp \
$(WINDOWSDIR)/interface.rc \
$(WINDOWSDIR)/interfaceApp.h \
$(WINDOWSDIR)/interfaceApp.cpp \
$(WINDOWSDIR)/interfaceRes.h \
$(WINDOWSDIR)/stdafx.cpp \
$(WINDOWSDIR)/stdafx.h

    WININTERFACEMODFILES = \
$(WINDOWSDIR)/interfaceXfm.h \
$(WINDOWSDIR)/interfaceXfm.cpp \
$(WINDOWSDIR)/interfaceXfm.rc \
$(WINDOWSDIR)/interfaceFmr.h

$(WINAPPPFILES) $(WININTERFACEMODFILES):: /User/wf_tutorial/
                                                    interface.fm
$(XFM) -target Windows /User/wf_tutorial/interface.fm
                                                    -targetfile $@

@echo Converting $@...
@xfmwincvt $@

windows:: /User/wf_tutorial/interface.fm
$(XFM) -target Windows <interface.fm
@echo Converting files...
@xfmwincvt

```

```
clean::
$(RM) $(WINDOWS_DIR)/.last.* $(WINOUTDIR)/

always:

cleanall:: clean
$(RM) $(WINOUTDIR)
```

4.10.8 Extend XFaceMaker - Main file: r_xfmXfmMain.c

```
/*
 * C file generated by XFaceMaker Version 3.5.
 * Do not remove this comment.
 * Timestamp:          907748915
 */

/*
 * Main program for building a new instance of XFaceMaker
 *
 *
 */

#include <stdio.h>
#include <Fm.h>

#if NeedFunctionPrototypes
extern void ERulerInit(void);
#else
extern void ERulerInit();
#endif

main(argc, argv)
int argc;
char **argv;
{
    Widget toplevel, parent;
    toplevel = FmAppInitialize(0, "R_xfm", 0, 0, &argc,
                              argv, 0, 0, 0);

#ifdef CCODE
    FmInitStand(FmGetFunctionsVector());
    XnslInitDrawLibrary(toplevel);
    ERulerInit();
#endif
}
```

```
#endif  
  
    FmCallEditor();  
}
```

CHAPTER 5: Advanced examples

This chapter describes two examples using advanced features of XFaceMaker/WF.

1. The Ruler widget class: Run time initialization.

In the previous chapter, the **Ruler** widget class example uses a draw file containing the complete drawing of the ruler. The scale is fixed to ten tick intervals, i.e. eleven tick marks and eleven labels. The labels can be changed to adapt to the minimum and maximum values set by the user, but their number cannot be changed.

It is also possible to use an **Initialize** resource when you create the new widget class to initialize the widget when it is created at run time.

This example describes the modifications to make to the **Ruler** widget class to enable the creation of the scale during initialization at run time with as many ticks, sub-ticks and labels as the user wishes (the number of labels being the same as that of the ticks and one more than the number of tick intervals that the user will specify).

2. The Ruler widget class : FM only version.

In the previous chapter, the **Ruler** widget class example used the **drawFile** resource to associate the drawing's data file (**ruler.draw**) generated by **XDrawMaker** to the Draw widget of the new class being created.

It is also possible to generate .fm files within **XDrawMaker**, which can then be read into **XFaceMaker** directly. In this case, the drawing's objects are available in **XFaceMaker** as gadgets and their resources can be modified directly in the resource window.

This example describes the creation of the **Ruler** widget class when using the **Ruler.fm** file generated by **XDrawMaker** instead of using the **ruler.draw** file.

5.1 Run Time Initialization

In the previous chapter, you have created a drawing in **XDrawMaker**, a widget class based on this drawing in **XFaceMaker** and added resources to give it some functionality, then you have written an application which uses an instance of this new widget class. .

This application is able to change the labels of the **Ruler** when the minimum and maximum values change, but it cannot change the number of divisions and subdivisions because these are hard-coded in the original drawing of the widget.

You could change these values if you could modify the looks of the drawing while your application is running. This can be done through the use of functions and of the **Initialize** resource.

You will now modify your **Ruler** widget class to allow the generation and modification of the scale and labels at run time. To keep things separate, create a new directory in which you will do this work, say `w_scale`. Copy the files `ruler.draw`, `Ruler.fm`, `Ruler.bm` and `interface.fm` into this directory. When you modify `.fm` files (the class and the application) do it from `xfmwf`, rather than using the `r_xfm` you built earlier. You can build a new extended **XFaceMaker** with the new class definition at the end of this section, if you like.

5.1.1 Modify the drawing in XDrawMaker

To modify the drawing in **XDrawMaker**, you simply open the **ruler.draw** file in **XDrawMaker** through the ***File/Open*** menu option.

Since the scale will be created during initialization, you can remove all the **RulerTickn** and **RulerLabeln** objects, except **RulerLabel0**.

To remove an object, you select it either in the drawing or in the **Tree** window and use the **Edit/Delete** menu option.

You should have only four objects in your drawfile:

- **RulerBackground**
- **RulerLabel0**
- **RulerSelectZone**
- **RulerPosition**

Since there is only one label in your drawing, rename the **RulerLabel0** object to **RulerLabel**. Change its **drawLabelString** resource to hide the label by removing the text of the label in the Label Box.

The script associated to the **RulerSelectZone** object needs to be modified since some of the widgets it referenced are no longer available. The minimum and maximum positions can be determined from the bottom and the top of the **RulerSelectZone** object since the division lines no longer exist. The **XnslDrawFaceGetObjectXPoints** returns a face array containing the coordinates of the characteristic points of the referenced object in the form of x,y pairs. The second value gives the upper left corner's y value and the sixth gives the lower right's y value.

The script for the **RulerSelectZone** object becomes:

```
on Button1Down
  script MoveCursor:
    /* Get the positions of the Min and Max divisions */
    Int a[Int];
    a = XnslDrawFaceGetObjectXPoints(parent, self);
    Float ymin = citof(a[5]);
    Float ymax = citof(a[1]);
    destroy(a);
  /* Get the Y position of the mouse */
  Float y = citof( XnslDrawEventY (parent, $));
  /* Test if the click was within limits */
  if (y < ymax) y = ymax;
  if (y > ymin) y = ymin;
  /* compute a translation to move the cursor */
  Float ty = y - ymin;
  Widget RPos = XnslDrawNameToObject(parent,
                                     "RulerPosition");
  if (RPos) XnslDrawSetTranslation (parent, RPos, 0.0, ty);
  else printf("Ruler error:
              RulerPosition widget ID not found\n");
  /* Define the rulerData structure */
  struct rulerData
  {
    Float curpos;
    Float curval;
  };
  /* Create and initialize the structure */
  rulerData rd = new_struct(rulerData);
  rd -> curpos = y;
  rd -> curval = parent:rulerValue;
  /* Call valueChangedCallback call_data = the structure */
  XtCallCallbacks(parent, "valueChangedCallback", rd);
  Free (String(rd));
```

```
/* Show the changes */  
XnslDrawRedisplay (parent);  
end  
on Button1Motion  
    script MoveCursor;  
end
```

Once these modifications are done, save your draw file again. It should look like the following figure:

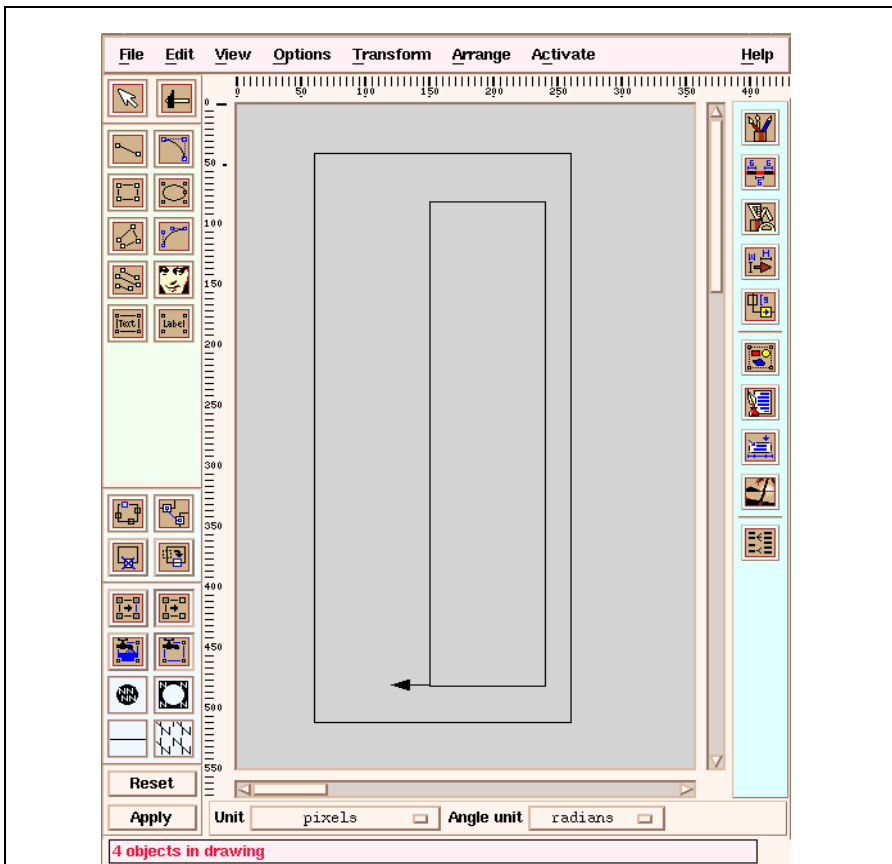


Figure 5.1: The modified Ruler widget

5.1.2 Modify the Ruler class

To modify the **Ruler** class in **XFaceMaker**, load the **Ruler** class with the *Modules/Classes/Load* menu option. Don't forget to start from `xfmwf` rather than the extended **XFaceMaker** you built earlier. Edit the **Ruler** class with the *Modules/Classes/Edit* menu option. Open the *Active Values* window with the draw widget selected to modify the active values.

The biggest modification in the **Ruler** class comes from the generation and modification of the scale and labels at run time. This operation is achieved through the use of the **Initialize** resource which is called when the widget is instantiated. In this resource, you need to create the scale and the labels with some default values. You also need to prepare for the fact that the user will want to modify these values interactively and thus you need to be able to delete the current scale and labels to create new ones corresponding to the new values.

To create the scale, you will define two new resources:

- `rulerTicks`
- `rulerSubTicks`

These two resources are of type **Int**, can have respectively 10 and 5 as default values and have the following set scripts.

Set script for **rulerTicks**:

```
if (!@ || @ < 2) @ = 2;  
RulerRecomputeScale(RULER_BOTH);
```

Set script for **rulerSubTicks**:

```
if (!@ || @ < 1) @ = 1;  
RulerRecomputeScale(RULER_SCALE);
```

These two scripts use the `@` variable, which refers to the resource's new value. The first line of each script is there to initialize the resource if it does not have a value (`!@`) and to reset the value to the minimum allowed value if it has been set lower than the minimum (`@ < 2` or `@ < 1`). The **rulerTicks** resource will have at least a value of 2 and the **rulerSubTicks** a value of 1.

These two scripts call the function **RulerRecomputeScale** which you will define in the **Initialize** resource. This function takes one parameter to specify what needs to be recomputed: the scale (`RULER_SCALE`), the labels (`RULER_LABEL`) or both (`RULER_BOTH`).

When you change the number of ticks, you need to recreate both the scale and the labels. When you change the number of sub ticks, you only need to recreate the scale since no labels correspond to the sub ticks.

You will define a resource named **rulerFormat** to allow the user to specify the format of the labels. Set its type to **String** and its default value to `%7.2f`.

The set script for this resource is:

```
if (!@) @ = "%7.2f";
RulerRecomputeScale(RULER_LABEL);
```

When the format of the labels is changed, only the labels will be recreated through the use of the `RULER_LABEL` parameter.

The **rulerMin** and **rulerMax** resources need to modify the label part of the scale. Change the **rulerMin** and **rulerMax** set scripts to:

```
RulerRecomputeScale(RULER_LABEL);
```

You can also change the default values of the **rulerMin** and **rulerMax** resources to `-50.0` and `50.0` to use both negative and positive values in the scale's labels.

The **rulerValue** resource's get and set scripts also need some rewriting.

The get script should become:

```
/*Retrieve the min and max Y positions */
Widget RSel = XnsldrawNameToObj(self,"RulerSelectZone");
if (!RSel) printf("Ruler Error: RulerSelectZone widget ID
                                not found\n");
else {
    Int a[Int];
    a = XnsldrawFaceGetObjXPoints(self,RSel);
    Float ymin = citof(a[5]);
    Float ymax = citof(a[1]);
    destroy(a);
/* Retrieve the cursor's current Y value */
    Widget RPos = XnsldrawNameToObj(self,"RulerPosition");
    if (!RPos) printf("Ruler Error: RulerPosition widget ID
                                not found\n");
    else {
        Float y = XnsldrawGetTranslationY(self,RPos) + ymin;

/* Compute the value */
        Float min = Float(self:rulerMin);
        Float max = Float(self:rulerMax);
```

```
    Float v = min + (y - ymin) * (max - min) / (ymax - ymin);  
/* Copy it to the resource value */  
    @ = v;  
}  
}
```

The Set script for **rulerValue** should become:

```
/*Retrieve the min and max Y positions */  
Widget RSel = XnslDrawNameToObject(self,"RulerSelectZone");  
if (!RSel) printf("Ruler Error: RulerSelectZone widget ID  
                    not found\n");  
else {  
    Int a[Int];  
    a = XnslDrawFaceGetObjectXPoints(self,RSel);  
    Float ymin = citof(a[5]);  
    Float ymax = citof(a[1]);  
    destroy(a);  
/* Retrieve the cursor's current Y value */  
    Widget RPos = XnslDrawNameToObject(self,"RulerPosition");  
    if (!RPos) printf("Ruler Error: RulerPosition widget ID  
                    not found\n");  
    else {  
        Float y = XnslDrawGetCenterY(self,RPos);  
/* Get the resource value contained in the @ variable */  
        Float v = Float(@);  
/* Check it is valid */  
        Float min = Float(self:rulerMin);  
        Float max = Float(self:rulerMax);  
        if (v > max) v = max;  
        if (v < min) v = min;  
/* compute the translation for the new position */  
        Float ty = (v - min) * (ymax - ymin) / (max - min);  
        XnslDrawSetTranslation(self,RPos, 0.0, ty);  
/* Redisplay the widget */  
        XnslDrawRedisplay(self);  
    }  
}
```

You must now create the **Initialize** resource, which is a predefined method, so you will find it in the list shown by the **Methods...** button in the *Active Values* window. The **Initialize** active value will be the first to be executed when the widget is initial-

ized. The **Initialize** active value should also be the first to be read when parsing the **Ruler.fm** file since it will define functions used by other active values. It will avoid unnecessary warnings in the message area of **XFaceMaker**.

To modify the order of the active values in the active values list, you select the active value whose position you want to change and click on the up/down arrows in the lower part of the box until the active value has reached the desired position. Once you have defined the Initialize active value, you can move it to the top of the list by clicking once on the down arrow.

The **Initialize** set script is the following:

```
type ScaleType = [RULER_]scale, [RULER_]label, [RULER_]both;

function None RulerCreateScale(ScaleType type)
{
/* Get the widget ids */
Widget RLab = XnslDrawNameToObject(self,"RulerLabel");
Widget RSel = XnslDrawNameToObject(self,
                                   "RulerSelectZone");

if(!RLab || !RSel) {
    printf("Ruler: Can't find label or select zone
                                   object\n");
    return();
}
/* Get the active values */
Float min = self:rulerMin:Float;
Float max = self:rulerMax:Float;
Int ticks = self:rulerTicks:Int;
Int subticks = self:rulerSubTicks:Int;
/* Get the min and max Y positions in pixels */
Int a[Int];
a = XnslDrawFaceGetObjectXPoints(self,RSel);
Float ymin = citof(a[5]);
Float ymax = citof(a[1]);
/* Read the Label's coordinates */
Int b[Int];
b = XnslDrawFaceGetObjectXPoints(self,RLab);
/* Get the min, middle and max X positions */
Float x1 = citof(b[4]);
Float x3 = citof(a[0]);
destroy(a);
destroy(b);
Float x2 = x1 + (x3 - x1)/2.0;
```



```
/* Calculate the Y deltas */
Float dt = (max - min) / citof(ticks);
Float dy = (ymax - ymin) / citof(ticks);
Float sdy;
if (subticks > 0) sdy = dy / citof(subticks+1);
/* Create the scale and/or labels */
Int do_scale = 0;
Int do_label = 0;
if ( type == RULER_SCALE) do_scale = 1;
else if ( type == RULER_LABEL) do_label = 1;
else {
    do_scale = 1;
    do_label = 1;
}
if(do_label) {
    Float y = 0.0;
    Float t = min;
    String buf = malloc(20);
    for (Int i = 0; i <= ticks; i = i+1) {
        /* Duplicate the label, call it "_label" */
        /* generate the label text and set it */
        Widget lab = XnsldrawDuplicateObject(self, RLab);
        if (!lab) {
            printf("Ruler: can't duplicate label object\n");
            free(buf);
            return();
        }
        sprintf(buf, self:rulerFormat, t);
        XnsldrawTextInsertString(self, lab, buf);
        if (y!=0.0) XnsldrawTranslateObject(self, lab, 0.0, y);
        XnsldrawSetObjectName(self, lab, "_label");
        y = y + dy;
        t = t + dt;
    }
    free(buf);
}
if (do_scale) {
    Float y = 0.0;
    Float tab[Int]; tab = new_array();
    Int index = 0;
    for (Int i = 0; i <= ticks; i = i + 1) {
/* Create the tick part of the segment array */
        tab[index] = x1; index = index + 1;
```

```
        tab[index] = ymin + y; index = index + 1;
        tab[index] = x3; index = index + 1;
        tab[index] = ymin + y; index = index + 1;
        /* Create the sub ticks part of the segment array */
        if (subticks >= 1 && i < ticks) {
            Float sy = y + sdy;
            for (Int j = 1; j <= subticks; j = j+1) {
                tab[index] = x2; index = index + 1;
                tab[index] = ymin + sy; index = index + 1;
                tab[index] = x3; index = index + 1;
                tab[index] = ymin + sy; index = index + 1;
                sy = sy + sdy;
            }
        }
        y = y + dy;
    }
    /* Create, rename, show the segment */
    Widget seg = XnslDrawCreateSegment(self, data(tab),
                                       size(tab));

    if (!seg) {
        printf ("Ruler: can't create segment object \n");
        destroy(tab);
        return();
    }
    XnslDrawSetObjectName (self, seg, "_scale");
    destroy(tab);
    XnslDrawShowObject(self, seg);
}
}
function None RulerRecomputeScale(ScaleType type)
{
    if (@2 && @ != @1) {
        Int do_scale = 0;
        Int do_label = 0;
        if (type == RULER_SCALE) do_scale = 1;
        else if (type == RULER_LABEL) do_label = 1;
        else {
            do_scale = 1;
            do_label = 1;
        }
        if (do_label) {
            /* Delete every widget named "_label"*/
            Widget wid;
```

```
        for (wid=XnslDrawNameToObject(self,"_label");wid!=0;
            wid = XnslDrawNameToObject(self,"_label")) {
            XnslDrawHideObject (self, wid);
            XtDestroyWidget(wid);
        }
    }
    if (do_scale) {
/* Delete the scale widget named "_scale" */
        Widget wid;
        wid = XnslDrawNameToObject(self, "_scale");
        if (!wid) {
            printf ("Ruler: Can't find segment object \n");
            return();
        }
        XnslDrawHideObject(self,wid);
        XtDestroyWidget(wid);
    }
/* Create the scale */
    RulerCreateScale(type);
}
}
RulerCreateScale(RULER_BOTH);
XnslDrawRedisplay(self);
```

The script starts with the definition of the new type `ScaleType`:

```
type ScaleType = [RULER_]scale, [RULER_]label, [RULER_]both;
```

This type has a common part shown between brackets and differing ends.

It is used in the **RulerCreateScale** and **RulerRecomputeScale** functions. The rest of the script consists in the definition of the **RulerCreateScale** function, the definition of the **RulerRecomputeScale** function and finally the call to the **RulerCreateScale** function to initialize the widget.

The **RulerCreateScale** function uses the **RulerLabel** and **RulerSelectZone** widgets to choose the ticks' positions, the **rulerMin**, **rulerMax**, **rulerTicks** and **rulerSubTicks** resources to do the calculations and set the labels' texts. The beginning of the function prepares all these parameters.

Then, if needed, the label part will be created by duplicating the label that has been kept in the **ruler.draw** file, giving it a text and a name and then placing it in the correct position. The purpose of giving the same name “_label” to all the labels is to be able to delete them without having to manage a table of names in the application.

Afterwards, if needed, the scale part will be created as a segment. A segment is created with an array of (x1,y1,x2,y2) coordinate quadruples giving the beginning and ending positions of each line segment. Creating the scale consists in creating this coordinate array and then creating the segment widget with the “_scale” name. Since the widgets created this way are not automatically managed, a call to **XnsIDDrawShowObject** is included after the creation to show the segment. Creating the scale as a segment is recommended since it reduces the number of Draw objects in the drawing, thereby speeding up the calculation and redrawing processes.

The **RulerRecomputeScale** function destroys the widgets corresponding to the type parameter (labels, scale or both) and then calls the **RulerCreateScale** function to recreate the same widgets with the current resource values. It starts with the line :

```
if (@2 && @ != @1)
```

This introduces the notions of current value and previous value. The **@** symbol refers to the current value (new value) of the active value where the call to the **RulerRecomputeScale** function takes place. The **@1** symbol refers to the previous value of the active value. The **@2** symbol refers to the widget ID. Its value is 0 when the script is called from the Initialize active value, since the widget does not yet exist.

This test line at the beginning of the **RulerRecomputeScale** function can be read as: if this function is not called during initialization and if the current value (or new value) of the calling active value is different from the previous one, then the rest of the function can be executed.

Once this script has been typed and the new resources applied to the draw widget, you can save the **Ruler** class.

5.1.3 Modify the interface

You will modify the interface to add three new labels and text widgets, to allow the user to specify the number of ticks, the number of sub ticks and the label format for the **Ruler** widget. The explanations text also needs to be completed to incorporate the new facilities and since the text will be longer and the space allowed for this widget smaller, you will place it in a scrolled window. Also, the 3 original text widgets need to be initialize so as to agree with the settings of the rulerMin, rulerMax, and rulerValue resources.

After these modifications, the interface will look like this:

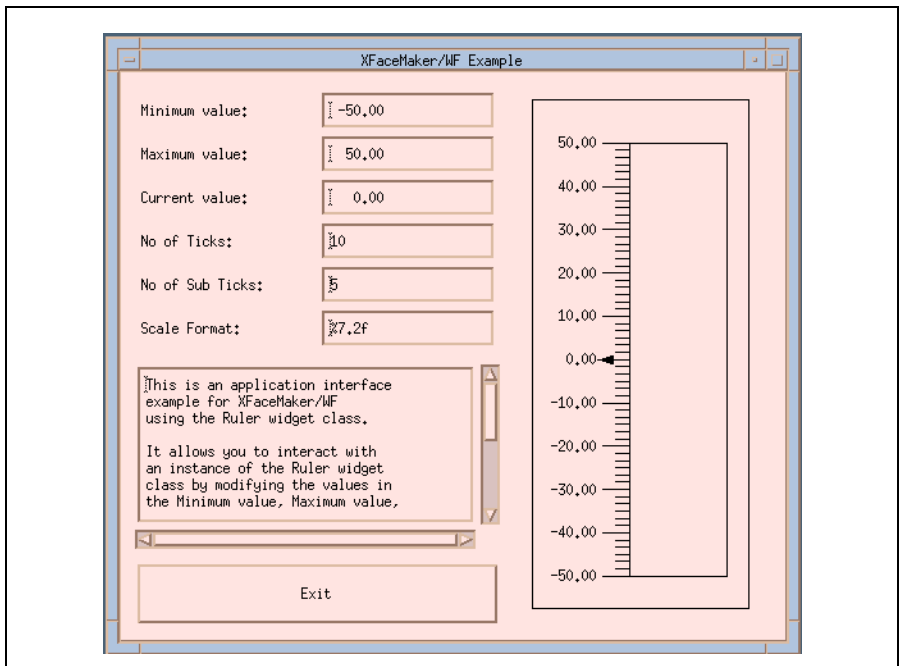


Figure 5.2: The modified interface

To modify the interface, open the **interface.fm** file.

First, let's initialize the three original text fields. We can remove the default value we had specified previously, since we will initialize them with the values found in the ruler0 widget resources when the interface is created. We will use the Form's

createCallback to do this: when the callback is executed, all the text widgets and the ruler widget exist. Select the Form and enter the following script in its xfmCreateCallback:

```
Float v; String str = malloc(10);
v = self.ruler0:rulerMin;
sprintf(str,"%7.2f",v);
self.RC.xmText0:value = str;
v = self.ruler0:rulerMax;
sprintf(str,"%7.2f",v);
self.RC.xmText1:value = str;
v = self.ruler0:rulerValue;
sprintf(str,"%7.2f",v);
self.RC.xmText2:value = str;
free(str);
```

Verify, in Build mode, that the values displayed in the Minimum value, Maximum value and Current value text fields reflect the values you specify for the **rulerMin**, **rulerMax** and **rulerValue** resources of the ruler0 widget. Save your interface and proceed with adding the new label/text pairs.

First, select the RowColumn widget and change its **numColumns** resource from 3 to 6, then add the label/text pairs (you can *Copy* a pair and *Paste* it three times, then edit the resources of each of the widgets).

Name the labels xmLabel3 to xmLabel5 and change the labelStrings to “No of Ticks:”, “No of Sub Ticks:” and “Label Format:”.

Name the text widgets xmText3 to xmText5 and set their activate callbacks as below. We don’t specify default values , we will set their values in the Form’s createCallback, according to the settings of the resources in the ruler widget, as we did for the other texts.

The text widget showing the number of ticks (xmText3) can have the following activate callback script:

```
script ticks_changed:
Int v = Int(ConvertString(self,self:value,"Int"));
parent.parent.ruler0:rulerTicks = v;
```

The text widget showing the number of sub ticks (xmText4) can have the following activate callback script:

```
script subticks_changed:
Int v = Int(ConvertString(self,self:value,"Int"));
parent.parent.ruler0:rulerSubTicks = v;
```

The text widget showing the format of the scale labels (xmText5) can have the following activate callback script:

```
parent.parent.ruler0:rulerFormat = self:value;
```

Now, add the following to the Form's xfmCreateCallback, in order to initialize the texts to reflect the resource settings of the ruler.

```
self.RC.xmText3:value = itoa(self.ruler0:rulerTicks);
self.RC.xmText4:value = itoa(self.ruler0:rulerSubTicks);
self.RC.xmText5:value = self.ruler0:rulerFormat;
```

Rebuild the interface to check that all the texts are correctly initialized in Build mode (select the ApplicationShell and do Rebuild either from the popup menu in the widget tree or list, or from the resource window).

The last thing we have to do is place the explanation text in a scrolled window; first, remove any attachments for that widget (xmText3, direct child of the Form - its parent will no longer be an xmForm). Add a scrolledWindow widget in the form and, using the Cut and Paste mechanism, remove the text widget from the form and place it in the scrolledWindow; do the attachments for the scrolledWindow.

Then modify the explanations text to mention the new resources:

```
This is an application interface
example for XFaceMaker/WF
using the Ruler widget class.
```

```
It allows you to interact with
an instance of the Ruler widget
class by modifying the values in
the Minimum value, Maximum value,
Current value, No of Ticks, No of
Sub Ticks and Scale Format Text
widgets.
```

```
You can also use the mouse to
click in the selection area of
the Ruler and this action will
be reflected in the Current
value Text.
```

The widget tree of this modified interface is shown in the figure below.



Figure 5.3: Widget tree of the modified interface

The interface is now completely modified. You can save it and test it, either in **XFaceMaker** or, after compiling and linking the compiled and interpreted versions as before, directly as a compiled or interpreted interface.

Before we push the tests further, let’s see how the rulerData structure can be handled.

5.1.4 Declare the rulerData Structure

We should be able to use the rulerData structure in Face scripts and in C programs that use the class without having to declare it. Also, when XFaceMaker generates C code for our .fm files, it has to know what to generate if we use the structure. This section explains how to handle all the aspects of the structure declaration.

1. Declare the structure in the class Initialize method. Once the structure is declared in the Initialize method, you can use it in any interface script that uses the class without having to re-declare it: if the Ruler.fm file is loaded in the XFaceMaker executable you are using, the Face interpreter knows the structure. Add the following at the beginning of the Initialize method, just after the enumeration declaration:

```
struct rulerData{  
    Float curpos;  
    Float curval;  
};
```

You can now remove the structure declaration from the interface.fm file (in the valueChangedCallback of the ruler0 widget).

2. Declare the rulerData type in the Ruler.face file. This tells XFaceMaker what C code to generate for application files that use the structure. For example, if in the valueChangedCallback of ruler0 we call an application function to which we pass the call_data, XFaceMaker needs to know how to prototype the function. In the Ruler .face file, add the following line at the end:

```
type rulerData "rulerData *";
```

Normally, we would also declare the structure itself, here, with :

```
struct rulerData "rulerData*" {  
    Float curpos;  
    Float curval;  
};
```

However, remember that the structure is also declared in the ruler.draw file. We need to keep it there in case we decide to re-load our ruler.draw file in XDraw-Maker. Since some compilers scream when a structure is multiply declared, it is better to leave this declaration out as long as we continue to use the ruler.draw file.

3. Declare the rulerData structure in the Ruler.h file. In order to use the structure in our C code, we have to add the structure declaration in the Ruler.h file. Add the following at the end of the ruler.h file:

```
typedef struct {
```

```
float curpos;  
float curval;  
}rulerData ;
```

4. C files that contain application functions using the rulerData structure will just need to include the Ruler.h file with:

```
#include <Ewf/Ruler.h>
```

Having done the changes above, re-do an extended XFaceMaker (in a separate directory to keep things clear) with the new Ruler.face extension file. You might build a small interface that instantiates the Ruler class. Check that you are able to edit all the Ruler widget's resources. Try calling an application function from the ruler widget's valueChangedCallback. You might have the following script in the callback:

```
rulerData cbs;  
cbs = $;  
function None foo(rulerData cb);\  
foo($);
```

Generate the application files and check that function foo is correctly prototyped in the App.c file. Enter the body of the function, e.g. to print the curval and curpos values and add the line to include the Ruler.h file:

```
#include <Ewf/Ruler.h>  
  
#if NeedFunctionPrototypes  
void foo(rulerData * cb)  
#else  
void foo(cb)  
rulerData * cb;  
#endif  
{  
    printf("Structure items %f %f \n",cb->curpos, cb->curval);  
}
```

Build the application, test it.

5.1.5 Source listings

In the **XFaceMaker** files, you will see that the lines in the callback scripts are followed by the \ character to indicate that the callback continues on the next line. In the **XDrawMaker** files, you will see that the “ inside a script must be preceded with a \ because the whole script is surrounded with “ characters.

The listings in this section assume that the rulerData structure has been defined in the Initialize method, as explained in the previous section.

5.1.5.1 Draw file : ruler.draw

```
{
    drawUnits = no_units;
    drawAngleUnits = no_angle_units;
    drawGridOneOf = 1;
    drawGridSnapTo = True;
}
XnsldDrawRectangle RulerBackground {
    drawShowBorder = True;
    drawPoints = "60, 40, 200, 470";
    drawNumPoints = 2;
}
XnsldDrawRectangle RulerSelectZone {
    drawShowBorder = True;
    drawEventMask = Button1Down|Button1Motion;
    drawActivateCallbackList = "on Button1Down
script MoveCursor:
/* Get the positions of the Min and Max divisions */
    Int a[Int];
    a = XnsldDrawFaceGetObjectXPoints(parent, self);
    Float ymin = citof(a[5]);
    Float ymax = citof(a[1]);
    destroy(a);
/* Get the Y position of the mouse */
    Float y = citof( XnsldDrawEventY (parent, $));
/* Test if the click was within limits */
    if (y < ymax) y = ymax;
    if (y > ymin) y = ymin;
/* compute a translation to move the cursor */
    Float ty = y - ymin;
    Widget RPos = XnsldDrawNameToObject(parent,
                                         \"RulerPosition\");
    if (RPos) XnsldDrawSetTranslation (parent, RPos, 0.0, ty);
    else printf(\"Ruler error: RulerPosition widget ID
                                         not found\n\");
/* Define the rulerData structure */
    struct rulerData
    {
        Float curpos;
        Float curval;
    };
/* Create and initialize the structure */
    rulerData rd = new_struct(rulerData);
```

```

    rd -> curpos = y;
    rd -> curval = parent:rulerValue;
/* Call valueChangedCallback with call_data=the structure */
    XtCallCallbacks(parent, \"valueChangedCallback\", rd);
    Free (String(rd));

/* Show the changes */
    XnslDrawRedisplay (parent);
end
on Button1Motion
    script MoveCursor;
end
";
    drawPoints = "150, 80, 90, 400";
    drawNumPoints = 2;
    drawSelected = True;
}
XnslDrawLabel RulerLabel {
    drawShowBorder = False;
    drawSelectCondition = "draw_inside draw_on_border";
    drawPoints = "60, 470, 60, 20";
    drawNumPoints = 2;
    drawLabelString = "";
    drawLabelAlignment = draw_label_align_east;
    drawLabelTransformString = False;
}
XnslDrawLine RulerPosition {
    drawShowBorder = True;
    drawPoints = "120, 480, 150, 480";
    drawNumPoints = 2;
    drawYTranslation = -1.000000;
    drawArrowDir = draw_from_arrow;
}

{
    drawUnits = pixels;
    drawAngleUnits = no_angle_units;
    drawDoubleBuffer = True;
}
XnslDrawRectangle RulerBackground {
    drawShowBorder = True;
    drawPoints = "50, 25, 200, 425";

```

```
    drawNumPoints = 2;
}
XnslDrawLabel RulerLabel {
    drawShowBorder = False;
    drawSelectCondition = "draw_inside draw_on_border";
    drawPoints = "50, 415, 75, 20";
    drawNumPoints = 2;
    drawFont = *helvetica-bold-r-*-14-*;
    drawLabelString = "" ;
    drawLabelAlignment = draw_label_align_right;
    drawLabelTransformString = False;
}
XnslDrawRectangle RulerSelectZone {
    drawShowBorder = True;
    drawEventMask = Button1Down|Button1Motion;
    drawActivateCallbackList = "on Button1Down
script MoveCursor :
/* Get the positions of the Min and Max Ticks */
    Int a[Int];
    a = XnslDrawFaceGetObjectXPoints(parent,self);
    Float ymin = citof(a[5]);
    Float ymax = citof(a[1]);
    destroy(a);
/* Get the Y position of the mouse */
    Float y = citof(XnslDrawEventY(parent, $));
/* Test if the click was in the limits */
    if (y < ymax) y = ymax;
    if (y > ymin) y = ymin;
/* Compute a translation to move the RulerPosition */
    Float ty = y - ymin;
    Widget RPos = XnslDrawNameToObject(parent,
        \"RulerPosition\");
    if (RPos) XnslDrawSetTranslation(parent, RPos, 0.0, ty);
    else printf(\"Ruler Error: RulerPosition widget id not
        found\n\");
/* Define the rulerData structure */
    struct rulerData
    {
        Float curpos;
        Float curval;
    };
/* Initialize the structure and set values to its fields */
    rulerData rd = new_struct(rulerData);
```

```
rd->curpos = y;
rd->curval = parent:rulerValue;
/* Call the valueChangedCallback with the structure as
                                call_data */
XtCallCallbacks(parent, \"valueChangedCallback\", rd);
free(String(rd));

/* Show the changes */
XnslDrawRedisplay(parent);
end
on Button1Motion
  script MoveCursor;
end
";
  drawPoints = "150, 50, 75, 375";
  drawNumPoints = 2;
}
XnslDrawLine RulerPosition {
  drawShowBorder = True;
  drawPoints = "150, 425, 225, 425";
  drawNumPoints = 2;
  drawForeground = black;
  drawFillMode = draw_filled;
  drawArrowDir = draw_from_arrow;
}
```

5.1.5.2 Ruler Xt class file : Ruler.fm

```
widget XnslDraw {
    flag {
        autosize
        autopos
    }
    activevalue {
        name = Initialize
        set = type ScaleType = [RULER_]scale, [RULER_]label,
                                [RULER_]both;\
}
struct rulerData{
    Float curpos;
    Float curval;\
};\
function None RulerCreateScale(ScaleType type)\
{
/* Get the widget ids */
    Widget RLab = XnslDrawNameToObject(self,"RulerLabel");
    Widget RSel=XnslDrawNameToObject(self,"RulerSelectZone");

if(!RLab || !RSel) {
    printf("Ruler: Can't find label or select zone
                                object \n");
    return();
}
/* Get the active values */
    Float min = self:rulerMin:Float;
    Float max = self:rulerMax:Float;
    Int ticks = self:rulerTicks:Int;
    Int subticks = self:rulerSubTicks:Int;
/* Get the min and max Y positions in pixels */
    Int a[Int];
    a = XnslDrawFaceGetObjectXPoints(self,RSel);
    Float ymin = citof(a[5]);
    Float ymax = citof(a[1]);
/* Read the Label's coordinates */
    Int b[Int];
    b = XnslDrawFaceGetObjectXPoints(self,RLab);
/* Get the min, middle and max X positions */
    Float x1 = citof(b[4]);
    Float x3 = citof(a[0]);
    destroy(a);
    destroy(b);
```

```
Float x2 = x1 + (x3 - x1)/2.0;
/* Calculate the Y deltas */
Float dt = (max - min) / citof(ticks);
Float dy = (ymax - ymin) / citof(ticks);
Float sdy;
if (subticks > 0) sdy = dy / citof(subticks+1);

/* Create the scale and/or labels */
Int do_scale = 0;
Int do_label = 0;
if ( type == RULER_SCALE) do_scale = 1;
else if ( type == RULER_LABEL) do_label = 1;
else {
    do_scale = 1;
    do_label = 1;
}
if(do_label) {
    Float y = 0.0;
    Float t = min;
    String buf = malloc(20);
    for (Int i = 0; i <= ticks; i = i+1) {
/* Duplicate the label, call it "_label" */
/* generate the label text and set it */
        Widget lab = XnslDrawDuplicateObject(self, RLab);
        if (!lab) {
            printf("Ruler: can't duplicate label object\n");
            free(buf);
            return();
        }
        sprintf(buf, self:rulerFormat, t);
        XnslDrawTextInsertString(self, lab, buf);
        if (y!=0.0) XnslDrawTranslateObject(self, lab, 0.0, y);
        XnslDrawSetName(self, lab, "_label");
        y = y + dy;
        t = t + dt;
    }
    free(buf);
}
if (do_scale) {
    Float y = 0.0;
    Float tab[Int]; tab = new_array();
    Int index = 0;
    for (Int i = 0; i <= ticks; i = i + 1) {
```



```
/* Create the tick part of the segment array */
    tab[index] = x1; index = index + 1;
    tab[index] = ymin + y; index = index + 1;
    tab[index] = x3; index = index + 1;
    tab[index] = ymin + y; index = index + 1;
/* Create the sub ticks part of the segment array */
    if (subticks >= 1 && i < ticks) {
        Float sy = y + sdy;
        for (Int j = 1; j <= subticks; j = j+1) {
            tab[index] = x2; index = index + 1;
            tab[index] = ymin + sy; index = index + 1;
            tab[index] = x3; index = index + 1;
            tab[index] = ymin + sy; index = index + 1;
            sy = sy + sdy;
        }
    }
    y = y + dy;
}
/* Create, rename, show the segment */
Widget seg = XnslDrawCreateSegment(self, data(tab),
                                   size(tab));

if (!seg) {
    printf ("Ruler: can't create segment object \n");
    destroy(tab);
    return();
}
XnslDrawSetObjectName (self, seg, "_scale");
destroy(tab);
XnslDrawShowObject(self, seg);
}\
}
\
function None RulerRecomputeScale(ScaleType type)\
{
    if (@2 && @ != @1) {
        Int do_scale = 0;
        Int do_label = 0;
        if (type == RULER_SCALE) do_scale = 1;
        else if (type == RULER_LABEL) do_label = 1;
        else {
            do_scale = 1;
            do_label = 1;
        }
    }
}
```

```
    if (do_label) {
        /* Delete every widget named "_label" */
        Widget wid;
        for (wid = XnslDrawNameToObject(self, "_label");
            wid != 0; wid = XnslDrawNameToObject(self, "_label")) {
            XnslDrawHideObject (self, wid);
            XtDestroyWidget(wid);
        }
    }
    if (do_scale) {
        /* Delete the scale widget named "_scale" */
        Widget wid;
        wid = XnslDrawNameToObject(self, "_scale");
        if (!wid) {
            printf ("Ruler: Can't find segment object \n");
            return();
        }
        XnslDrawHideObject(self, wid);
        XtDestroyWidget(wid);
    }
    /* Create the scale */
    RulerCreateScale(type);
} \
} \
RulerCreateScale(RULER_BOTH); \
XnslDrawRedisplay(self);
    storage = object
    automatic = true
    usage = class
}
activevalue {
    name = rulerPositionColor
    set = Widget pos_wid =
        XnslDrawNameToObject(self, "RulerPosition"); \
if (pos_wid) XnslDrawSetForegroundPixel(self, pos_wid, @); \
XnslDrawRedisplay(self);
    type = Pixel
    storage = object
    automatic = true
    usage = class
    default_type = String
    default_value = black
}
```

```
    activevalue {
        name = rulerMin
        set = RulerRecomputeScale(RULER_LABEL);
        type = Float
        storage = object
        automatic = true
        usage = class
        default_type = String
        default_value = -50.00
    }
    activevalue {
        name = rulerMax
        set = RulerRecomputeScale(RULER_LABEL);
        type = Float
        storage = object
        automatic = true
        usage = class
        default_type = String
        default_value = 50.00
    }
    activevalue {
        name = rulerValue
        get = /*Retrieve the min and max Y positions */\
Widget RSel = XnsldrawNameToObject(self,"RulerSelectZone");\
if (!RSel) printf("Ruler Error: RulerSelectZone widget ID
                                not found\n");\
    else {\
        Int a[Int];\
        a = XnsldrawFaceGetObjectXPoints(self,RSel);\
        Float ymin = citof(a[5]);\
        Float ymax = citof(a[1]);\
        destroy(a);\
/* Retrieve the cursor's current Y value */\
        Widget RPos = XnsldrawNameToObject(self,"RulerPosition");\
        if (!RPos) printf("Ruler Error: RulerPosition widget ID
                                not found\n");\
    else {\
        Float y = XnsldrawGetTranslationY(self,RPos) + ymin;\
    \
/* Compute the value */\
        Float min = Float(self:rulerMin);\
        Float max = Float(self:rulerMax);\
        Float v = min + (y - ymin) * (max - min) / (ymax - ymin);
```

```
\
/* Copy it to the resource value */
@ = v;\
}\
}

    set = /*Retrieve the min and max Y positions */\
Widget RSel = XnslDrawNameToObject(self,"RulerSelectZone");\
if (!RSel) printf("Ruler Error: RulerSelectZone widget ID
                    not found\n");\
else {\
    Int a[Int];\
    a = XnslDrawFaceGetObjectXPoints(self,RSel);\
    Float ymin = citof(a[5]);\
    Float ymax = citof(a[1]);\
    destroy(a);\
/* Retrieve the cursor's current Y value */\
Widget RPos = XnslDrawNameToObject(self,"RulerPosition");\
if (!RPos) printf("Ruler Error: RulerPosition widget ID
                    not found\n");\

    else {\
        Float y = XnslDrawGetCenterY(self,RPos);\
/* Get the resource value contained in the @ variable */\
        Float v = Float(@);\
/* Check it is valid */\
        Float min = Float(self:rulerMin);\
        Float max = Float(self:rulerMax);\
        if (v > max) v = max;\
        if (v < min) v = min;\
    }
/* compute the translation for the new position */\
    Float ty = (v - min) * (ymax - ymin) / (max - min);\
    XnslDrawSetTranslation(self,RPos, 0.0, ty);\
/* Redisplay the widget */\
    XnslDrawRedisplay(self);\
}\
}

    type = Float
    storage = object
    automatic = true
    usage = class
    default_type = String
    default_value = 0.00
}
```

```
    activevalue {
        name = valueChangedCallback
        type = Callback
        storage = object
        automatic = true
        usage = class
    }
    activevalue {
        name = rulerFont
        set = Int nchildren = self:numChildren;\
Widget child[Int];\
child = new_c_array(Pointer(self:children),nchildren);\
for (i = 0; i < nchildren; i = i+1) {
    w = child[i];
    if (XnslIsDrawLabel(w)) w:drawFont:DrawFont = @;\
}\
XnslDrawRedisplay(self);
    type = DrawFont
    storage = object
    automatic = true
    usage = class
    default_type = String
    default_value = fixed
}
    activevalue {
        name = rulerTicks
        set = if (!@ || @ < 2) @ = 2;\
RulerRecomputeScale(RULER_BOTH);
        type = Int
        storage = object
        automatic = true
        usage = class
        default_type = String
        default_value = 10
    }
    activevalue {
        name = rulerSubTicks
        set = if (!@ || @ < 1) @ = 1;\
RulerRecomputeScale(RULER_SCALE);
        type = Int
        storage = object
        automatic = true
        usage = class
```

```
        default_type = String
        default_value = 5
    }
    activevalue {
        name = rulerFormat
        set = if (!@) @ = "%7.2f";\
RulerRecomputeScale(RULER_LABEL);
        type = String
        storage = object
        automatic = true
        usage = class
        default_type = String
        default_value = %7.2f
    }
    resources {
        autoResize = true
        drawFile = ruler.draw
    }
} xnsldraw0
```

5.1.5.3 XFM Extension - Ruler.face file

```

include "XnslDraw.face";

/*****
 * Class information.
 *****/

define _XFM_EXT_CODE_Ruler '
(*fns->add_widget_class)(
&ewfRulerWidgetClass,
0,
FM_PRIMITIVE,
FM_NORMAL,
0,
"#include <Ewf/Ruler.h>",
"ewfRulerWidgetClass",
0,
0,
0,
"Ruler.bm");';

/*****
 * Build information.
 *****/

define _XFM_EXT_C_LIBRARY_Ruler'';
define _XFM_EXT_E_LIBRARY_Ruler'RulerFm';

define _XFM_EXT_BUILD_C_INIT_FUNCALL_Ruler
    'XtInitializeWidgetClass(ewfRulerWidgetClass);';
define _XFM_EXT_BUILD_INIT_FUNCTION_Ruler'ERulerInit';
define _XFM_EXT_BUILD_INIT_FUNCALL_Ruler 'ERulerInit();';
define _XFM_EXT_BUILD_INCLUDES_Ruler '
    #include <Ewf/Ruler.h>';
define _XFM_EXT_BUILD_LIBRARIES_Ruler '-lRuler';
define _XFM_EXT_BUILD_C_LIBRARIES_Ruler'';
define _XFM_EXT_BUILD_I_LIBRARIES_Ruler '';
define _XFM_EXT_BUILD_E_LIBRARIES_Ruler '-lRulerFm';
define _XFM_EXT_BUILD_INCLUDE_PATH_Ruler
    '-I. -I$(NSLHOME)/include';
define _XFM_EXT_BUILD_LIBRARY_PATH'-L./Ewf
    -L$(NSLHOME)/lib';
type rulerData "rulerData *";

```

5.1.5.4 Interface example --- Fm file : interface.fm

```
widget ApplicationShell {
    flag {
        mapped
    }
    resources {
        x = 714
        y = 431
        width = 521
        height = 445
        title = XFaceMaker/WF Example
    }
    children {
        widget XmForm {
            flag {
                mapped
                autosize
                autopos
            }
            createproc {
                Float v;\
String str = malloc(10);\
v = self.ruler0:rulerMin;\
sprintf(str,"%7.2f",v);\
self.RC.xmText0:value = str;\
v = self.ruler0:rulerMax;\
sprintf(str,"%7.2f",v);\
self.RC.xmText1:value = str;\
v = self.ruler0:rulerValue;\
sprintf(str,"%7.2f",v);\
self.RC.xmText2:value = str;\
free(str);\
self.RC.xmText3:value = itoa(self.ruler0:rulerTicks);\
self.RC.xmText4:value = itoa(self.ruler0:rulerSubTicks);\
self.RC.xmText5:value = self.ruler0:rulerFormat;\
            }
            resources {
            }
            children {
                widget XmRowColumn {
                    flag {
                        mapped
                        autosize
                    }
                }
            }
        }
    }
}
```



```
        autopos
    }
    resources {
        leftAttachment = attach_form
        leftOffset = 12
        topAttachment = attach_form
        topOffset = 12
        orientation = horizontal
        packing = pack_column
        numColumns = 6
    }
    children {
        widget XmLabel {
            flag {
                mapped
                autosize
                autopos
            }
            resources {
                labelString = Minimum value:
            }
        } xmLabel0
        widget XmText {
            flag {
                mapped
                autosize
                autopos
            }
            resources {
                columns = 15
                activateCallback = \

script min_changed:\
Float min = Float(ConvertString(self, self:value,
                                "Float"));\\
/* Get relative path to ruler widget - depends on tree */\\
Widget w_ruler = parent.parent.ruler0;\
w_ruler:rulerMin = min;\
Float max = w_ruler:rulerMax;\
if (max < min) {
    w_ruler:rulerMax = min + 10.0;
    String str; str = malloc(10);
    sprintf(str, "%7.2f",w_ruler:rulerMax);
    parent.xmText1:value = str;
```

```

    free(str);\
}\
Float v ;\
v = Float(ConvertString(self,parent.xmlText2:value,
"Float"));\
w_ruler:rulerValue = v;
    }
} xmText0
widget XmLabel {
    flag {
        mapped
        autosize
        autopos
    }
    resources {
        labelString = Maximum value:
    }
} xmLabel1
widget XmText {
    flag {
        mapped
        autosize
        autopos
    }
    resources {
        columns = 15
        activateCallback = \
script max_changed:\
Float max = Float(ConvertString(self, self:value,
                                "Float"));\
/* Get relative path to ruler widget - depends on tree */\
Widget w_ruler = parent.parent.ruler0;\
w_ruler:rulerMax = max;\
Float min = w_ruler:rulerMin;\
if (min >= max) {
    w_ruler:rulerMin = max - 10.0;
    String str; str = malloc(10);
    sprintf(str, "%7.2f",w_ruler:rulerMin);
    parent.xmlText0:value = str;
    free(str);\
}\
Float v = Float(ConvertString(self,parent.xmlText2:value,
                                "Float"));\

```

```
w_ruler:rulerValue = v;
    }
} xmText1
widget XmLabel {
    flag {
        mapped
        autosize
        autopos
    }
    resources {
        labelString = Current value:
    }
} xmLabel2
widget XmText {
    flag {
        mapped
        autosize
        autopos
    }
    resources {
        columns = 15
        activateCallback = \
script cur_changed:\
Float v = Float(ConvertString(self, self:value, "Float")); \
/* Get relative path to ruler widget - depends on tree */ \
Widget w_ruler = parent.parent.ruler0; \
if (v > w_ruler:rulerMax) {
v = w_ruler:rulerMax;
String str; str = malloc(10);
sprintf(str, "%7.2f", v);
self:value = str;
free(str); \
} \
if (v < w_ruler:rulerMin) {
v = w_ruler:rulerMin;
String str; str = malloc(10);
sprintf(str, "%7.2f", v);
self:value = str;
free(str); \
} \
w_ruler:rulerValue = v;
    }
} xmText2
```

```
        widget XmLabel {
            flag {
                mapped
                autosize
                autopos
            }
            resources {
                labelString = No of Ticks:
            }
        } xmLabel3
        widget XmText {
            flag {
                mapped
                autosize
                autopos
            }
            resources {
                activateCallback = \
script ticks_changed:\
Int v = Int(ConvertString(self,self:value,"Int"));\
parent.parent.ruler0:rulerTicks = v;
            }
        } xmText3
        widget XmLabel {
            flag {
                mapped
                autosize
                autopos
            }
            resources {
                labelString = No of Sub Ticks:
            }
        } xmLabel4
        widget XmText {
            flag {
                mapped
                autosize
                autopos
            }
            resources {
                columns = 15
                activateCallback = \
script subticks_changed:\
```

```
Int v = Int(ConvertString(self,self:value,"Int"));\  
parent.parent.ruler0:rulerSubTicks = v;  
    }  
    } xmText4  
    widget XmLabel {  
        flag {  
            mapped  
            autosize  
            autopos  
        }  
        resources {  
            labelString = Scale Format:  
        }  
    } xmLabel5  
    widget XmText {  
        flag {  
            mapped  
            autosize  
            autopos  
        }  
        resources {  
            activateCallback = \  
parent.parent.ruler0:rulerFormat = self:value;  
        }  
    } xmText5  
    }  
} RC  
widget XmPushButton {  
    flag {  
        mapped  
        autopos  
    }  
    resources {  
        bottomAttachment = attach_form  
        bottomOffset = 12  
        leftAttachment = attach_opposite_widget  
        leftWidget = RC  
        rightAttachment = attach_opposite_widget  
        rightWidget = RC  
        height = 50  
        width = 285  
        labelString = Exit  
        activateCallback = \  

```

```

/* declare the Ruler_exit function */\
function None Ruler_exit();\
/* call it */\
Ruler_exit();

    }
} xmPushButton0
widget Ruler {
    flag {
        mapped
        autopos
    }
    resources {
        rightAttachment = attach_form
        rightOffset = 17
        topAttachment = attach_form
        topOffset = 12
        bottomAttachment = attach_form
        bottomOffset = 16
        height = 417
        width = 195
        leftAttachment = attach_widget
        leftOffset = 12
        leftWidget = RC
        valueChangedCallback = \
/* Initialize the rulerData structure */\
rulerData rd = $;\
/* Put the current value in the text widget */\
String str; str = malloc(10);\
sprintf(str,"%7.2f",rd->curval);\
parent.RC.xmText2:value = str;\
free(str);

    }
} ruler0
widget XmScrolledWindow {
    flag {
        mapped
        autosize
        autopos
    }
    resources {
        leftAttachment = attach_opposite_widget
        leftWidget = RC
        rightAttachment = attach_opposite_widget

```

```
        rightWidget = RC
        topAttachment = attach_widget
        topWidget = RC
        bottomAttachment = attach_widget
        bottomOffset = 10
        bottomWidget = xmPushButton0
        topOffset = 10
    }
    children {
        widget XmText {
            flag {
                mapped
                autosize
                autopos
            }
            resources {
                editMode = multi_line_edit
                value = \
```

This is an application interface\
example for XFaceMaker/WF \
using the Ruler widget class.\

\

It allows you to interact with \
an instance of the Ruler widget\
class by modifying the values in\
the Minimum value, Maximum value, \
Current value, No of Ticks, No of\
Sub Ticks and Scale Format Text \
widgets.\

\

You can also use the mouse to \
click in the selection area of\
the Ruler and this action will \
be reflected in the Current\
value Text.

```
                                editable = false
                                traversalOn = false
                                }
                            } xmText3
                        }
                    } SW
                }
            } xmForm0
```

```
    }  
} applicationShell0
```


5.2 The Ruler widget class : FM only version

The **Ruler** widget class example has, until now, used the **drawFile** resource to associate the drawing's data file (**ruler.draw**) generated by **XDrawMaker** to the Draw widget of the new class being created.

It is also possible to generate .fm files within **XDrawMaker**, which can then be read into **XFaceMaker** directly. In this case, the drawing's objects are available in **XFaceMaker** as gadgets and their resources can be modified directly in the resource window.

To create the FM only version of the **Ruler** widget class, you will reuse most of the work done in the first version of that class: the only file which is changed is the **Ruler.fm** file. The other files, **ruler.draw**, **Ruler.bm**, **interface.fm**, **interfaceMain.c**, **interfaceApp.c**, **interfaceApp.h** and **interfaceMake** can be used directly.

If you have already gone through the creation of the **Ruler** widget class in “A Guided Tour” on page 37, you may find that the explanations in the following pages remind you of those you have already followed to create the **Ruler**'s resources.

This is made necessary because the **Ruler.fm** file is recreated in **XDrawMaker** and will not contain the active values created in the first two examples. The creation of the active values in this example is slightly different in that you can reference the draw gadgets directly instead of getting their widget id through the **XnsIDrawnNameToWidget** function.

5.2.1 Create the Ruler.fm file in XDrawMaker

The first step is to create the **Ruler.fm** file in **XDrawMaker**, starting with the **ruler.draw** file created in “Run Time Initialization” on page 108.

To do this, launch **XDrawMaker** by typing :

```
dm &
```

If this command is not known, you should check with your product administrator that everything is correctly installed. You can also check the explanations given at the beginning of “Draw the Widget in XDrawMaker” on page 38 concerning the environment variables.

You should now have a running version of **XDrawMaker** on the screen.

Load the **ruler.draw** file with the **File/Open** menu option.

To export the drawing as an XFaceMaker file, use the ***File/Export/XFaceMaker file...*** menu option. Call the file **Ruler.fm** in the Selection field of the File Selection box and **OK** your choices.

You can now exit **XDrawMaker** with the ***File/Exit*** command.

5.2.2 Modify the Ruler.fm file in XFaceMaker

The drawing created with **XDrawMaker** is not yet a widget since no resources or callbacks are defined. However, you can now load the drawing in **XFaceMaker** and define its resources. To do so, you must launch a version of **XFaceMaker** that contains the Draw widget. This can be done with the following command:

```
xfmwf &
```

Load the **Ruler.fm** file with the ***File/Open*** menu option.

Notice that the draw widget is by default called **draw_edit** and that the elements of the drawing are available in the Widget Tree as gadgets.

The widget tree for the file **Ruler.fm** is as follows:

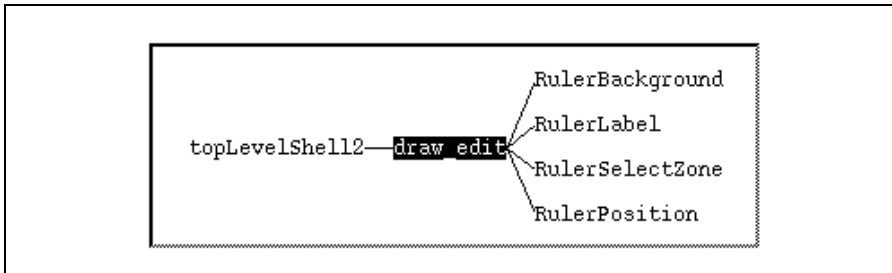


Figure 5.4: Widget tree of Ruler.fm

5.2.2.1 Modify the RulerSelectZone gadget

Select the **RulerSelectZone** gadget in the Widget Tree and open a ***New Resource Window*** by choosing that option in the ***Windows*** menu. In the Current Widget Resources window, edit the **drawActivateCallbackList** resource.

Since you now have direct access to the **RulerPosition** gadget, you no longer need to use the **XnslDrawNameToObject** function to retrieve its widget id.

The **drawActivateCallbackList** should now be :

```
on Button1Down
    script MoveCursor :
/* Get the positions of the Min and Max Ticks */
    Int a[Int];
    a = XnslDrawFaceGetObjectXPoints(parent,self);
    Float ymin = citof(a[5]);
    Float ymax = citof(a[1]);
    destroy(a);
/* Get the Y position of the mouse */
    Float y = citof(XnslDrawEventY(parent, $));
/* Test if the click was in the limits */
    if (y < ymax) y = ymax;
    if (y > ymin) y = ymin;
/* Compute a translation to move the RulerPosition */
    Float ty = y - ymin;
    XnslDrawSetTranslation(parent, parent.RulerPosition,
                           0.0, ty);
/* Define the rulerData structure */
    struct rulerData
    {
        Float curpos;
        Float curval;
    };
/* Initialize the structure and set values to its fields */
    rulerData rd = new_struct(rulerData);
    rd->curpos = y;
    rd->curval = parent:rulerValue;
/* Call the valueChangedCallback with the structure as
    call_data */
    XtCallCallbacks(parent, "valueChangedCallback", rd);
    free(String(rd));
/* Show the changes */
    XnslDrawRedisplay(parent);
end
on Button1Motion
    script MoveCursor;
end
```

5.2.2.2 Modify the RulerLabel gadget

Select the **RulerLabel** gadget in the Widget Tree. In the Current Widget Resources window, select the **drawLabelString** resource. This resource was set to a NULL string in **XDrawMaker** to hide the label. The **Motif** specification for the **XmNlabelString** resource states that if this resource is NULL, it is initialized by converting the name of the widget to a compound string. To avoid the appearance of “RulerLabel” as the label gadget’s labelString value and keep the label gadget invisible, set the **drawLabelString** resource to ‘space’ (in the edit window, XFaceMaker will display this as “\ “ - the backslash character followed by a space character). Apply your changes.

5.2.2.3 Modify the draw_edit widget

Select the **draw_edit** widget in the Widget Tree. In the Current Widget Resources window, set the **autoResize** resource to **True**. Apply your changes.

This widget can now be saved as a new class, using the *Modules/Classes/Save As* menu option. Call the new class **Ruler** and the file **Ruler.fm**.

5.2.2.4 Define a rulerPositionColor resource

The new class being prepared, you can now add resources to it. Edit the class using the Active values window by selecting the *Windows/Active Values* menu option.

Set the usage to widget class by clicking on the “Widget class” toggleButton at the top of the window.

Define a new resource named **rulerPositionColor** by typing its name in the textField below the label “New Resource Name”. Give it a type by typing “Pixel” in the **Type:** textField or by choosing one of the options offered when you click on the ... button next to it. Give it a default value by typing “black” in the **Value:** textField or by choosing one of the colors offered when you click on the ... button next to it.

You also need to define the behavior of the widget when this resource is either set to a new value or when the application wants to get its value. This is done by defining two scripts, a **Get** script and a **Set** script. For this resource, no get script is needed since the application will use the resource value directly when it accesses it. However, if the application changes the value of this resource, the drawing should be updated to reflect the new value. This is done by the following set script, to be typed in the Set script text field:

```
XnsLDrawSetForegroundPixel(self, self.RulerPosition, @);  
XnsLDrawRedisplay(self)
```

The first line sets the color according to the resource value @.

The second line refreshes the widget.

Once all the fields (name, type, value, get and set scripts) have been prepared, click on the **Add** button to add this new resource to the widget class. The name of the new resource appears in the “New Resources” list.

Save the class. When you create an instance of this new widget in an application, the list of resources for this widget will now include the new resource named **rulerPositionColor**. You will then be able to edit this resource to change the color of the cursor.

You will now recreate all the other resources defined in the first version of the **Ruler** widget class. You can copy a great deal of what you did the first time but, beware, there are some changes. The resources are:

Table 5.1: Ruler resources

Ressource	Defines
Initialize	the scale creation and modification functions
rulerMin	the minimum value allowed
rulerMax	the maximum value allowed
rulerValue	the current value in between
rulerTicks	the number of main divisions in the scale
rulerSubTicks	the number of subdivisions in the scale
rulerFormat	the format for the labels
valueChangedCallback	the callback called when the RulerPosition moves
rulerFont	the font used to draw the label strings

5.2.2.1 Define an Initialize resource

You must now create the **Initialize** resource, which is a predefined method, so you will find it in the list shown by the **Methods...** button in the **Active Values** window. The **Initialize** active value will be the first to be executed when the widget is instantiated. The **Initialize** active value should also be the first to be read when parsing the **Ruler.fm** file since it will define functions used by other active values. This will avoid unnecessary warnings in the message area of XFaceMaker.

To modify the order of the active values in the active values list, you select one active value and then click the up/down arrows to circulate it up or down in the list. You can modify the order of the active values until the **Initialize** active value is the first, once you are finished with defining the active values, if it is not already the case.

The Initialize set script is the following:

```
type ScaleType = [RULER_]scale, [RULER_]label, [RULER_]both;
struct rulerData{
    Float curpos;
    Float curval;
};
function None RulerCreateScale(ScaleType type)
{
    Float min = self:rulerMin:Float;
    Float max = self:rulerMax:Float;
    Int ticks = self:rulerTicks:Int;
    Int subticks = self:rulerSubTicks:Int;
    /* Get the min and max Y positions in pixels */
    Int a[Int];
    a = XnslDrawFaceGetObjectXPoints(self,
                                     self.rulerSelectZone);

    Float ymin = citof(a[5]);
    Float ymax = citof(a[1]);
    /* Get the min X position */
    Int b[Int];
    b = XnslDrawFaceGetObjectXPoints(self, self.rulerLabel);
    /* Get the min, middle and max X positions */
    Float x1 = citof(b[4]);
    Float x3 = citof(a[0]);
    destroy(a);
    destroy(b);
    Float x2 = x1 + (x3 - x1)/2.0;
    /* Calculate the Y deltas */
    Float dt = (max - min) / citof(ticks);
```

```
Float dy = (ymax - ymin) / citof(ticks);
Float sdy;
if (subticks > 0) sdy = dy / citof(subticks+1);
/* Create the scale and/or labels */
Int do_scale = 0;
Int do_label = 0;
if ( type == RULER_SCALE) do_scale = 1;
else if ( type == RULER_LABEL) do_label = 1;
else {
    do_scale = 1;
    do_label = 1;
}
if(do_label) {
    Float y = 0.0;
    Float t = min;
    String buf = malloc(20);
    for (Int i = 0; i <= ticks; i = i+1) {
        /* Duplicate the label, call it "_label" */
        /* generate the label text and set it */
        Widget lab = XnslDrawDuplicateObject(self,
                                              self.rulerLabel);
        if (!lab) {
            printf("Ruler: can't duplicate label object\n");
            free(buf);
            return();
        }
        sprintf(buf, self:rulerFormat, t);
        XnslDrawTextInsertString(self, lab, buf);
        if (y!=0.0) XnslDrawTranslateObject(self, lab, 0.0, y);
        XnslDrawSetObjectName(self, lab, "_label");
        y = y + dy;
        t = t + dt;
    }
    free(buf);
}
if (do_scale) {
    Float y = 0.0;
    Float tab[Int]; tab = new_array();
    Int index = 0;
    for (Int i = 0; i <= ticks; i = i + 1) {
        /* Create the tick part of the segment array */
        tab[index] = x1; index = index + 1;
        tab[index] = ymin + y; index = index + 1;
    }
}
```

```
    tab[index] = x3; index = index + 1;
    tab[index] = ymin + y; index = index + 1;
    /* Create the sub ticks part of the segment array */
    if (subticks >= 1 && i < ticks) {
        Float sy = y + sdy;
        for (Int j = 1; j <= subticks; j = j+1) {
            tab[index] = x2; index = index + 1;
            tab[index] = ymin + sy; index = index + 1;
            tab[index] = x3; index = index + 1;
            tab[index] = ymin + sy; index = index + 1;
            sy = sy + sdy;
        }
    }
    y = y + dy;
}
/* Create, rename, show the segment */
Widget seg = XnslDrawCreateSegment(self, data(tab),
                                   size(tab));

if (!seg) {
    printf ("Ruler: can't create segment object \n");
    destroy(tab);
    return();
}
XnslDrawSetObjectName (self, seg, "_scale");
destroy(tab);
XnslDrawShowObject(self, seg);
}
}
function None RulerRecomputeScale(ScaleType type)
{
    if (@2 && @ != @1) {
        Int do_scale = 0;
        Int do_label = 0;
        if (type == RULER_SCALE) do_scale = 1;
        else if (type == RULER_LABEL) do_label = 1;
        else {
            do_scale = 1;
            do_label = 1;
        }
        if (do_label) {
            /* Delete every widget named "_label"*/
            Widget wid;
            for (wid = XnslDrawNameToObject(self, "_label");
```



```
        wid !=0; wid = XnslDrawNameToObject(self,"_label")) {
            XnslDrawHideObject (self, wid);
            XtDestroyWidget(wid);
        }
    }
    if (do_scale) {
        /* Delete the scale widget named "_scale" */
        Widget wid;
        wid = XnslDrawNameToObject(self, "_scale");
        if (!wid) {
            printf ("Ruler: Can't find segment object \n");
            return();
        }
        XnslDrawHideObject(self,wid);
        XtDestroyWidget(wid);
    }
    /* Create the scale */
    RulerCreateScale(type);
}
}
RulerCreateScale(RULER_BOTH);
XnslDrawRedisplay(self);
```

The script starts with the definition of the new type `ScaleType`:

```
type ScaleType = [RULER_]scale, [RULER_]label, [RULER_]both;
```

This type has a common part shown between brackets and differing ends.

It is used in the **RulerCreateScale** and **RulerRecomputeScale** functions.

The script then defines the `rulerData` structure.

The rest of the script consists in the definition of the **RulerCreateScale** function, the definition of the **RulerRecomputeScale** function and finally the call to the **RulerCreateScale** function to initialize the widget.

The **RulerCreateScale** function uses the **RulerLabel** and **RulerSelectZone** widgets to choose the ticks' positions, the **rulerMin**, **rulerMax**, **rulerTicks** and **rulerSub-Ticks** resources to do the calculations and set the labels' texts. The beginning of the function prepares all these parameters.

Then, if necessary, the label part is created by duplicating the label left in the basic objects of the drawing, giving it a text and a name and then placing it in the correct position. The purpose of giving the same name “_label” to all the labels is to be able to delete them without having to manage a table of names in the application.

Afterwards, if necessary, the scale part is created as a segment. A segment is created with an array of (x1,y1,x2,y2) coordinates quadruples giving the beginning and ending positions of each line segment. Creating the scale consists of creating this coordinates array and then creating the segment widget with the “_scale” name. Since the widgets created this way are not automatically managed, a call to **XnsIDDrawShowObject** is included after the creation to show the segment. Creating the scale as a segment is recommended since it reduces the number of Draw objects in the drawing, thereby speeding up the calculation and redrawing processes.

The **RulerRecomputeScale** function destroys the widgets corresponding to the type parameter (labels, scale or both) and then calls the **RulerCreateScale** function to recreate the same widgets with the current resource values. It starts with the line :

```
if (@2 && @ != @1)
```

This introduces the notions of current value and previous value. The **@** symbol refers to the current value (new value) of the active value where the call to the **RulerRecomputeScale** function takes place. The **@1** symbol refers to the previous value of the active value. The **@2** symbol refers to the widget ID. Its value is 0 when the script is called from the Initialize active value, since the widget does not yet exist.

This test line at the beginning of the **RulerRecomputeScale** function can be read as: if this function is not called during initialization and if the current value (or new value) of the calling active value is different from the previous one, then the rest of the function can be executed.

5.2.2.2 Define a rulerMin resource

Define a new resource named **rulerMin** set its **Type** to Float and its **Default Value** to -50.00 and then edit its set script. The set script needs to change the labels of the widget to reflect the new value. Since this operation has already been defined in the **Initialize** resource, you only need to call the **RulerRecomputeScale** function with **RULER_LABEL** as parameter. Here is the source of the set script, which is self explanatory:

```
RulerRecomputeScale(RULER_LABEL);
```

No get script is necessary for the **rulerMin** resource since no operation needs be done to retrieve the value of this resource.

5.2.2.3 Define a rulerMax resource

Define a new resource named **rulerMax** with **Type:** Float and Default **Value:** 50.00 and then edit its set script. The set script needs to change the labels of the widget to reflect the new value. Since this operation has already been defined in the **Initialize** resource, you only need to call the **RulerRecomputeScale** function with **RULER_LABEL** as parameter. Here is the source of the set script:

```
RulerRecomputeScale(RULER_LABEL);
```

No get script is necessary for the **rulerMax** resource since no operation needs be done to retrieve the value of this resource.

5.2.2.4 Define a rulerValue resource

The **rulerValue** resource is more complicated: you have to connect the position of the cursor and the value of this new resource.

Define a new resource named **rulerValue** with **Type:** Float and Default **Value:** 0.00 and then edit its scripts. The set script must take the resource value and move the ruler accordingly. Here is the source of the set script:

```
/* Retrieve the min and max Y positions */
Int a[Int];
a = XnsIDrawFaceGetObjectXPoints(self,
                                self.RulerSelectZone);

Float ymin = citof(a[5]);
Float ymax = citof(a[1]);
destroy(a);

/* Retrieve the position of the cursor */
Float y = XnsIDrawGetCenterY(self, self.RulerPosition);
/* Get the resource value contained in the @ variable */
Float v = Float(@);

/* Check if the value is valid */
Float min = Float(self:rulerMin);
Float max = Float(self:rulerMax);
if (v > max) v = max;
if (v < min) v = min;

/* Compute a translation to reflect the new resource value */
Float ty = (v - min) * (ymax - ymin) / (max - min);
XnsIDrawSetTranslation(self, self.RulerPosition, 0.0, ty);

/* Redisplay the widget */
```

```
XnslDrawRedisplay(self);
```

The get script must compute the value of the resource when the ruler position changes. The get script can be:

```
/* Retrieve the min and max Y positions */
Int a[Int];
a = XnslDrawFaceGetObjectXPoints(self,
                                   self.RulerSelectZone);

Float ymin = citof(a[5]);
Float ymax = citof(a[1]);
destroy(a);
/* Retrieve the translation defined for the cursor */
Float y = XnslDrawGetTranslationY(self,
                                   self.RulerPosition) + ymin;
/* Compute a value corresponding to the position of the
                                   ruler */
Float min = Float(self:rulerMin);
Float max = Float(self:rulerMax);
Float v = min + (y - ymin) * (max - min) / (ymax - ymin);
/* Set the resource value */
@ = v;
```

5.2.2.5 Define a rulerTicks resource

Define a new resource named **rulerTicks** with **Type:** Int and Default **Value:** 10 and then edit its set script. The set script needs to check the value of the resource and then to change the labels and the ticks of the scale to reflect this new value. Since this operation has already been defined in the **Initialize** resource, you only need to call the **RulerRecomputeScale** function with **RULER_BOTH** as parameter. Here is the source of the set script:

```
if (!@ || @ < 2) @ = 2;
RulerRecomputeScale(RULER_BOTH);
```

No get script is necessary for the **rulerTicks** resource since no operation needs be done to retrieve the value of this resource.

5.2.2.6 Define a rulerSubTicks resource

Define a new resource named **rulerSubTicks** with **Type:** Int and Default **Value:** 5 and then edit its set script. The set script needs to check the value of the resource and then to change the ticks of the scale to reflect this new value. Since this operation has

already been defined in the **Initialize** resource, you only need to call the **RulerRecomputeScale** function with **RULER_SCALE** as parameter. Here is the source of the set script:

```
if (!@ || @ < 1) @ = 1;  
RulerRecomputeScale(RULER_SCALE);
```

No get script is necessary for the **rulerSubTicks** resource since no operation needs be done to retrieve the value of this resource.

5.2.2.7 Define a rulerFormat resource

Define a new resource named **rulerFormat** with **Type:** String and **Value:** %7.2f and then edit its set script. The set script needs to check the value of the resource and then to change the ticks of the scale to reflect this new value. Since this operation has already been defined in the **Initialize** resource, you only need to call the **RulerRecomputeScale** function with **RULER_LABEL** as parameter. Here is the source of the set script:

```
if (!@) @ = "%7.2f";  
RulerRecomputeScale(RULER_LABEL);
```

No get script is necessary for the **rulerFormat** resource since no operation needs be done to retrieve the value of this resource.

5.2.2.8 Define a valueChangedCallback resource

Widgets usually have Callbacks. You can define a callback resource for your new widget class. Define a resource named **valueChangedCallback** with **type:** Callback. No set or get script is necessary.

This callback is triggered in the **drawActivateCallbackList** of the **RulerSelect-Zone** widget.

5.2.2.9 Define a rulerFont resource

Define a new resource named **rulerFont** with **Type:** DrawFont and Default **Value:** fixed and then edit its set script. The set script needs find all the DrawLabel gadgets and change their drawFont resource. Here is the source of the set script:

```
Int nchildren = self:numChildren;
```

```
Widget child[Int];
child = new_c_array(Pointer(self:children),nchildren);
for (i = 0; i < nchildren; i = i+1) {
    w = child[i];
    if (XnsIsDrawLabel(w)) w:drawFont:DrawFont = @;
}
```

No get script is necessary for the **rulerFont** resource since no operation needs be done to retrieve the value of this resource.

Now that all the resources are defined, save the class. In the Active Values box, click the Save button. This saves the Ruler.fm file and redefines the class for XFaceMaker.

5.2.3 Test the Ruler Class within XFaceMaker

Before generating the C code for the class, you should test that the class works as expected. To do so, create an ApplicationShell, a BulletinBoard, place an instance of the Ruler class in it. Make it big enough to see the effect of changing each of the resources. Verify that, as you interact with the resources, XFaceMaker does not output error messages and the fields are updated as expected. We are not yet ready to test the class outside of XFaceMaker.

5.2.4 Make the Class Library and Includes

We want to make our class library with stand alone C code, in order to be able to use it without linking with the Fm_c library, e.g., in interpreted applications.

5.2.4.1 Generate the Ruler class C file

Edit the **Ruler** widget class with the *Modules/Classes/Edit* menu option. The draw_edit XnsI Draw widget should be selected.

Open C Code options window with the *Options/C Code* menu option. Check the *Save C Code* option, check the Save As **class** radio button, specify the class prefix as **Ewf**, check the **stand-alone C** radio button, click Apply.

Select the *Modules/Classes/Save As* menu option and save the class named **Ruler** in the **Ruler.fm** and **Ruler.c** files. This creates the Ruler.c file with stand alone C code, and the Ruler.h and RulerP.h files in the current directory.

Create directory Ewf in the current directory, and move the two .h files just created to Ewf. If you have a look at the Ruler.c file, you will see that it has an include directive that is `#include <Ewf/RulerP.h>`, i.e. our Ruler.c file follows the class prefix convention, as expected.

5.2.4.2 Make the Ruler class library

Now, we have to compile the Ruler.c file and build the libRuler.a library. The set of commands will look like the following, depending on the operating system you are using.

```
cc -c Ruler.c -I. -I$NSLHOME/include
ar cr libRuler.a Ruler.o
ranlib libRuler.a
```

Again, class prefix conventions will lead us to tell XFaceMaker to fetch this library in the Ewf directory so we have to move our newly created library, libRuler.a, to the Ewf directory.

We can do away, now, with Ruler.o

5.2.5 Make an Extended XFaceMaker

We first define the Ruler.face extension file. The file is similar to the one we did in “Define the Ruler.face Extension File” on page 74. It provides the build information for code generation of applications using the class, and the initialization information for an extended XFaceMaker.

It also has to define the rulerData type and the rulerData structure - remember, in the previous extension, the rulerData structure was defined in the ruler.draw file which was part of the class: the Face interpreter knew the structure because the ruler.draw file was included in the extended XFaceMaker. Now, the rulerData structure can be (and must be) declared in the Ruler.face file. As for the rulerData type definition, it is necessary, in order to tell XFaceMaker how to generate the C code for function calls in which the rulerData structure pointer is passed as a parameter.

In addition to the build information, then, the Ruler.face file contains:

- the structure definition to make rulerData known to the Face interpreter

```
struct rulerData “rulerData*” {
    Float curpos;
    Float curval;
};
```

- the rulerData type definition:

```
type rulerData “rulerData *”;
```

The rulerData structure declaration has to be added in our Ruler.h file so we can use the structure in C code files without having to redeclare the structure. Thus, in the Ruler.h file, add the following:

```
typedef struct {  
    float curpos;  
    float curval;  
}rulerData ;
```

When the Ruler.h file and the Ruler.face file are correct, create an extended XFaceMaker with the class, as explained in “Use the Ruler Extension in XFaceMaker” on page 77.

5.2.6 Test the Extended XFaceMaker

Launch your new XFaceMaker executable. Build a small interface with an instance of EwfRuler. Check that you are able to edit all the resources. Verify that you are able to use the rulerData structure in the valueChangedCallback. You might just print the values, as in the example below:

```
rulerData cb;  
cb = $;  
printf("position is %3.1f value is %3.1f\n",cb->curpos,  
                                              cb->curval);
```

Switch to Try mode, verify that the values printed seem correct. You should also make sure that you are able to do the same in an application function. Add the following in your valueChangedCallback script:

```
function None foo(rulerData cd);  
foo($);
```

Save the .fm file as simple.fm; generate the application files (**File/SaveAppliFiles/All**). Edit the simpleApp.c file to include the Ewf/Ruler.h file, and specify the body of function foo to be the printf command. Your simpleApp.c file content will be:

```
* C file generated by XFaceMaker Version 3.5.  
* Do not remove this comment.  
* Timestamp:          908296963  
*/  
/*  
* XFaceMaker 3.5 Application Functions Definitions.  
*/  
  
#ifndef STANDALONE  
#include <Fm.h>  
#endif  
#include <Ewf/Ruler.h>  
#if NeedFunctionPrototypes  
void foo(rulerData* cd)
```



```
#else
void foo(cd)
rulerData* cd;
#endif
{
    printf("in appli pos %3.1f val %3.1f\n", cd->curpos,
                                                cd->curval);
}
```

Build the compiled and interpreted applications, using the barMake makefile:

```
make -f simpleMake
```

launch **csimple** and **isimple**. Verify that the position and value printed by the interface and the application are the same for both the compiled and the interpreted applications.

You might also want to test that you are able to use the interface.fm interface that we built earlier. Note that the class name is no longer Ruler but EwfRuler now that we have specified a class prefix. You will have to edit your interface.fm file to change the class name for the ruler instance.

5.2.7 Source listings

5.2.7.1 Ruler Xt class file : Ruler.fm

In the XFaceMaker files, you will see that the lines in the callback scripts are followed by the \ character to indicate that the callback continues on the next lines.

```
widget XnslDraw {
    flag {
    }
    activevalue {
        name = Initialize
        set = type ScaleType = [RULER_]scale, [RULER_]label,
                                                [RULER_]both;\
    }
    struct rulerData {
        Float curpos;
        Float curval;\
    };
    function None RulerCreateScale(ScaleType type)\
    {
        Float min = self:rulerMin:Float;
        Float max = self:rulerMax:Float;
        Int ticks = self:rulerTicks:Int;
        Int subticks = self:rulerSubTicks:Int;
```

```
/* Get the min and max Y positions in pixels */
Int a[Int];
a = XnslDrawFaceGetObjectXPoints(self,
                                   self.RulerSelectZone);

Float ymin = citof(a[5]);
Float ymax = citof(a[1]);
/* Get the min X position */
Int b[Int];
b = XnslDrawFaceGetObjectXPoints(self, self.RulerLabel);
/* Get the min, middle and max X positions */
Float x1 = citof(b[4]);
Float x3 = citof(a[0]);
destroy(a);
destroy(b);
Float x2 = x1 + (x3 - x1)/2.0;
/* Calculate the Y deltas */
Float dt = (max - min) / citof(ticks);
Float dy = (ymax - ymin) / citof(ticks);
Float sdy;
if (subticks > 0) sdy = dy / citof(subticks+1);
/* Create the scale and/or labels */
Int do_scale = 0;
Int do_label = 0;
if ( type == RULER_SCALE) do_scale = 1;
else if ( type == RULER_LABEL) do_label = 1;
else {
    do_scale = 1;
    do_label = 1;
}
if(do_label) {
    Float y = 0.0;
    Float t = min;
    String buf = malloc(20);
    for (Int i = 0; i <= ticks; i = i+1) {
/* Duplicate the label, call it "_label" */
/* generate the label text and set it */
        Widget lab = XnslDrawDuplicateObject(self,
                                                self.RulerLabel);

        if (!lab) {
            printf("Ruler: can't duplicate label object\n");
            free(buf);
            return();
        }
    }
}
```

```
        sprintf(buf, self:rulerFormat, t);
        XnslDrawTextInsertString(self, lab, buf);
        if (y!=0.0) XnslDrawTranslateObject(self, lab, 0.0,
                                                    y);

        XnslDrawSetObjectName(self, lab, "_label");
        y = y + dy;
        t = t + dt;
    }
    free(buf);
}
if (do_scale) {
    Float y = 0.0;
    Float tab[Int]; tab = new_array();
    Int index = 0;
    for (Int i = 0; i <= ticks; i = i + 1) {
/* Create the tick part of the segment array */
        tab[index] = x1; index = index + 1;
        tab[index] = ymin + y; index = index + 1;
        tab[index] = x3; index = index + 1;
        tab[index] = ymin + y; index = index + 1;
/* Create the sub ticks part of the segment array */
        if (subticks >= 1 && i < ticks) {
            Float sy = y + sdy;
            for (Int j = 1; j <= subticks; j = j+1) {
                tab[index] = x2; index = index + 1;
                tab[index] = ymin + sy; index = index + 1;
                tab[index] = x3; index = index + 1;
                tab[index] = ymin + sy; index = index + 1;
                sy = sy + sdy;
            }
        }
        y = y + dy;
    }
/* Create, rename, show the segment */
    Widget seg = XnslDrawCreateSegment(self, data(tab),
                                        size(tab));

    if (!seg) {
        printf ("Ruler: can't create segment object \n");
        destroy(tab);
        return();
    }
    XnslDrawSetObjectName (self, seg, "_scale");
    destroy(tab);
}
```

```
        XnslDrawShowObject(self, seg);
    }\
}\
function None RulerRecomputeScale(ScaleType type)\
{
    if (@2 && @ != @1) {
        Int do_scale = 0;
        Int do_label = 0;
        if (type == RULER_SCALE) do_scale = 1;
        else if (type == RULER_LABEL) do_label = 1;
        else {
            do_scale = 1;
            do_label = 1;
        }
        if (do_label) {
/* Delete every widget named "_label" */
            Widget wid;
            for (wid = XnslDrawNameToObject(self, "_label");
                wid!=0; wid=XnslDrawNameToObject(self, "_label")) {
                XnslDrawHideObject (self, wid);
                XtDestroyWidget(wid);
            }
        }
        if (do_scale) {
/* Delete the scale widget named "_scale" */
            Widget wid;
            wid = XnslDrawNameToObject(self, "_scale");
            if (!wid) {
                printf ("Ruler: Can't find segment object \n");
                return();
            }
            XnslDrawHideObject(self,wid);
            XtDestroyWidget(wid);
        }
/* Create the scale */
        RulerCreateScale(type);
    }\
}\
RulerCreateScale(RULER_BOTH);\
XnslDrawRedisplay(self);
    storage = object
    automatic = true
    usage = class
```

```
    }
    activevalue {
        name = rulerPositionColor
        set =
XnslDrawSetForegroundPixel(self,self.RulerPosition, @);\
    XnslDrawRedisplay(self);
        type = Pixel
        storage = object
        automatic = true
        usage = class
        default_type = String
        default_value = black
    }
    activevalue {
        name = rulerMin
        set = RulerRecomputeScale(RULER_LABEL);
        type = Float
        storage = object
        automatic = true
        usage = class
        default_type = String
        default_value = -50.00
    }
    activevalue {
        name = rulerMax
        set = RulerRecomputeScale(RULER_LABEL);
        type = Float
        storage = object
        automatic = true
        usage = class
        default_type = String
        default_value = 50.00
    }
    activevalue {
        name = rulerValue
        get = /*Retrieve the min and max Y positions */ \
Int a[Int];\
        a =
XnslDrawFaceGetObjectXPoints(self,self.RulerSelectZone);\
        Float ymin = citof(a[5]);\
        Float ymax = citof(a[1]);\
        destroy(a);\
    /* Retrieve the cursor's current Y value */\
}
```

```
Float y = XnslDrawGetTranslationY(self,
                                   self.RulerPosition) + ymin;\n
/* Compute the value */\n
Float min = Float(self:rulerMin);\n
Float max = Float(self:rulerMax);\n
Float v = min + (y - ymin) * (max - min) / (ymax - ymin);\n
/* Copy it to the resource value */\n
@ = v;\n
    set = /*Retrieve the min and max Y positions */\n
Int a[Int];\n
a = XnslDrawFaceGetObjectXPoints(self,\n                                   self.RulerSelectZone);\n
Float ymin = citof(a[5]);\n
Float ymax = citof(a[1]);\n
destroy(a);\n
/* Retrieve the cursor's current Y value */\n
Float y = XnslDrawGetCenterY(self,self.RulerPosition);\n
/* Get the resource value contained in the @ variable */\n
Float v = Float(@);\n
/* Check it is valid */\n
Float min = Float(self:rulerMin);\n
Float max = Float(self:rulerMax);\n
if (v > max) v = max;\n
if (v < min) v = min;\n
/* compute the translation for the new position */\n
Float ty = (v - min) * (ymax - ymin) / (max - min);\n
XnslDrawSetTranslation(self,self.RulerPosition, 0.0,\n                       ty);\n
/* Redisplay the widget */\n
XnslDrawRedisplay(self);\n
    type = Float\n
    storage = object\n
    automatic = true\n
    usage = class\n
    default_type = String\n
    default_value = 0.00\n
}\n
    activevalue {\n
        name = rulerTicks\n
        set = if (!@ || @ < 2) @ = 2;\n
RulerRecomputeScale(RULER_BOTH);\n
        type = Int\n
        storage = object
```

```
        automatic = true
        usage = class
        default_type = String
        default_value = 10
    }
    activevalue {
        name = rulerSubTicks
        set = if (!@ || @ < 2) @ = 1;\
RulerRecomputeScale(RULER_BOTH);
        type = Int
        storage = object
        automatic = true
        usage = class
        default_type = String
        default_value = 5
    }
    activevalue {
        name = rulerFormat
        set = if (!@ ) @ = "%7.2f";\
RulerRecomputeScale(RULER_LABEL);
        type = String
        storage = object
        automatic = true
        usage = class
        default_type = String
        default_value = %7.2f
    }
    activevalue {
        name = valueChangedCallback
        type = Callback
        storage = object
        automatic = true
        usage = class
    }
    activevalue {
        name = rulerFont
        set = Int nchildren = self:numChildren;\
Widget child[Int];\
child = new_c_array(Pointer(self:children),nchildren);\
for (i = 0; i < nchildren; i = i+1) {
    w = child[i];
    if (XnslIsDrawLabel(w)) w:drawFont:DrawFont = @;\
}
```

```
        type = DrawFont
        storage = object
        automatic = true
        usage = class
        default_type = String
        default_value = fixed
    }
    createproc {
        XnslDrawFakeLoad(self);
    }
    resources {
        width = 280
        height = 530
        drawUnits = no_units
        drawAngleUnits = no_angle_units
        drawGridOneOf = 1
        drawGridSnapTo = True
        x = 0
        y = 0
        autoResize = true
    }
    children {
        widget XnslDrawRectangle {
            flag {
                mapped
                autosize
                autopos
                gadget
            }
            resources {
                drawShowBorder = True
                drawPoints = 60, 40, 200, 470
                drawNumPoints = 2
            }
        } RulerBackground
        widget XnslDrawRectangle {
            flag {
                mapped
                autosize
                autopos
                gadget
            }
            resources {
```



```
        drawShowBorder = True
        drawEventMask = Button1Down|Button1Motion
        drawActivateCallbackList = on Button1Down
script MoveCursor:
/* Get the positions of the Min and Max divisions */
Int a[Int];
a = XnslDrawFaceGetObjectXPoints(parent, self);
Float ymin = citof(a[5]);
Float ymax = citof(a[1]);
destroy(a);
/* Get the Y position of the mouse */
Float y = citof( XnslDrawEventY (parent, $));
/* Test if the click was within limits */
if (y < ymax) y = ymax;
if (y > ymin) y = ymin;
/* compute a translation to move the cursor */
Float ty = y - ymin;
XnslDrawSetTranslation (parent, parent.RulerPosition, 0.0,
                        ty);

/* Create and initialize the rulerData structure */
rulerData rd = new_struct(rulerData);
rd -> curpos = y;
rd -> curval = parent:rulerValue;
/* Call valueChangedCallback call_data = the structure */
XtCallCallbacks(parent, "valueChangedCallback", rd);
Free (String(rd));\

/* Show the changes */
XnslDrawRedisplay (parent);\
end\
on Button1Motion
script MoveCursor;\
end

        drawPoints = 150, 80, 90, 400
        drawNumPoints = 2
        drawSelected = True
    }
} RulerSelectZone
widget XnslDrawLabel {
    flag {
        mapped
        autosize
        autopos
```

```

        gadget
    }
    resources {
        drawShowBorder = False
        drawSelectCondition = draw_inside draw_on_border
        drawPoints = 60, 470, 60, 20
        drawNumPoints = 2
        drawLabelAlignment = draw_label_align_east
        drawLabelTransformString = False
        drawLabelString = \
    }
} RulerLabel
widget XnslDrawLine {
    flag {
        mapped
        autosize
        autopos
        gadget
    }
    resources {
        drawShowBorder = True
        drawPoints = 120, 480, 150, 480
        drawNumPoints = 2
        drawYTranslation = -1.000000
        drawArrowDir = draw_from_arrow
    }
} RulerPosition
}
} draw_edit

```

5.2.7.2 Ruler.face file

```
include "XnslDraw.face";

/*****
 * Class information.
 *****/

define _XFM_EXT_CODE_Ruler '
(*fns->add_widget_class)(
&ewfRulerWidgetClass,
0,
FM_PRIMITIVE,
FM_NORMAL,
0,
"#include <Ewf/Ruler.h>",
"ewfRulerWidgetClass",
0,
0,
0,
"Ruler.bm");';

/*****
 * Build information.
 *****/

define _XFM_EXT_C_LIBRARY_Ruler '';
define _XFM_EXT_E_LIBRARY_Ruler 'RulerFm';

define _XFM_EXT_BUILD_C_INIT_FUNCALL_Ruler
    'XtInitializeWidgetClass(ewfRulerWidgetClass);';
define _XFM_EXT_BUILD_INIT_FUNCTION_Ruler 'ERulerInit';
define _XFM_EXT_BUILD_INIT_FUNCALL_Ruler 'ERulerInit();';

define _XFM_EXT_BUILD_INCLUDES_Ruler
    '#include <Ewf/Ruler.h>';
define _XFM_EXT_BUILD_LIBRARIES_Ruler '-lRuler';
define _XFM_EXT_BUILD_C_LIBRARIES_Ruler '';
define _XFM_EXT_BUILD_I_LIBRARIES_Ruler '';
define _XFM_EXT_BUILD_E_LIBRARIES_Ruler '-lRulerFm';
define _XFM_EXT_BUILD_INCLUDE_PATH_Ruler
    '-I. -I$(NSLHOME)/include';
define _XFM_EXT_BUILD_LIBRARY_PATH'
    -L./Ewf -L$(NSLHOME)/lib';
```

```
struct rulerData "rulerData*" {  
    Float curpos;  
    Float curval;  
};  
  
type rulerData "rulerData *";
```